

PROMPT PARA AGENTE ESPECIALISTA EN DOCUMENTACIÓN TÉCNICA Y DESARROLLO FULL STACK

CONTEXTO

Voy a proporcionarte un documento que contiene una guía completa elaborada por un docente para desarrollar una aplicación web utilizando:

- Node.js
- Express
- MySQL
- Angular

La guía original es profesional y técnicamente correcta, pero está diseñada de forma general para enseñar múltiples conceptos y escenarios.

Mi objetivo NO es crear otra guía académica.

Mi objetivo es crear una guía extremadamente práctica, simplificada y orientada específicamente al parcial.

Durante el parcial únicamente se trabajará con:

- Una sola tabla o entidad.
- Un backend Node.js + Express.
- Una base de datos MySQL.
- Un frontend Angular.
- Operaciones CRUD básicas.
- Conexión completa entre frontend y backend.

La tabla o modelo exacto será proporcionado posteriormente por el docente.

La nueva guía debe permitir que cualquier estudiante, incluso con conocimientos limitados, pueda completar exitosamente el parcial siguiendo instrucciones paso a paso.

TU ROL

Actúa simultáneamente como:

- Arquitecto de Software Senior.
- Desarrollador Full Stack Senior.
- Instructor Universitario.
- Especialista en Node.js.
- Especialista en Angular.
- Especialista en MySQL.
- Especialista en documentación técnica.
- Diseñador de experiencias de aprendizaje.

Debes detectar automáticamente:

- Pasos faltantes.
- Dependencias omitidas.
- Configuraciones implícitas.
- Errores comunes.
- Problemas frecuentes de instalación.
- Suposiciones realizadas por el docente.
- Requisitos que un principiante podría no entender.

Nunca asumas que el lector sabe algo.

OBJETIVO PRINCIPAL

Transformar la guía original en una guía operativa optimizada para el parcial.

La nueva guía debe:

- Ser secuencial.
 - Ser precisa.
 - Ser corta cuando sea posible.
 - Ser detallada cuando sea necesario.
 - Evitar teoría innecesaria.
 - Centrarse únicamente en lo que permite aprobar el parcial.
-

FILOSOFÍA DE LA NUEVA GUÍA

Cada paso debe responder:

1. Qué hacer.

2. Por qué se hace.
 3. Comando exacto.
 4. Resultado esperado.
 5. Cómo verificar que funcionó.
 6. Errores comunes.
 7. Solución rápida.
-

ESTRUCTURA OBLIGATORIA

Genera la guía organizada exactamente así:

FASE 0 - PREPARACIÓN DEL ENTORNO

Qué instalar

- Node.js
- Angular CLI
- MySQL
- VS Code
- Extensiones recomendadas

Verificación

Mostrar comandos:

```
node -v  
npm -v  
ng version  
mysql --version
```

Indicar exactamente qué debe aparecer.

FASE 1 - CREACIÓN DEL BACKEND

Crear proyecto

Comandos exactos.

Instalar dependencias

Explicar para qué sirve cada una.

Estructura de carpetas

Mostrar árbol completo.

Configuración inicial

Archivos requeridos.

Variables de entorno

Configuración completa.

FASE 2 - CREACIÓN DE LA BASE DE DATOS

Crear base de datos

Script SQL completo.

Crear tabla principal

Plantilla adaptable.

Verificación

Consultas SQL para comprobar funcionamiento.

FASE 3 - MODELO

Explicar:

- Qué es.
- Cómo se crea.
- Qué partes cambian según la tabla.

Generar plantilla reutilizable.

FASE 4 - CONTROLADOR

Explicar:

- Crear
- Obtener
- Obtener por ID
- Actualizar
- Eliminar

Mostrar ejemplos completos.

FASE 5 - RUTAS

Explicar:

- Definición
 - Registro
 - Verificación
-

FASE 6 - PRUEBAS CON POSTMAN

Para cada endpoint:

- Método
 - URL
 - Body
 - Respuesta esperada
-

FASE 7 - CREACIÓN DEL FRONTEND ANGULAR

Crear proyecto

Estructura mínima

Componentes necesarios

Servicio HTTP

Configuración del entorno

Conexión con backend

FASE 8 - CRUD COMPLETO EN ANGULAR

Crear:

- Listar
- Crear
- Editar
- Eliminar

Mostrar flujo completo.

FASE 9 - CONEXIÓN TOTAL

Explicar:

Frontend → API → Base de Datos

Incluir diagrama textual.

FASE 10 - CHECKLIST DEL PARCIAL

Lista verificable:

- ☐ Backend inicia.
- ☐ Angular inicia.
- ☐ Base de datos conecta.
- ☐ Crear funciona.
- ☐ Listar funciona.
- ☐ Editar funciona.
- ☐ Eliminar funciona.
- ☐ No existen errores en consola.
- ☐ API responde correctamente.
- ☐ Proyecto listo para entregar.

REGLAS IMPORTANTES

1. Eliminar teoría innecesaria.
2. Mantener únicamente lo requerido para el parcial.
3. Detectar pasos faltantes automáticamente.
4. Corregir errores conceptuales encontrados en la guía original.
5. Completar configuraciones omitidas.
6. Explicar como si el lector tuviera conocimientos mínimos.
7. Priorizar la ejecución práctica sobre la teoría.
8. Convertir fragmentos complejos en instrucciones simples.
9. Crear plantillas reutilizables donde la única modificación sea el nombre de la tabla o entidad.
10. Indicar explícitamente qué partes deben reemplazarse cuando el docente entregue el modelo del parcial.

SEGUNDA TAREA

Después de generar la guía optimizada:

Diseña una aplicación web en Flask para desplegar esta guía.

La aplicación debe incluir:

- Interfaz moderna y profesional.
- Navegación por fases.
- Barra lateral.
- Buscador.
- Modo oscuro.
- Sistema de pasos expandibles.
- Copia rápida de comandos.
- Resaltado de código.
- Checklist interactivo.
- Generación automática de código según el nombre de la entidad ingresada por el usuario.

Genera:

- Arquitectura completa.
- Estructura de carpetas.
- Base de datos necesaria.
- Backend Flask.
- Frontend HTML/CSS/JS.
- Diseño UI/UX.
- Plan de implementación.
- Código necesario.

Antes de construir cualquier sección, analiza completamente el documento que te proporcionaré y detecta vacíos, inconsistencias y mejoras.

También integra faker para que se generen los datos en la tabla o db eso integrado en la guía en el lugar correspondiente según la secuencia **y las validaciones en los campos**

Introducción: NodeJs

Es un programa escrito en C++ con el interprete de Google Chrome V8 extendido. Es decir, que utiliza un interprete de Javascript y lo extiende para agregarle bastantes características propias de un servidor.

Nodejs es un servidor servidor, permite crear servicios, servicios que son consumidos por uno o varios clientes (navegadores web por ejemplo). Esto es lo que llamamos **Modelo Cliente-Servidor**

Requisitos Previos

- Install Windows Subsystem for Linux
- Set up Visual Studio Code
- Node Js

Instalar una versión específica de Node.js usando NVM

Otra forma de instalar Node.js en Ubuntu es hacer uso del **Administrador de versiones de nodos (NVM)**, un **script bash** que se utiliza para administrar múltiples versiones de Node.js.

Instalar Node.js utilizando el método NVM es ideal si desea utilizar una versión específica. Para comenzar, descargue e instale NVM usando **wget**:

1. Abra la consola de Ubuntu y utilice el siguiente comando:

```
sudo apt-get install wget
```

1. Para instalar una versión diferente, reemplace **v0.39.0** por otro valor o use el siguiente comando para descargar la versión más reciente:

```
wget -qO- https://raw.githubusercontent.com/creationix/nvm/v0.39.0/install.sh | bash
```

1. Permita que el script NVM se ejecute desde el perfil bash de su usuario:

```
source ~/.profile
```

Una vez instalado NVM,

Instalar NVM en su Sistema

1. Compruebe todas las versiones disponibles de Node.js mediante NVM ejecutando el siguiente comando:

```
nvm ls-remote
```

1. Si bien puede instalar cualquier versión Node.js, le recomendamos que utilice la versión más reciente para garantizar su compatibilidad y confiabilidad. Para este ejemplo, instalaremos la última versión de LTS en el momento de escribir este artículo:

```
nvm install 22.15.0
```

1. Compruebe si se ha instalado correctamente consultando el número de versión actualmente activo:

```
node -v
```

Consideraciones antes de de Crear un Proyecto

Dentro del terminal de ubuntu, crear una carpeta donde pretende alojar el proyecto.

Paso 1:

`mkdir sitealmacen` y luego

```
sudo chmod -R 777 sitealmacen/  
cd sitealmacen
```

Una vez dentro de la carpeta ingresar `code .` para entrar a visual studio code y desde allí trabajar mejor el proyecto.

Una vez dentro de Visual Studio Code, ingresar a una nueva terminal (Menu de Opciones > Terminal > Nueva Terminal)

Ahora si iniciemos los pasos:

Paso 0:

```
npm init -y  
# Dependencias de desarrollo  
npm install -D typescript @types/node nodemon ts-node  
npm install -D @types/express @types/morgan @types/cors  
  
# Crear archivo de configuración TypeScript  
npx tsc --init
```

Nota: Antes de continuar, hacemos conexión con VS CODE en el entorno para ver los archivos generados

En el archivo `tsconfig.json`, cambiar

```
{
  "compilerOptions": {
    "target": "es2016",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "verbatimModuleSyntax": false,
    "skipLibCheck": true
  }
}
```

En el archivo `package.json`, cambiar

```
{
  "scripts": {
    "build": "tsc",
    "dev": "nodemon src/server.ts --exec ts-node"
  },
  "type": "commonjs"
}
```

Paso 2:

2.1 Instalación de Dependencias Core

```
npm install express cors morgan dotenv
```

2.2 Crear estructura de directorios

```
src/  
├─ config/  
├─ controllers/  
│   └─ authorization/  
├─ database/  
├─ faker/  
├─ http/  
│   └─ authorization/  
├─ middleware/  
├─ models/  
│   └─ authorization/  
└─ routes/  
    └─ authorization/
```

Desglose detallado:

src/

- **config/** - Configuración principal de la aplicación

- **controllers/** - Controladores para manejar la lógica de negocio
- **database/** - Configuración y conexión a base de datos
- **faker/** - Scripts para generar datos de prueba
- **http/** - Archivos .http para testing de endpoints
- **middleware/** - Middleware personalizado (autenticación, etc.)
- **models/** - Modelos de Sequelize
 - **authorization/** - Modelos específicos del sistema de autorización
- **routes/** - Definición de rutas de la API

Paso 3: Configuración del Servidor

Crear el archivo `src/server.ts` y `src/config/index.ts`

Crear el archivo `src/server.ts`

```
import { App } from './config/index';

async function main() {
  const app = new App();
  await app.listen();
}

main();
```

Luego, crear el archivo `src/config/index.ts`

```
import dotenv from "dotenv";
import express, { Application } from "express";
import morgan from "morgan";
var cors = require("cors");

dotenv.config();

export class App {
  public app: Application;

  constructor(private port?: number | string) {
    this.app = express();
    this.settings();
    this.middlewares();
    this.routes();
    this.dbConnection();
  }

  private settings(): void {
    this.app.set('port', this.port || process.env.PORT || 4000);
  }

  private middlewares(): void {
    this.app.use(morgan('dev'));
    this.app.use(cors());
    this.app.use(express.json());
    this.app.use(express.urlencoded({ extended: false }));
  }
}
```

```

private routes(): void {
    // Las rutas se configurarán más adelante
}

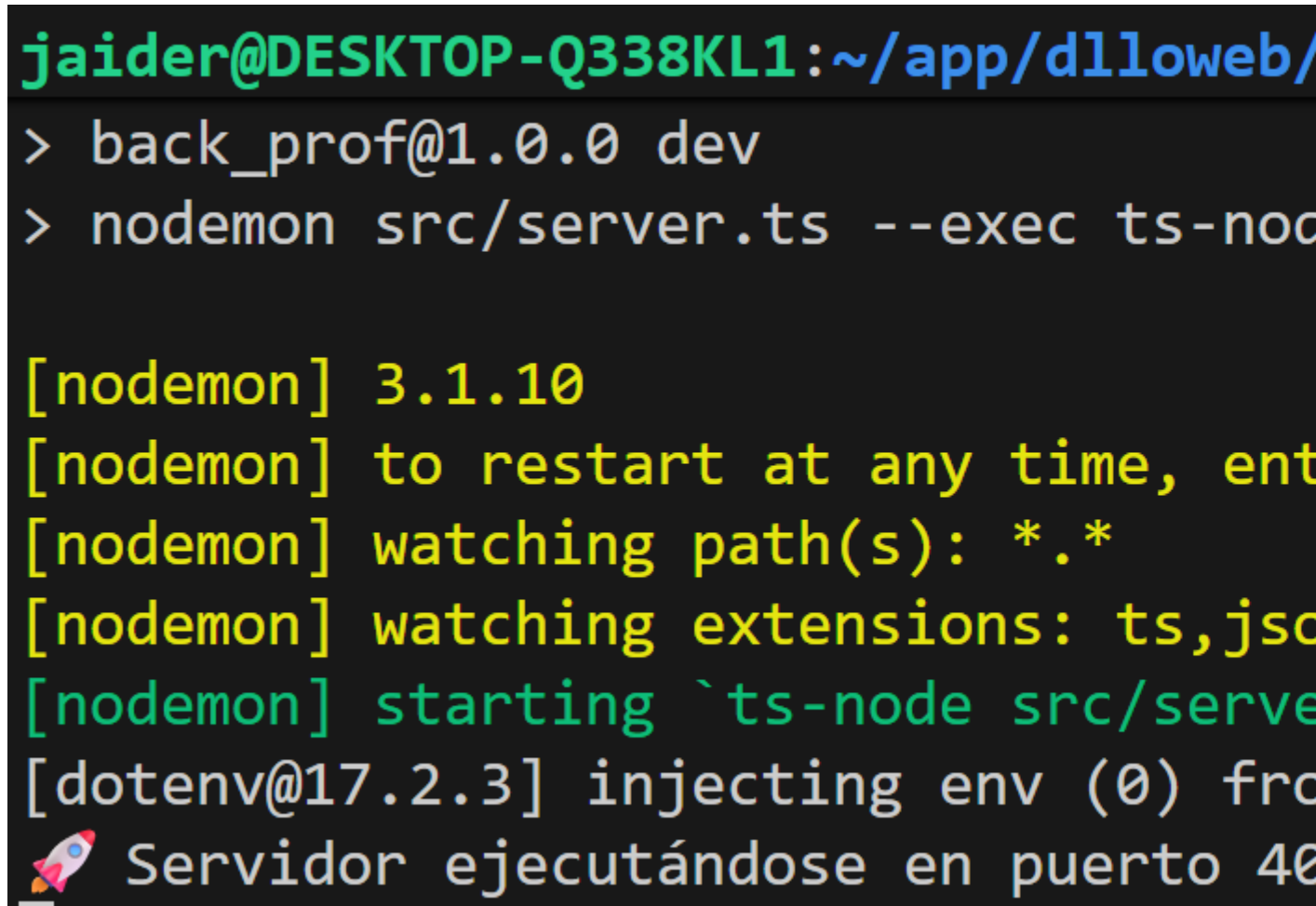
private async dbConnection(): Promise<void> {
    // Conexión, configuración y sincronizar BD
}

async listen() {
    await this.app.listen(this.app.get('port'));
    console.log(`🚀 Servidor ejecutándose en puerto ${this.app.get('port')}`);
}
}

```

Una vez realizado estos dos archivos, ejecutamos con

```
npm run dev
```



```

jaider@DESKTOP-Q338KL1:~/app/dlloweb/
> back_prof@1.0.0 dev
> nodemon src/server.ts --exec ts-node

[nodemon] 3.1.10
[nodemon] to restart at any time, enter
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,jsc
[nodemon] starting `ts-node src/serve
[dotenv@17.2.3] injecting env (0) fro
🚀 Servidor ejecutándose en puerto 40

```

Y si nos dirigimos al navegador y en la URL, colocamos

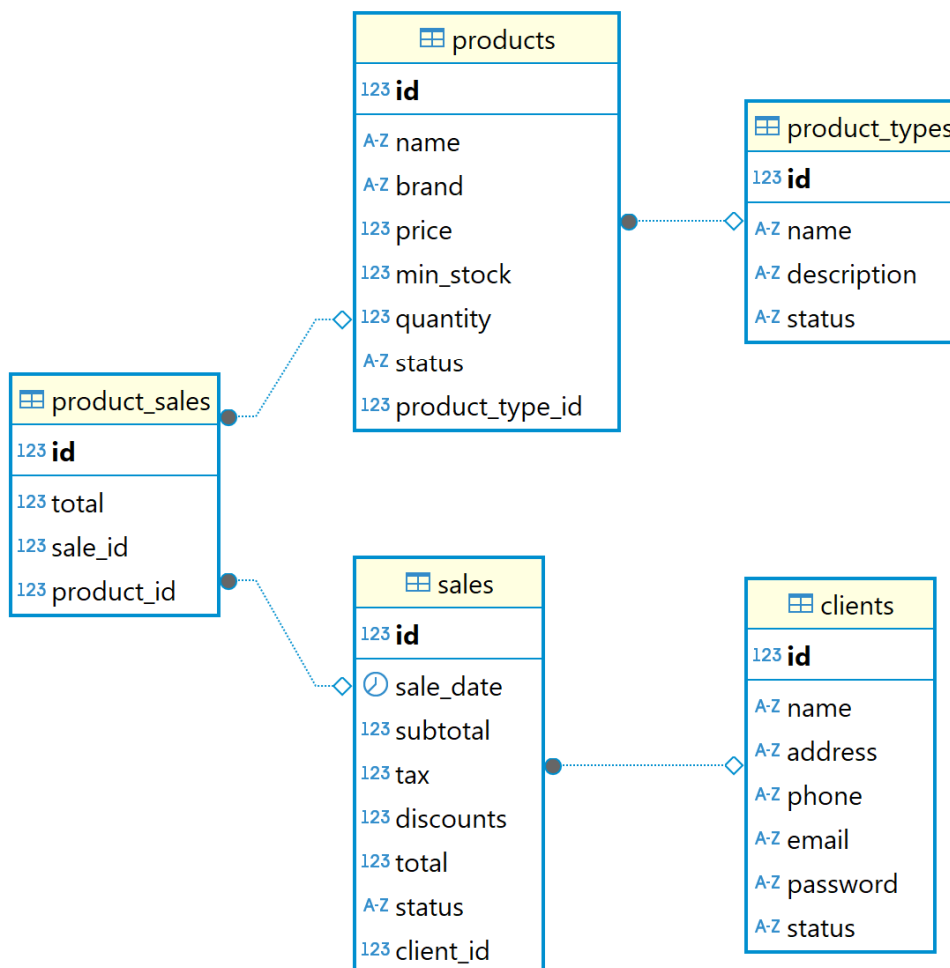
localhost:4000, nos muestra lo siguiente:

MODELOS:

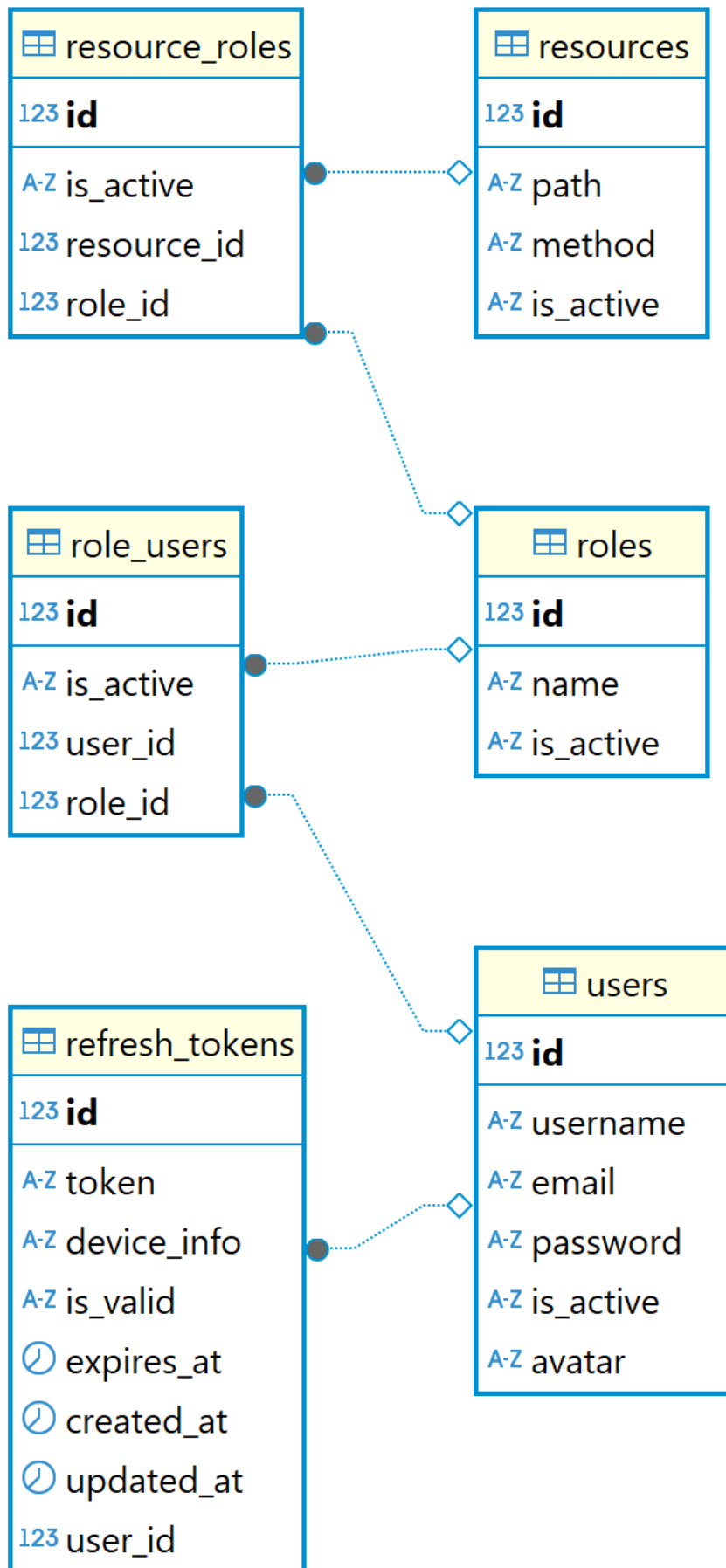
Creación de Modelos

Los modelos hacen referencia a las tablas del diseño de su base de datos. Para este caso asumimos el siguiente diseño.

Modelos del Core del Negocio



Modelos del Esquema de Autorizacion



Detalle de los campos de las tablas

Tablas de Autorización

1. users

- [id](#) (PK, AUTO_INCREMENT)
- [username](#) (STRING, NOT NULL)
- [email](#) (STRING, NOT NULL, UNIQUE)
- [password](#) (STRING, NOT NULL)
- [is_active](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")
- [avatar](#) (STRING, NULLABLE)

2. roles

- [id](#) (PK, AUTO_INCREMENT)
- [name](#) (STRING, NOT NULL)
- [is_active](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

3. role_users (Tabla pivote)

- [id](#) (PK, AUTO_INCREMENT)
- [role_id](#) (FK → roles.id)
- [user_id](#) (FK → users.id)
- [is_active](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

4. resources

- [id](#) (PK, AUTO_INCREMENT)
- [path](#) (STRING, NOT NULL)
- [method](#) (STRING, NOT NULL)
- [is_active](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

5. resource_roles (Tabla pivote)

- [id](#) (PK, AUTO_INCREMENT)
- [resource_id](#) (FK → resources.id)
- [role_id](#) (FK → roles.id)
- [is_active](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

6. refresh_tokens

- [id](#) (PK, AUTO_INCREMENT)
- [user_id](#) (FK → users.id)
- [token](#) (STRING, NOT NULL)
- [device_info](#) (STRING, NOT NULL)
- [is_valid](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")
- [expires_at](#) (DATE)
- [created_at](#) (DATE)
- [updated_at](#) (DATE)

Tablas de Negocio

7. clients

- [id](#) (PK, AUTO_INCREMENT)
- [name](#) (STRING)
- [address](#) (STRING)
- [phone](#) (STRING)
- [email](#) (STRING, UNIQUE)
- [password](#) (STRING)
- [status](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

8. product_types

- [id](#) (PK, AUTO_INCREMENT)
- [name](#) (STRING, NOT NULL)
- [description](#) (STRING, NOT NULL)
- [status](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

9. products

- [id](#) (PK, AUTO_INCREMENT)
- [name](#) (STRING, NOT NULL)
- [brand](#) (STRING, NOT NULL)
- [price](#) (BIGINT, NOT NULL)
- [min_stock](#) (INTEGER, NOT NULL)
- [quantity](#) (INTEGER, NOT NULL)

- [product_type_id](#) (FK → product_types.id)
- [status](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")

10. sales

- [id](#) (PK, AUTO_INCREMENT)
- [sale_date](#) (DATE, NOT NULL)
- [subtotal](#) (BIGINT, NOT NULL)
- [tax](#) (BIGINT, NOT NULL)
- [discounts](#) (BIGINT, NOT NULL)
- [total](#) (BIGINT, NOT NULL)
- [status](#) (ENUM: "ACTIVE"/"INACTIVE", DEFAULT: "ACTIVE")
- [client_id](#) (FK → clients.id)

11. product_sales (Tabla pivote)

- [id](#) (PK, AUTO_INCREMENT)
- [total](#) (BIGINT, NOT NULL)
- [product_id](#) (FK → products.id)
- [sale_id](#) (FK → sales.id)

Relaciones

Sistema de Autorización:

1. **users** ↔ **role_users** ↔ **roles** (Many-to-Many)
2. **roles** ↔ **resource_roles** ↔ **resources** (Many-to-Many)
3. **users** → **refresh_tokens** (One-to-Many)

Sistema de Negocio:

1. **clients** → **sales** (One-to-Many)
2. **product_types** → **products** (One-to-Many)
3. **sales** ↔ **product_sales** ↔ **products** (Many-to-Many)

Antes de crear los modelos de la aplicación es necesario que configuremos nuestra base de datos.

Especificación de la base de datos

Esto nos indica que procedemos a varios procesos.

Debe contar con la instalacion de los Motores de bases de datos:

- MySQL Server
- PostgreSQL
- MS SQL Server
- Oracle Server\

Paso 1: Configuración de Base de Datos

1.1 Instalar dependencias de base de datos

```
npm install sequelize mysql2 pg pg-hstore tedious oracledb  
npm install -D @types/sequelize
```

1.2 Crear archivo .env

```
PORT=3000  
  
# Variable para seleccionar el motor de base de datos  
DB_ENGINE=mysql  
  
# Configuración para MySQL  
MYSQL_HOST=localhost  
MYSQL_USER=admin  
MYSQL_PASSWORD=password  
MYSQL_NAME=almacen_2025_iisem_node  
MYSQL_PORT=3306  
  
# Configuración para PostgreSQL  
POSTGRES_HOST=localhost  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=password  
POSTGRES_NAME=almacen_2025_iisem_node  
POSTGRES_PORT=5432  
  
# Configuración para SQL Server  
MSSQL_HOST=localhost  
MSSQL_USER=sa  
MSSQL_PASSWORD=password  
MSSQL_NAME=almacen_2025_iisem_node  
MSSQL_PORT=1433  
  
# Configuración para Oracle  
ORACLE_HOST=localhost  
ORACLE_USER=ALMACENDB_ADMIN  
ORACLE_PASSWORD=password  
ORACLE_NAME=xe  
ORACLE_PORT=1521  
  
# JWT Secret  
JWT_SECRET=your_jwt_secret_key_here
```

1.3 Crear configuración de base de datos

src/database/db.ts

```
import { Sequelize } from "sequelize";
import dotenv from "dotenv";

dotenv.config();

interface DatabaseConfig {
  dialect: string;
  host: string;
  username: string;
  password: string;
  database: string;
  port: number;
}

const dbConfigurations: Record<string, DatabaseConfig> = {
  mysql: {
    dialect: "mysql",
    host: process.env.MYSQL_HOST || "localhost",
    username: process.env.MYSQL_USER || "root",
    password: process.env.MYSQL_PASSWORD || "",
    database: process.env.MYSQL_NAME || "test",
    port: parseInt(process.env.MYSQL_PORT || "3306")
  },
  postgres: {
    dialect: "postgres",
    host: process.env.POSTGRES_HOST || "localhost",
    username: process.env.POSTGRES_USER || "postgres",
    password: process.env.POSTGRES_PASSWORD || "",
    database: process.env.POSTGRES_NAME || "test",
    port: parseInt(process.env.POSTGRES_PORT || "5432")
  }
};

const selectedEngine = process.env.DB_ENGINE || "mysql";
const selectedConfig = dbConfigurations[selectedEngine];

if (!selectedConfig) {
  throw new Error(`Motor de base de datos no soportado: ${selectedEngine}`);
}

console.log(`📡 Conectando a base de datos: ${selectedEngine.toUpperCase()}`);

export const sequelize = new Sequelize(
  selectedConfig.database,
  selectedConfig.username,
  selectedConfig.password,
  {
    host: selectedConfig.host,
    port: selectedConfig.port,
    dialect: selectedConfig.dialect as any,
    logging: process.env.NODE_ENV === 'development' ? console.log : false,
    pool: {
      max: 5,
      min: 0,
      acquire: 30000,
      idle: 10000
    }
  }
);

export const getDatabaseInfo = () => {
  return {
```

```

    engine: selectedEngine,
    config: selectedConfig,
    connectionString:
`${selectedConfig.dialect}://${selectedConfig.username}@${selectedConfig.host}:${selectedConfig.port}/${selectedConfig.database}`
  });
};

export const testConnection = async (): Promise<boolean> => {
  try {
    await sequelize.authenticate();
    console.log(`✓ Conexión exitosa a ${selectedEngine.toUpperCase()}`);
    return true;
  } catch (error) {
    console.error(`✗ Error de conexión a ${selectedEngine.toUpperCase()}:`, error);
    return false;
  }
};

```

Paso 2: Modelos de Base de Datos

2.1 Instalar dependencias de autenticación

```

npm install bcryptjs jsonwebtoken path-to-regexp
npm install -D @types/bcryptjs @types/jsonwebtoken

```

2.2 Crear modelos de autorización

2.2.1 MODELO USER

src/models/authorization/User.ts

```

import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

export class User extends Model {
  id!: number;
  public username!: string;
  public email!: string;
  public password!: string;
  public is_active!: "ACTIVE" | "INACTIVE";
  public avatar!: string;

  public async checkPassword(password: string): Promise<boolean> {
    return bcrypt.compare(password, this.password);
  }

  public generateToken(): string {
    return jwt.sign({ id: this.id }, process.env.JWT_SECRET || 'secret', {
      expiresIn: '10m',
    });
  }

  public generateRefreshToken(): { token: string, expiresAt: Date } {
    // const expiresIn = '24H';
    const expiresIn = '5m';
    const token = jwt.sign({ id: this.id }, process.env.JWT_SECRET || 'secret', {
      expiresIn,
    });
  }
}

```

```

    });
    const expiresAt = new Date(Date.now() + 5 * 60 * 1000); // 1 minutos
    // const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000); // 24 horas
    return { token, expiresAt };
  }
}

```

```

export interface UserI {
  id?: number;
  username: string;
  email: string;
  password: string;
  is_active: "ACTIVE" | "INACTIVE";
  avatar?: string;
}

```

```

User.init(
  {
    username: {
      type: DataTypes.STRING,
      allowNull: false
    },
    email: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: true
    },
    password: {
      type: DataTypes.STRING,
      allowNull: false
    },
    is_active: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    },
    avatar: {
      type: DataTypes.STRING,
      allowNull: true
    }
  },
  {
    {
      tableName: "users",
      sequelize: sequelize,
      timestamps: false,
      hooks: {
        beforeCreate: async (user: User) => {
          if (user.password) {
            const salt = await bcrypt.genSalt(10);
            user.password = await bcrypt.hash(user.password, salt);
          }
        },
        beforeUpdate: async (user: User) => {
          if (user.password) {
            const salt = await bcrypt.genSalt(10);
            user.password = await bcrypt.hash(user.password, salt);
          }
        }
      }
    }
  }
);

```

2.2.2 MODELO ROLE

src/models/authorization/Role.ts

```
import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";

export class Role extends Model {
  public id!: number;
  public name!: string;
  public is_active!: "ACTIVE" | "INACTIVE";
}

export interface RoleI {
  id?: number;
  name: string;
  is_active: "ACTIVE" | "INACTIVE";
}

Role.init(
  {
    name: {
      type: DataTypes.STRING,
      allowNull: false
    },
    is_active: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    }
  },
  {
    tableName: "roles",
    sequelize: sequelize,
    timestamps: false
  }
);
```

2.2.3 MODELO ROLEUSER (MODELO PIVOTE RELACIÓN MUCHOS A MUCHOS)

src/models/authorization/RoleUser.ts

```
import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";

export class RoleUser extends Model {
  public id!: number;
  public role_id!: number;
  public user_id!: number;
  public is_active!: "ACTIVE" | "INACTIVE";
}

export interface RoleUserI {
  id?: number;
  role_id: number;
  user_id: number;
  is_active: "ACTIVE" | "INACTIVE";
}

RoleUser.init(
  {
    is_active: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    }
  }
);
```



```

    },
    {
      tableName: "role_users",
      sequelize: sequelize,
      timestamps: false
    }
  );

```

2.2.4 MODELO RESOURCE

src/models/authorization/Resource.ts

```

import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";

export class Resource extends Model {
  public id!: number;
  public path!: string;
  public method!: string;
  public is_active!: "ACTIVE" | "INACTIVE";
}

export interface ResourceI {
  id?: number;
  path: string;
  method: string;
  is_active: "ACTIVE" | "INACTIVE";
}

Resource.init(
  {
    path: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Path cannot be empty" },
      },
    },
    method: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Method cannot be empty" },
      },
    },
    is_active: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    },
  },
  {
    tableName: "resources",
    sequelize: sequelize,
    timestamps: false,
  }
);

```

2.2.5 MODELO RESOURCEROLE(MODELO PIVOTE RELACIÓN MUCHOS A MUCHOS)

src/models/authorization/ResourceRole.ts

```

import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";

export class ResourceRole extends Model {
  public id!: number;
  public resource_id!: number;
  public role_id!: number;
  public is_active!: "ACTIVE" | "INACTIVE";
}

export interface ResourceRoleI {
  id?: number;
  resource_id: number;
  role_id: number;
  is_active: "ACTIVE" | "INACTIVE";
}

ResourceRole.init(
  {
    is_active: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    },
  },
  {
    tableName: "resource_roles",
    sequelize: sequelize,
    timestamps: false,
  }
);

```

2.2.6 MODELO REFRESHTOKEN

src/models/authorization/ResfreshToken.ts

```

import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

export class RefreshToken extends Model {
  id!: number;
  public user_id!: number;
  public token!: string;
  public device_info!: string;
  public is_valid!: "ACTIVE" | "INACTIVE";
  public expires_at!: Date;
  public created_at!: Date;
  public updated_at!: Date;
}

export interface RefreshTokenI {
  user_id?: number;
  token: string;
  device_info: string;
  is_valid: "ACTIVE" | "INACTIVE";
  expires_at: Date;
  created_at: Date;
  updated_at: Date;
}

```

```

RefreshToken.init(
{
  token: {
    type: DataTypes.STRING,
    allowNull: false
  },
  device_info: {
    type: DataTypes.STRING,
    allowNull: false
  },
  is_valid: {
    type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
    defaultValue: "ACTIVE",
  },
  expires_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  created_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  updated_at: {
    type: DataTypes.DATE,
    allowNull: true
  }
},
{
  tableName: "refresh_tokens",
  sequelize: sequelize,
  timestamps: false,
  hooks: {
    beforeCreate: (refreshToken: RefreshToken) => {
      const currentDate = new Date();
      refreshToken.created_at = currentDate;
      refreshToken.updated_at = currentDate;
    },
    beforeUpdate: (refreshToken: RefreshToken) => {
      const currentDate = new Date();
      refreshToken.updated_at = currentDate;
    }
  }
}
);

```

2.3 Crear modelos de negocio

2.3.1 MODELO CLIENT

src/models/Client.ts

```

import { DataTypes, Model } from "sequelize";
import { sequelize } from "../database/db";
import bcrypt from 'bcryptjs';

export interface ClientI {
  id?: number;
  name: string;
  address: string;
  phone: string;
  email: string;
  password: string;
}

```

```

    status: "ACTIVE" | "INACTIVE";
}

export class Client extends Model {
  public id!: number;
  public name!: string;
  public address!: string;
  public phone!: string;
  public email!: string;
  public password!: string;
  public status!: "ACTIVE" | "INACTIVE";
}

Client.init(
{
  name: {
    type: DataTypes.STRING,
    allowNull: true,
  },
  address: {
    type: DataTypes.STRING,
    allowNull: true,
  },
  phone: {
    type: DataTypes.STRING,
    allowNull: true,
    validate: {
      notEmpty: { msg: "Phone cannot be empty" },
    },
  },
  email: {
    type: DataTypes.STRING,
    allowNull: true,
    unique: true,
    validate: {
      isEmail: { msg: "Email must be a valid email address" }, // Validate email format
    },
  },
  password: {
    type: DataTypes.STRING,
    allowNull: true,
  },
  status: {
    type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
    defaultValue: "ACTIVE",
  },
},
{
  sequelize,
  modelName: "Client",
  tableName: "clients",
  timestamps: false,
  hooks: {
    beforeCreate: async (client: Client) => {
      if (client.password) {
        const salt = await bcrypt.genSalt(10);
        client.password = await bcrypt.hash(client.password, salt);
      }
    },
    beforeUpdate: async (client: Client) => {
      if (client.password) {
        const salt = await bcrypt.genSalt(10);
        client.password = await bcrypt.hash(client.password, salt);
      }
    }
  }
}

```

```

    }
  }
});

```

2.3.2 MODELO PRODUCTTYPE

src/models/ProductType.ts

```

import { DataTypes, Model } from "sequelize";
import { sequelize } from "../database/db";

export interface ProductTypeI {
  id?: number;
  name: string;
  description: string;
  status: "ACTIVE" | "INACTIVE";
}

export class ProductType extends Model {
  public id!: number;
  public name!: string;
  public description!: string;
  public status!: "ACTIVE" | "INACTIVE";
}

ProductType.init(
  {
    name: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Name cannot be empty" },
      },
    },
    description: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Description cannot be empty" },
      },
    },
    status: {
      type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
      defaultValue: "ACTIVE",
    },
  },
  {
    sequelize,
    modelName: "ProductType",
    tableName: "product_types",
    timestamps: false,
  }
);

```

2.3.3 MODELO PRODUCT

src/models/Product.ts

```

import { DataTypes, Model } from "sequelize";
import { sequelize } from "../database/db";

```

```

export interface ProductI {
  id?: number;
  name: string;
  brand: string;
  price: number;
  min_stock: number;
  quantity: number;
  product_type_id: number;
  status: "ACTIVE" | "INACTIVE";
}

export class Product extends Model {
  public id!: number;
  public name!: string;
  public brand!: string;
  public price!: number;
  public min_stock!: number;
  public quantity!: number;
  public product_type_id!: number;
  public status!: "ACTIVE" | "INACTIVE";
}

Product.init(
{
  name: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Name cannot be empty" },
    },
  },
  brand: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Brand cannot be empty" },
    },
  },
  price: {
    type: DataTypes.BIGINT,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Price cannot be empty" },
      isFloat: { msg: "Price must be a valid number" },
    },
  },
  min_stock: {
    type: DataTypes.INTEGER,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Minimum stock cannot be empty" },
      isInt: { msg: "Minimum stock must be an integer" },
    },
  },
  quantity: {
    type: DataTypes.INTEGER,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Quantity cannot be empty" },
      isInt: { msg: "Quantity must be an integer" },
    },
  },
  status: {
    type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
    defaultValue: "ACTIVE",
  },
}

```

```

    },
  },
  {
    sequelize,
    modelName: "Product",
    tableName: "products",
    timestamps: false,
  }
);

```

2.3.4 MODELO SALE

src/models/Sale.ts

```

import { DataTypes, Model } from "sequelize";
import { sequelize } from "../database/db";

export interface SaleI {
  id?: number;
  sale_date: Date;
  subtotal: number;
  tax: number;
  discounts: number;
  total: number;
  status: "ACTIVE" | "INACTIVE";
  client_id: number;
}

export class Sale extends Model {
  public id!: number;
  public sale_date!: Date;
  public subtotal!: number;
  public tax!: number;
  public discounts!: number;
  public total!: number;
  public status!: "ACTIVE" | "INACTIVE";
  public client_id!: number;
}

Sale.init(
  {
    sale_date: {
      type: DataTypes.DATE,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Sale date cannot be empty" },
        isDate: { args: true, msg: "Must be a valid date" },
      },
    },
    subtotal: {
      type: DataTypes.BIGINT,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Subtotal cannot be empty" },
        isFloat: { msg: "Subtotal must be a valid number" },
      },
    },
    tax: {
      type: DataTypes.BIGINT,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Tax cannot be empty" },
        isFloat: { msg: "Tax must be a valid number" },
      },
    },
  },
  { sequelize }
);

```

```

    },
  },
  discounts: {
    type: DataTypes.BIGINT,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Discounts cannot be empty" },
      isFloat: { msg: "Discounts must be a valid number" },
    },
  },
  total: {
    type: DataTypes.BIGINT,
    allowNull: false,
    validate: {
      notEmpty: { msg: "Total cannot be empty" },
      isFloat: { msg: "Total must be a valid number" },
    },
  },
  status: {
    type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
    defaultValue: "ACTIVE",
  },
},
{
  sequelize,
  modelName: "Sale",
  tableName: "sales",
  timestamps: false,
}
);

```

2.3.5 MODELO PRODUCTSALE

`src/models/ProductSale.ts`

```

import { DataTypes, Model } from "sequelize";
import { sequelize } from "../database/db";

export interface ProductSaleI {
  id?: number;
  total: number;
  product_id: number;
  sale_id: number;
}

export class ProductSale extends Model {
  public id!: number;
  public total!: number;
  public product_id!: number;
  public sale_id!: number;
}

ProductSale.init(
  {
    total: {
      type: DataTypes.BIGINT,
      allowNull: false,
      validate: {
        notEmpty: { msg: "Name cannot be empty" },
      },
    },
  },
  {
    sequelize,
    modelName: "ProductSale",
    tableName: "product_sales",
    timestamps: true,
  }
);

```



```

    sequelize,
    modelName: "ProductSale",
    tableName: "product_sales",
    timestamps: false,
  }
);

```

2.4 Configurar relaciones entre modelos

Agregar al final de cada archivo de modelo las relaciones correspondientes y se tienen en cuenta las relaciones en ambos sentidos

```

// En User.ts
....
....
....
import { RoleUser } from "../RoleUser";

...
...
...

```

Al final del archivo

```

User.hasMany(RoleUser, {
  foreignKey: 'user_id',
  sourceKey: "id",
});
RoleUser.belongsTo(User, {
  foreignKey: 'user_id',
  targetKey: "id",
});

```

Quedando finalmente el archivo de la siguiente manera:

src/models/User.ts

```

import { Model, DataTypes } from "sequelize";
import { sequelize } from "../../database/db";
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { RoleUser } from "../RoleUser";

export class User extends Model {
  id!: number;
  public username!: string;
  public email!: string;
  public password!: string;
  public is_active!: "ACTIVE" | "INACTIVE";
  public avatar!: string;

  public async checkPassword(password: string): Promise<boolean> {
    return bcrypt.compare(password, this.password);
  }

  public generateToken(): string {
    return jwt.sign({ id: this.id }, process.env.JWT_SECRET || 'secret', {
      expiresIn: '10m',
    });
  }
}

```

```

    });
}

public generateRefreshToken(): { token: string, expiresAt: Date } {
    // const expiresIn = '24H';
    const expiresIn = '5m';
    const token = jwt.sign({ id: this.id }, process.env.JWT_SECRET || 'secret', {
        expiresIn,
    });
    const expiresAt = new Date(Date.now() + 5 * 60 * 1000); // 1 minutos
    // const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000); // 24 horas
    return { token, expiresAt };
}
}

```

```

export interface UserI {
    id?: number;
    username: string;
    email: string;
    password: string;
    is_active: "ACTIVE" | "INACTIVE";
    avatar?: string;
}

```

```

User.init(
{
    username: {
        type: DataTypes.STRING,
        allowNull: false
    },
    email: {
        type: DataTypes.STRING,
        allowNull: false,
        unique: true
    },
    password: {
        type: DataTypes.STRING,
        allowNull: false
    },
    is_active: {
        type: DataTypes.ENUM("ACTIVE", "INACTIVE"),
        defaultValue: "ACTIVE",
    },
    avatar: {
        type: DataTypes.STRING,
        allowNull: true
    }
},
{
    tableName: "users",
    sequelize: sequelize,
    timestamps: false,
    hooks: {
        beforeCreate: async (user: User) => {
            if (user.password) {
                const salt = await bcrypt.genSalt(10);
                user.password = await bcrypt.hash(user.password, salt);
            }
        },
        beforeUpdate: async (user: User) => {
            if (user.password) {
                const salt = await bcrypt.genSalt(10);
                user.password = await bcrypt.hash(user.password, salt);
            }
        }
    }
}
)

```

```

    }
  }
}
);

User.hasMany(RoleUser, {
  foreignKey: 'user_id',
  sourceKey: "id",
});
RoleUser.belongsTo(User, {
  foreignKey: 'user_id',
  targetKey: "id",
});

```

De la misma manera realizar las distintas relaciones mencionadas a continuacion:

```

// En Role.ts
....
....
....
import { RoleUser } from "../RoleUser";

...
...
...

```

Al **final** del archivo

```

Role.hasMany(RoleUser, {
  foreignKey: 'role_id',
  sourceKey: "id",
});
RoleUser.belongsTo(Role, {
  foreignKey: 'role_id',
  targetKey: "id",
});

```

// En ResourceRole.ts

```

....
....
....
import { Resource } from "../Resource";
import { Role } from "../Role";

...
...
...

```

Al **final** del archivo

```

Resource.hasMany(ResourceRole, {
  foreignKey: "resource_id",
  sourceKey: "id",
});
ResourceRole.belongsTo(Resource, {

```

```
    foreignKey: "resource_id",
    targetKey: "id",
  });

Role.hasMany(ResourceRole, {
  foreignKey: "role_id",
  sourceKey: "id",
});
ResourceRole.belongsTo(Role, {
  foreignKey: "role_id",
  targetKey: "id",
});
```

// En RefreshToken.ts

```
....
....
....

import { User } from "../User";

...
...
...
```

Al **final** del archivo

```
User.hasMany(RefreshToken, {
  foreignKey: 'user_id',
  sourceKey: "id",
});
RefreshToken.belongsTo(User, {
  foreignKey: 'user_id',
  targetKey: "id",
});
```

// En Client.ts

```
....
....
....

import { Sale } from "../Sale";

...
...
...
```

Al **final** del archivo

```
Client.hasMany(Sale, {
  foreignKey: "client_id",
  sourceKey: "id",
});
Sale.belongsTo(Client, {
  foreignKey: "client_id",
```

```
    targetKey: "id",
  });
```

```
// En ProductType.ts
```

```
....
....
....
```

```
import { Product } from "../Product";
```

```
...
...
...
```

Al `final` del archivo

```
ProductType.hasMany(Product, {
  foreignKey: "product_type_id",
  sourceKey: "id",
});
Product.belongsTo(ProductType, {
  foreignKey: "product_type_id",
  targetKey: "id",
});
```

```
// En Product.ts
```

```
....
....
....
```

```
import { ProductSale } from "../ProductSale";
```

```
...
...
...
```

Al `final` del archivo

```
Product.hasMany(ProductSale, {
  foreignKey: "product_id",
  sourceKey: "id",
});
ProductSale.belongsTo(Product, {
  foreignKey: "product_id",
  targetKey: "id",
});
```

```
// En Sale.ts
```

```
....
```

```
....  
....
```

```
import { ProductSale } from "../ProductSale";
```

```
...  
...  
...
```

Al final del archivo

```
Sale.hasMany(ProductSale, {  
  foreignKey: "sale_id",  
  sourceKey: "id",  
});  
ProductSale.belongsTo(Sale, {  
  foreignKey: "sale_id",  
  targetKey: "id",  
});
```

Paso 3: Creacion, conexion y sincronizacion de la base de datos

Es importante saber que en el archivo .env se encuentran los datos de conexion y nombre de la base de datos, por ende se debe crear las bases de datos con dicho nombre en los motores de base de datos indicados:

Por ejemplo en MySQL

```
# Configuración para MySQL  
  
MYSQL_HOST=localhost  
MYSQL_USER=admin  
MYSQL_PASSWORD=MiNiCo57**  
MYSQL_NAME=almacen_2025_iisem_node  
MYSQL_PORT=3306
```

Para mas informacion revisar archivo .env

Esto quiere decir que los motores deben existir al igual que las bases de datos en blanco

Luego se debe sincronizar en la configuracion asi:

src/config/index.ts

Agregar el metodo dbConnection

```
import dotenv from "dotenv";  
import express, { Application } from "express";  
import morgan from "morgan";  
import { sequelize, testConnection, getDatabaseInfo } from "../database/db";  
var cors = require("cors");
```

```

dotenv.config();

export class App {
  public app: Application;

  constructor(private port?: number | string) {
    this.app = express();
    this.settings();
    this.middlewares();
    this.routes();
    this.dbConnection();
  }

  private settings(): void {
    this.app.set('port', this.port || process.env.PORT || 4000);
  }

  private middlewares(): void {
    this.app.use(morgan('dev'));
    this.app.use(cors());
    this.app.use(express.json());
    this.app.use(express.urlencoded({ extended: false }));
  }

  private routes(): void {
    // Las rutas se configurarán más adelante
  }

  private async dbConnection(): Promise<void> {
    try {
      // Mostrar información de la base de datos seleccionada
      const dbInfo = getDatabaseInfo();
      console.log(`🔗 Intentando conectar a: ${dbInfo.engine.toUpperCase()}`);

      // Probar la conexión
      const isConnected = await testConnection();

      if (!isConnected) {
        throw new Error(`No se pudo conectar a la base de datos ${dbInfo.engine.toUpperCase()}`);
      }

      // Sincronizar la base de datos
      await sequelize.sync({ force: false });
      console.log(`📦 Base de datos sincronizada exitosamente`);

    } catch (error) {
      console.error("❌ Error al conectar con la base de datos:", error);
      process.exit(1); // Terminar la aplicación si no se puede conectar
    }
  }

  async listen() {
    await this.app.listen(this.app.get('port'));
    console.log(`🚀 Servidor ejecutándose en puerto ${this.app.get('port')}`);
  }
}

```

Una vez realizado estos cambios, ejecutamos con

```
npm run dev
```

jaider@DESKTOP-Q338KL1:~/app/dlloweb

[nodemon] starting `ts-node src/serv

[dotenv@17.2.3] injecting env (23) fr



Conectando a base de datos: ORACL



Host: localhost:1521



Base de datos: xe

[dotenv@17.2.3] injecting env (0) fr



Intentando conectar a: ORACLE



Servidor ejecutándose en puerto 4



Conexión exitosa a ORACLE



Base de datos sincronizada exitos

CRUD:

CRUD: la base de la gestión de datos

El concepto CRUD está estrechamente vinculado a la gestión de datos digitales. CRUD hace referencia a **un acrónimo** en el que se reúnen las primeras letras de las cuatro operaciones fundamentales de aplicaciones persistentes en sistemas de bases de datos:

- **Create** (Crear registros)
- **Read** bzw. **Retrieve** (Leer registros)
- **Update** (Actualizar registros)
- **Delete** bzw. **Destroy** (Borrar registros)

En pocas palabras, CRUD resume las funciones requeridas por un usuario para crear y gestionar datos. Varios procesos de gestión de datos están basados en CRUD, en los que **dichas operaciones están específicamente adaptadas a los requisitos del sistema y de usuario**, ya sea para la gestión de bases de datos o para el uso de aplicaciones. Para los expertos, las operaciones son las herramientas de acceso típicas e indispensables para comprobar, por ejemplo, los problemas de la base de datos, mientras que para los usuarios, CRUD significa crear una cuenta (create) y utilizarla (read), actualizarla (update) o borrarla (delete) en cualquier momento. Dependiendo de la configuración regional, las operaciones CRUD pueden implementarse de diferentes maneras, como lo muestra la siguiente tabla:

CRUD-Operation	SQL	RESTful HTTP	XQuery
Create	INSERT	POST, PUT	insert
Read	SELECT	GET, HEAD	copy/modify/return
Update	UPDATE	PUT, PATCH	replace, rename

CRUD- Operation	SQL	RESTful HTTP	XQuery
Delete	DELETE	DELETE	delete

Debemos crear los 6 metodos por cada modelo segun demande los requerimientos del aplicativo.

- Buscar todos los registros
- Buscar por Id
- Crear o Insertar un Registro
- Actualizar o Cambiar el valor de uno o varios registros
- Eliminar fisicamente uno o varios registros
- Eliminar Logicamente uno o varios registros

Para ello se realizan los controladores y las rutas de de cada modelo, al igual que las configuraciones de la rutas de la siguiente manera:

Paso 1: Controladores

1.1 Controladores de los modelos de Autorizacion

1.1.1 CONTROLADOR USER

`src/controllers/Authorization/user.controller.ts`

```
import { Request, Response } from 'express';
import { User, UserI } from '../../models/authorization/User';

export class UserController {
  public async getAllUsers(req: Request, res: Response): Promise<void> {
    try {
      const users: UserI[] = await User.findAll();
      res.status(200).json({ users });
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener los usuarios' });
    }
  }
}
```

1.1.2 CONTROLADOR ROLE

`src/controllers/Authorization/role.controller.ts`

```
import { Request, Response } from 'express';
import { Role, RoleI } from '../../models/authorization/Role';
```

```

export class RoleController {
  // Obtener todos los roles
  public async getAllRoles(req: Request, res: Response): Promise<void> {
    try {
      const roles: RoleI[] = await Role.findAll();
      res.status(200).json({ roles });
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener los roles' });
    }
  }

  // Obtener un rol por ID
  public async getRoleById(req: Request, res: Response): Promise<void> {
    try {
      const { id } = req.params;
      const role = await Role.findOne({ where: { id, is_active: "ACTIVE" } });
      if (role) {
        res.status(200).json(role);
      } else {
        res.status(404).json({ error: 'Rol no encontrado o inactivo' });
      }
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener el rol' });
    }
  }

  // Crear un nuevo rol
  public async createRole(req: Request, res: Response): Promise<void> {
    const { name, is_active } = req.body;
    try {
      const newRole = await Role.create({ name, is_active });
      res.status(201).json(newRole);
    } catch (error: any) {
      res.status(400).json({ error: error.message });
    }
  }

  // Actualizar un rol
  public async updateRole(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    const { name, is_active } = req.body;
    try {
      const role = await Role.findOne({ where: { id, is_active: "ACTIVE" } });
      if (role) {
        await role.update({ name, is_active });
        res.status(200).json(role);
      } else {
        res.status(404).json({ error: 'Rol no encontrado o inactivo' });
      }
    } catch (error: any) {
      res.status(400).json({ error: error.message });
    }
  }

  // Eliminar un rol físicamente
  public async deleteRole(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    try {
      const role = await Role.findPk(id);
      if (role) {
        await role.destroy();
        res.status(200).json({ message: 'Rol eliminado correctamente' });
      } else {
        res.status(404).json({ error: 'Rol no encontrado' });
      }
    }
  }
}

```

```

    }
  } catch (error) {
    res.status(500).json({ error: 'Error al eliminar el rol' });
  }
}

// Eliminar un rol lógicamente
public async deleteRoleAdv(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  try {
    const role = await Role.findOne({ where: { id, is_active: "ACTIVE" } });
    if (role) {
      await role.update({ is_active: "INACTIVE" });
      res.status(200).json({ message: 'Rol marcado como inactivo' });
    } else {
      res.status(404).json({ error: 'Rol no encontrado' });
    }
  } catch (error) {
    res.status(500).json({ error: 'Error al marcar el rol como inactivo' });
  }
}
}

```

1.1.3 CONTROLADOR ROLEUSER

src/controllers/Authorization/role_user.controller.ts

```

import { Request, Response } from 'express';
import { RoleUser, RoleUserI } from '../../models/authorization/RoleUser';

export class RoleUserController {
  // Obtener todos los RoleUsers
  public async getAllRoleUsers(req: Request, res: Response): Promise<void> {
    try {
      const roleUsers: RoleUserI[] = await RoleUser.findAll();
      res.status(200).json({ roleUsers });
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener los usuarios de roles' });
    }
  }

  // Obtener un RoleUser por ID
  public async getRoleUserById(req: Request, res: Response): Promise<void> {
    try {
      const { id } = req.params;
      const roleUser = await RoleUser.findOne({ where: { id, is_active: "ACTIVE" } });
      if (roleUser) {
        res.status(200).json(roleUser);
      } else {
        res.status(404).json({ error: 'RoleUser no encontrado o inactivo' });
      }
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener el RoleUser' });
    }
  }

  // Crear un nuevo RoleUser
  public async createRoleUser(req: Request, res: Response): Promise<void> {
    const { role_id, user_id, is_active } = req.body;
    try {
      const newRoleUser = await RoleUser.create({ role_id, user_id, is_active });
      res.status(201).json(newRoleUser);
    } catch (error: any) {

```

```

        res.status(400).json({ error: error.message });
    }
}

// Actualizar un RoleUser
public async updateRoleUser(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    const { role_id, user_id, is_active } = req.body;
    try {
        const roleUser = await RoleUser.findOne({ where: { id, is_active: "ACTIVE" } });
        if (roleUser) {
            await roleUser.update({ role_id, user_id, is_active });
            res.status(200).json(roleUser);
        } else {
            res.status(404).json({ error: 'RoleUser no encontrado o inactivo' });
        }
    } catch (error: any) {
        res.status(400).json({ error: error.message });
    }
}

// Eliminar un RoleUser físicamente
public async deleteRoleUser(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    try {
        const roleUser = await RoleUser.findPk(id);
        if (roleUser) {
            await roleUser.destroy();
            res.status(200).json({ message: 'RoleUser eliminado correctamente' });
        } else {
            res.status(404).json({ error: 'RoleUser no encontrado' });
        }
    } catch (error) {
        res.status(500).json({ error: 'Error al eliminar el RoleUser' });
    }
}

// Eliminar un RoleUser lógicamente
public async deleteRoleUserAdv(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    try {
        const roleUser = await RoleUser.findOne({ where: { id, is_active: "ACTIVE" } });
        if (roleUser) {
            await roleUser.update({ is_active: "INACTIVE" });
            res.status(200).json({ message: 'RoleUser marcado como inactivo' });
        } else {
            res.status(404).json({ error: 'RoleUser no encontrado' });
        }
    } catch (error) {
        res.status(500).json({ error: 'Error al marcar el RoleUser como inactivo' });
    }
}
}

```

1.1.4 CONTROLADOR RESOURCE

src/controllers/Authorization/resource.controller.ts

```

import { Request, Response } from "express";
import { Resource, ResourceI } from "../../models/authorization/Resource";

export class ResourceController {
    // Obtener todos los recursos

```

```

public async getAllResources(req: Request, res: Response): Promise<void> {
  try {
    const resources: ResourceI[] = await Resource.findAll({
      where: { is_active: "ACTIVE" },
    });
    res.status(200).json({ resources });
  } catch (error) {
    res.status(500).json({ error: "Error al obtener los recursos" });
  }
}

```

// Obtener un recurso por ID

```

public async getResourceById(req: Request, res: Response): Promise<void> {
  try {
    const { id } = req.params;
    const resource = await Resource.findOne({
      where: { id, is_active: "ACTIVE" },
    });
    if (resource) {
      res.status(200).json(resource);
    } else {
      res.status(404).json({ error: "Recurso no encontrado o inactivo" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error al obtener el recurso" });
  }
}

```

// Crear un nuevo recurso

```

public async createResource(req: Request, res: Response): Promise<void> {
  const { path, method, is_active } = req.body;
  try {
    const newResource = await Resource.create({ path, method, is_active });
    res.status(201).json(newResource);
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

// Actualizar un recurso

```

public async updateResource(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  const { path, method, is_active } = req.body;
  try {
    const resource = await Resource.findOne({ where: { id, is_active: "ACTIVE" } });
    if (resource) {
      await resource.update({ path, method, is_active });
      res.status(200).json(resource);
    } else {
      res.status(404).json({ error: "Recurso no encontrado o inactivo" });
    }
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

// Eliminar un recurso físicamente

```

public async deleteResource(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  try {
    const resource = await Resource.findPk(id);
    if (resource) {
      await resource.destroy();
      res.status(200).json({ message: "Recurso eliminado correctamente" });
    } else {

```

```

        res.status(404).json({ error: "Recurso no encontrado" });
    }
    } catch (error) {
        res.status(500).json({ error: "Error al eliminar el recurso" });
    }
}

// Eliminar un recurso lógicamente
public async deleteResourceAdv(req: Request, res: Response): Promise<void> {
    const { id } = req.params;
    try {
        const resource = await Resource.findOne({ where: { id, is_active: "ACTIVE" } });
        if (resource) {
            await resource.update({ is_active: "INACTIVE" });
            res.status(200).json({ message: "Recurso marcado como inactivo" });
        } else {
            res.status(404).json({ error: "Recurso no encontrado" });
        }
    } catch (error) {
        res.status(500).json({ error: "Error al marcar el recurso como inactivo" });
    }
}
}

```

1.1.5 CONTROLADOR RESOURCEROLE

src/controllers/Authorization/resourceRole.controller.ts

```

import { Request, Response } from "express";
import { ResourceRole, ResourceRoleI } from "../../models/authorization/ResourceRole";

export class ResourceRoleController {
    // Obtener todos los ResourceRoles
    public async getAllResourceRoles(req: Request, res: Response): Promise<void> {
        try {
            const resourceRoles: ResourceRoleI[] = await ResourceRole.findAll({
                where: { is_active: "ACTIVE" },
            });
            res.status(200).json({ resourceRoles });
        } catch (error) {
            res.status(500).json({ error: "Error al obtener los ResourceRoles" });
        }
    }

    // Obtener un ResourceRole por ID
    public async getResourceRoleById(req: Request, res: Response): Promise<void> {
        try {
            const { id } = req.params;
            const resourceRole = await ResourceRole.findOne({
                where: { id, is_active: "ACTIVE" },
            });
            if (resourceRole) {
                res.status(200).json(resourceRole);
            } else {
                res.status(404).json({ error: "ResourceRole no encontrado o inactivo" });
            }
        } catch (error) {
            res.status(500).json({ error: "Error al obtener el ResourceRole" });
        }
    }

    // Crear un nuevo ResourceRole
    public async createResourceRole(req: Request, res: Response): Promise<void> {

```

```

const { resource_id, role_id, is_active } = req.body;
try {
  const newResourceRole = await ResourceRole.create({ resource_id, role_id, is_active });
  res.status(201).json(newResourceRole);
} catch (error: any) {
  res.status(400).json({ error: error.message });
}
}

// Actualizar un ResourceRole
public async updateResourceRole(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  const { resource_id, role_id, is_active } = req.body;
  try {
    const resourceRole = await ResourceRole.findOne({ where: { id, is_active: "ACTIVE" } });
    if (resourceRole) {
      await resourceRole.update({ resource_id, role_id, is_active });
      res.status(200).json(resourceRole);
    } else {
      res.status(404).json({ error: "ResourceRole no encontrado o inactivo" });
    }
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

// Eliminar un ResourceRole físicamente
public async deleteResourceRole(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  try {
    const resourceRole = await ResourceRole.findByPk(id);
    if (resourceRole) {
      await resourceRole.destroy();
      res.status(200).json({ message: "ResourceRole eliminado correctamente" });
    } else {
      res.status(404).json({ error: "ResourceRole no encontrado" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error al eliminar el ResourceRole" });
  }
}

// Eliminar un ResourceRole lógicamente
public async deleteResourceRoleAdv(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  try {
    const resourceRole = await ResourceRole.findOne({ where: { id, is_active: "ACTIVE" } });
    if (resourceRole) {
      await resourceRole.update({ is_active: "INACTIVE" });
      res.status(200).json({ message: "ResourceRole marcado como inactivo" });
    } else {
      res.status(404).json({ error: "ResourceRole no encontrado" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error al marcar el ResourceRole como inactivo" });
  }
}
}

```

1.1.6 CONTROLADOR REFRESHTOKEN

src/controllers/Authorization/refresh_token.controller.ts


```
import { Request, Response } from 'express';
import { RefreshToken, RefreshTokenI } from '../models/authorization/RefreshToken';

export class RefreshTokenController {
  public async getAllRefreshToken(req: Request, res: Response): Promise<void> {
    try {
      const referes_tokens: RefreshTokenI[] = await RefreshToken.findAll();
      res.status(200).json({ referes_tokens });
    } catch (error) {
      res.status(500).json({ error: 'Error al obtener los los refresh token' });
    }
  }
}
```

1.2 Controladores de los modelos de Negocio

1.2.1 CONTROLADOR CLIENT

src/controllers/client.controller.ts

```
import { Request, Response } from "express";
import { Client, ClientI } from "../models/Client";

export class ClientController {
  // Get all clients with status "ACTIVE"
  public async getAllClients(req: Request, res: Response) {
    try {
      const clients: ClientI[] = await Client.findAll({
        where: { status: 'ACTIVE' },
      });
      res.status(200).json({ clients });
    } catch (error) {
      res.status(500).json({ error: "Error fetching clients" });
    }
  }

  // Get a client by ID
  public async getClientById(req: Request, res: Response) {
    try {
      const { id: pk } = req.params;
      const client = await Client.findOne({
        where: {
          id: pk,
          status: 'ACTIVE' },
      });
      if (client) {
        res.status(200).json({ client });
      } else {
        res.status(404).json({ error: "Client not found or inactive" });
      }
    } catch (error) {
      res.status(500).json({ error: "Error fetching client" });
    }
  }

  // Create a new client
  public async createClient(req: Request, res: Response) {
    const { id, name, address, phone, email, password, status } = req.body;
    try {
      let body: ClientI = {
        name,
        address,
```

```
    phone,  
    email,  
    password,  
    status,  
  };  
};
```

```
    const newClient = await Client.create({ ...body });  
    res.status(201).json(newClient);  
  } catch (error: any) {  
    res.status(400).json({ error: error.message });  
  }  
}
```

```
// Update a client  
public async updateClient(req: Request, res: Response) {  
  const { id: pk } = req.params;  
  const { id, name, address, phone, email, password, status } = req.body;  
  try {  
    let body: ClientI = {  
      name,  
      address,  
      phone,  
      email,  
      password,  
      status,  
    };  
    ;
```

```
    const clientExist = await Client.findOne({  
      where: {  
        id: pk,  
        status: 'ACTIVE' },  
    });
```

```
    if (clientExist) {  
      await clientExist.update(body, {  
        where: { id: pk },  
      });  
      res.status(200).json(clientExist);  
    } else {  
      res.status(404).json({ error: "Client not found or inactive" });  
    }  
  } catch (error: any) {  
    res.status(400).json({ error: error.message });  
  }  
}
```

```
// Delete a client physically  
public async deleteClient(req: Request, res: Response) {  
  try {  
    const { id } = req.params;  
    const clientToDelete = await Client.findByPk(id);  
  
    if (clientToDelete) {  
      await clientToDelete.destroy();  
      res.status(200).json({ message: "Client deleted successfully" });  
    } else {  
      res.status(404).json({ error: "Client not found" });  
    }  
  } catch (error) {  
    res.status(500).json({ error: "Error deleting client" });  
  }  
}
```

```
// Delete a client logically (change status to "INACTIVE")  
public async deleteClientAdv(req: Request, res: Response) {
```

```

    try {
      const { id: pk } = req.params;
      const clientToUpdate = await Client.findOne({
        where: {
          id: pk,
          status: 'ACTIVE' },
      });

      if (clientToUpdate) {
        await clientToUpdate.update({ status: 'INACTIVE' });
        res.status(200).json({ message: "Client marked as inactive" });
      } else {
        res.status(404).json({ error: "Client not found" });
      }
    } catch (error) {
      res.status(500).json({ error: "Error marking client as inactive" });
    }
  }
}

```

1.2.2 CONTROLADOR PRODUCTTYPE

src/controllers/productType.controller.ts

```

import { Request, Response } from "express";
import { ProductType, ProductTypeI } from "../models/ProductType";

export class ProductTypeController {
  // Get all product types
  public async getAllProductTypes(req: Request, res: Response) {
    try {
      const productTypes: ProductTypeI[] = await ProductType.findAll({
        where: { status: true },
      });
      res.status(200).json({ productTypes });
    } catch (error) {
      res.status(500).json({ error: "Error fetching product types" });
    }
  }

  // Get a product type by ID
  public async getProductById(req: Request, res: Response) {
    try {
      const { id: pk } = req.params;
      const productType = await ProductType.findOne({
        where: { id: pk, status: true },
      });
      if (productType) {
        res.status(200).json(productType);
      } else {
        res.status(404).json({ error: "Product type not found or inactive" });
      }
    } catch (error) {
      res.status(500).json({ error: "Error fetching product type" });
    }
  }

  // Create a new product type
  public async createProductType(req: Request, res: Response) {
    const { name, description, status } = req.body;
    try {
      let body: ProductTypeI = {
        name,

```

```
    description,  
    status,  
  });  
}
```

```
    const newProductType = await ProductType.create({ ...body });  
    res.status(201).json(newProductType);  
  } catch (error: any) {  
    res.status(400).json({ error: error.message });  
  }  
}
```

// Update a product type

```
public async updateProductType(req: Request, res: Response) {  
  const { id: pk } = req.params;  
  const { name, description, status } = req.body;  
  try {  
    let body: ProductTypeI = {  
      name,  
      description,  
      status,  
    };  
    
```

```
    const productTypeExist = await ProductType.findOne({  
      where: { id: pk, status: true },  
    });  
  
```

```
    if (productTypeExist) {  
      await productTypeExist.update(body, {  
        where: { id: pk },  
      });  
      res.status(200).json(productTypeExist);  
    } else {  
      res.status(404).json({ error: "Product type not found or inactive" });  
    }  
  } catch (error: any) {  
    res.status(400).json({ error: error.message });  
  }  
}
```

// Delete a product type physically

```
public async deleteProductType(req: Request, res: Response) {  
  try {  
    const { id } = req.params;  
    const productTypeToDelete = await ProductType.findByPk(id);  
  
```

```
    if (productTypeToDelete) {  
      await productTypeToDelete.destroy();  
      res.status(200).json({ message: "Product type deleted successfully" });  
    } else {  
      res.status(404).json({ error: "Product type not found" });  
    }  
  } catch (error) {  
    res.status(500).json({ error: "Error deleting product type" });  
  }  
}
```

// Delete a product type logically (change status to false)

```
public async deleteProductTypeAdv(req: Request, res: Response) {  
  try {  
    const { id: pk } = req.params;  
    const productTypeToUpdate = await ProductType.findOne({  
      where: { id: pk, status: true },  
    });  
  
```

```
    if (productTypeToUpdate) {
```

```

        await productTypeToUpdate.update({ status: false });
        res.status(200).json({ message: "Product type marked as inactive" });
    } else {
        res.status(404).json({ error: "Product type not found" });
    }
} catch (error) {
    res.status(500).json({ error: "Error marking product type as inactive" });
}
}
}

```

1.2.3 CONTROLADOR PRODUCT

src/controllers/product.controller.ts

```

import { Request, Response } from "express";
import { Product, ProductI } from "../models/Product";

export class ProductController {
    // Get all products with status "ACTIVE"
    public async getAllProducts(req: Request, res: Response) {
        try {
            const products: ProductI[] = await Product.findAll({
                where: { status: "ACTIVE" },
            });
            res.status(200).json({ products });
        } catch (error) {
            res.status(500).json({ error: "Error fetching products" });
        }
    }

    // Get a product by ID
    public async getProductById(req: Request, res: Response) {
        try {
            const { id: pk } = req.params;
            const product = await Product.findOne({
                where: { id: pk, status: "ACTIVE" },
            });
            if (product) {
                res.status(200).json(product);
            } else {
                res.status(404).json({ error: "Product not found or inactive" });
            }
        } catch (error) {
            res.status(500).json({ error: "Error fetching product" });
        }
    }

    // Create a new product
    public async createProduct(req: Request, res: Response) {
        const { id, name, brand, price, min_stock, quantity, product_type_id, status } = req.body;
        try {
            let body: ProductI = {
                name,
                brand,
                price,
                min_stock,
                quantity,
                product_type_id,
                status,
            };

            const newProduct = await Product.create({ ...body });

```

```

    res.status(201).json(newProduct);
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

```

// Update a product
public async updateProduct(req: Request, res: Response) {
  const { id: pk } = req.params;
  const { id,name, brand, price, min_stock, quantity, product_type_id, status } = req.body;
  try {
    let body: ProductI = {
      name,
      brand,
      price,
      min_stock,
      quantity,
      product_type_id,
      status,
    };

```

```

    const productExist = await Product.findOne({
      where: { id: pk, status: "ACTIVE" },
    });

```

```

    if (productExist) {
      await productExist.update(body, {
        where: { id: pk },
      });
      res.status(200).json(productExist);
    } else {
      res.status(404).json({ error: "Product not found or inactive" });
    }
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

```

// Delete a product physically
public async deleteProduct(req: Request, res: Response) {
  try {
    const { id } = req.params;
    const productToDelete = await Product.findByPk(id);

```

```

    if (productToDelete) {
      await productToDelete.destroy();
      res.status(200).json({ message: "Product deleted successfully" });
    } else {
      res.status(404).json({ error: "Product not found" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error deleting product" });
  }
}

```

```

// Delete a product logically (change status to "INACTIVE")
public async deleteProductAdv(req: Request, res: Response) {
  try {
    const { id: pk } = req.params;
    const productToUpdate = await Product.findOne({
      where: { id: pk, status: "ACTIVE" },
    });

```

```

    if (productToUpdate) {
      await productToUpdate.update({ status: "INACTIVE" });

```

```

        res.status(200).json({ message: "Product marked as inactive" });
    } else {
        res.status(404).json({ error: "Product not found" });
    }
} catch (error) {
    res.status(500).json({ error: "Error marking product as inactive" });
}
}
}

```

1.2.4 CONTROLADOR SALE

src/controllers/sale.controller.ts

```

import { Request, Response } from "express";
import { Sale, SaleI } from "../models/Sale";

export class SaleController {
    // Get all sales with status "ACTIVE"
    public async getAllSales(req: Request, res: Response) {
        try {
            const sales: SaleI[] = await Sale.findAll({
                where: { status: "ACTIVE" },
            });
            res.status(200).json({ sales });
        } catch (error) {
            res.status(500).json({ error: "Error fetching sales" });
        }
    }

    // Get a sale by ID
    public async getSaleById(req: Request, res: Response) {
        try {
            const { id: pk } = req.params;
            const sale = await Sale.findOne({
                where: {
                    id: pk,
                    status: "ACTIVE" },
            });
            if (sale) {
                res.status(200).json(sale);
            } else {
                res.status(404).json({ error: "Sale not found or inactive" });
            }
        } catch (error) {
            res.status(500).json({ error: "Error fetching sale" });
        }
    }

    // Create a new sale
    public async createSale(req: Request, res: Response) {
        const { id, sale_date, subtotal, tax, discounts, total, status, client_id } = req.body;
        try {
            let body: SaleI = {
                sale_date,
                subtotal,
                tax,
                discounts,
                total,
                status,
                client_id,
            };
            console.log(body)

```

```

    const newSale = await Sale.create({ ...body });
    res.status(201).json(newSale);
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

// Update a sale
public async updateSale(req: Request, res: Response) {
  const { id: pk } = req.params;
  const { id, sale_date, subtotal, tax, discounts, total, status, client_id } = req.body;
  try {
    let body: SaleI = {
      sale_date,
      subtotal,
      tax,
      discounts,
      total,
      status,
      client_id,
    };

    const saleExist = await Sale.findOne({
      where: {
        id: pk,
        status: "ACTIVE" },
    });

    if (saleExist) {
      await saleExist.update(body, {
        where: { id: pk },
      });
      res.status(200).json(saleExist);
    } else {
      res.status(404).json({ error: "Sale not found or inactive" });
    }
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

// Delete a sale physically
public async deleteSale(req: Request, res: Response) {
  try {
    const { id } = req.params;
    const saleToDelete = await Sale.findByPk(id);

    if (saleToDelete) {
      await saleToDelete.destroy();
      res.status(200).json({ message: "Sale deleted successfully" });
    } else {
      res.status(404).json({ error: "Sale not found" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error deleting sale" });
  }
}

// Delete a sale logically (change status to "INACTIVE")
public async deleteSaleAdv(req: Request, res: Response) {
  try {
    const { id: pk } = req.params;
    const saleToUpdate = await Sale.findOne({
      where: {

```



```

        id: pk,
        status: "ACTIVE" },
    });

    if (saleToUpdate) {
        await saleToUpdate.update({ status: "INACTIVE" });
        res.status(200).json({ message: "Sale marked as inactive" });
    } else {
        res.status(404).json({ error: "Sale not found" });
    }
} catch (error) {
    res.status(500).json({ error: "Error marking sale as inactive" });
}
}
}

```

1.2.5 CONTROLADOR PRODUCTSALE

src/controllers/productSale.controller.ts

```

import { Request, Response } from "express";
import { ProductSale, ProductSaleI } from "../models/ProductSale";

export class ProductSaleController {
    // Get all product sales
    public async getAllProductSales(req: Request, res: Response) {
        try {
            const productSales: ProductSaleI[] = await ProductSale.findAll({
                where: { status: true },
            });
            res.status(200).json({ productSales });
        } catch (error) {
            res.status(500).json({ error: "Error fetching product sales" });
        }
    }

    // Get a product sale by ID
    public async getProductSaleById(req: Request, res: Response) {
        try {
            const { id: pk } = req.params;
            const productSale = await ProductSale.findOne({
                where: { id: pk, status: true },
            });
            if (productSale) {
                res.status(200).json(productSale);
            } else {
                res.status(404).json({ error: "Product sale not found or inactive" });
            }
        } catch (error) {
            res.status(500).json({ error: "Error fetching product sale" });
        }
    }

    // Create a new product sale
    public async createProductSale(req: Request, res: Response) {
        const { total, sale_id, product_id } = req.body;
        try {
            let body: ProductSaleI = {
                total,
                sale_id,
                product_id,
            };
        }
    }
}

```

```

    const newProductSale = await ProductSale.create({ ...body });
    res.status(201).json(newProductSale);
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

// Update a product sale

```

public async updateProductSale(req: Request, res: Response) {
  const { id: pk } = req.params;
  const { total, sale_id, product_id } = req.body;
  try {
    let body: ProductSaleI = {
      total,
      sale_id,
      product_id,
    };

```

```

    const productSaleExist = await ProductSale.findOne({
      where: { id: pk, status: true },
    });

```

```

    if (productSaleExist) {
      await productSaleExist.update(body, {
        where: { id: pk },
      });
      res.status(200).json(productSaleExist);
    } else {
      res.status(404).json({ error: "Product sale not found or inactive" });
    }
  } catch (error: any) {
    res.status(400).json({ error: error.message });
  }
}

```

// Delete a product sale physically

```

public async deleteProductSale(req: Request, res: Response) {
  try {
    const { id } = req.params;
    const productSaleToDelete = await ProductSale.findByPk(id);

```

```

    if (productSaleToDelete) {
      await productSaleToDelete.destroy();
      res.status(200).json({ message: "Product sale deleted successfully" });
    } else {
      res.status(404).json({ error: "Product sale not found" });
    }
  } catch (error) {
    res.status(500).json({ error: "Error deleting product sale" });
  }
}

```

// Delete a product sale logically (change status to false)

```

public async deleteProductSaleAdv(req: Request, res: Response) {
  try {
    const { id: pk } = req.params;
    const productSaleToUpdate = await ProductSale.findOne({
      where: { id: pk, status: true },
    });

```

```

    if (productSaleToUpdate) {
      await productSaleToUpdate.update({ status: false });
      res.status(200).json({ message: "Product sale marked as inactive" });
    } else {
      res.status(404).json({ error: "Product sale not found" });
    }

```

```

    }
  } catch (error) {
    res.status(500).json({ error: "Error marking product sale as inactive" });
  }
}
}

```

Paso 2: Rutas

2.1 Rutas de los modelos de Autorizacion

1.2.1 RUTAS DE USER

src/routes/authorization/user.ts

```

import { Application } from "express";
import { UserController } from '../../controllers/authorization/user.controller';

export class UserRoutes {
  public userController: UserController = new UserController();

  public routes(app: Application): void {
    app.route("/api/users").get(this.userController.getAllUsers);
    app.route("/api/users").post(this.userController.getAllUsers);
  }
}

```

1.2.2 RUTAS DE ROLE

src/routes/authorization/role.ts

```

import { Application } from "express";
import { RoleController } from '../../controllers/authorization/role.controller';

export class RoleRoutes {
  public roleController: RoleController = new RoleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/roles")
      .get(this.roleController.getAllRoles)
      .post(this.roleController.createRole);

    app.route("/api/roles/:id")
      .get(this.roleController.getRoleById)
      .patch(this.roleController.updateRole)
      .delete(this.roleController.deleteRole);

    app.route("/api/roles/:id/logic")
      .delete(this.roleController.deleteRoleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}

```

1.2.3 RUTAS DE ROLEUSER

src/routes/authorization/role_user.ts

```
import { Application } from "express";
import { RoleUserController } from '../controllers/authorization/role_user.controller';

export class RoleUserRoutes {
  public roleUserController: RoleUserController = new RoleUserController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/roleUsers")
      .get(this.roleUserController.getAllRoleUsers)
      .post(this.roleUserController.createRoleUser);

    app.route("/api/roleUsers/:id")
      .get(this.roleUserController.getRoleUserById)
      .patch(this.roleUserController.updateRoleUser)
      .delete(this.roleUserController.deleteRoleUser);

    app.route("/api/roleUsers/:id/logic")
      .delete(this.roleUserController.deleteRoleUserAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====

  }
}
```

1.2.4 RUTAS DE RESOURCE

src/routes/authorization/resource.ts

```
import { Application } from "express";
import { ResourceController } from '../controllers/authorization/resource.controller';

export class ResourceRoutes {
  public resourceController: ResourceController = new ResourceController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/resources")
      .get(this.resourceController.getAllResources)
      .post(this.resourceController.createResource);

    app.route("/api/resources/:id")
      .get(this.resourceController.getResourceById)
      .patch(this.resourceController.updateResource)
      .delete(this.resourceController.deleteResource);

    app.route("/api/resources/:id/logic")
      .delete(this.resourceController.deleteResourceAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

1.2.5 RUTAS DE RESOURCE_ROLE

src/routes/authorization/resourceRole.ts

```
import { Application } from "express";
import { ResourceRoleController } from
"../../controllers/authorization/resourceRole.controller";

export class ResourceRoleRoutes {
  public resourceRoleController: ResourceRoleController = new ResourceRoleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/resource-roles")
      .get(this.resourceRoleController.getAllResourceRoles)
      .post(this.resourceRoleController.createResourceRole);

    app.route("/api/resource-roles/:id")
      .get(this.resourceRoleController.getResourceRoleById)
      .patch(this.resourceRoleController.updateResourceRole)
      .delete(this.resourceRoleController.deleteResourceRole);

    app.route("/api/resource-roles/:id/logic")
      .delete(this.resourceRoleController.deleteResourceRoleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

1.2.6 RUTAS DE REFRESHTOKEN

src/routes/authorization/refresh_token.ts

```
import { Application } from "express";
import { RefreshTokenController } from
"../../controllers/authorization/refresh_token.controller";

export class RefreshTokenRoutes {
  public refreshTokenController: RefreshTokenController = new RefreshTokenController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/refresh-token")
      .get(this.refreshTokenController.getAllRefreshToken);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

2.2 Rutas de los modelos de Negocio

2.2.1 RUTAS DE CLIENT

src/routes/client.ts

```
import { Router, Application } from "express";
import { ClientController } from "../controllers/client.controller";

export class ClientRoutes {
  public clientController: ClientController = new ClientController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/clientes")
      .get(this.clientController.getAllClients)
      .post(this.clientController.createClient);

    app.route("/api/clientes/:id")
      .get(this.clientController.getClientById)
      .patch(this.clientController.updateClient)
      .delete(this.clientController.deleteClient);

    app.route("/api/clientes/:id/logic")
      .delete(this.clientController.deleteClientAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====

  }
}
```

2.2.2 RUTAS DE PRODUCTTYPE

src/routes/productType.ts

```
import { Application } from "express";
import { ProductTypeController } from "../controllers/productType.controller";

export class ProductTypeRoutes {
  public productTypeController: ProductTypeController = new ProductTypeController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/product-types")
      .get(this.productTypeController.getAllProductTypes)
      .post(this.productTypeController.createProductType);

    app.route("/product-types/:id")
      .get(this.productTypeController.getProductTypeById)
      .patch(this.productTypeController.updateProductType)
      .delete(this.productTypeController.deleteProductType);

    app.route("/product-types/:id/logic")
      .delete(this.productTypeController.deleteProductTypeAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

2.2.3 RUTAS DE PRODUCT

src/routes/product.ts

```
import { Application } from "express";
import { ProductController } from "../controllers/product.controller";

export class ProductRoutes {
  public productController: ProductController = new ProductController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/products")
      .get(this.productController.getAllProducts)
      .post(this.productController.createProduct);

    app.route("/products/:id")
      .get(this.productController.getProductById)
      .patch(this.productController.updateProduct)
      .delete(this.productController.deleteProduct);

    app.route("/products/:id/logic")
      .delete(this.productController.deleteProductAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

2.2.4 RUTAS DE SALE

src/routes/sale.ts

```
import { Application } from "express";
import { SaleController } from "../controllers/sale.controller";

export class SaleRoutes {
  public saleController: SaleController = new SaleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/ventas")
      .get(this.saleController.getAllSales)
      .post(this.saleController.createSale);

    app.route("/ventas/:id")
      .get(this.saleController.getSaleById)
      .patch(this.saleController.updateSale)
      .delete(this.saleController.deleteSale);

    app.route("/ventas/:id/logic")
      .delete(this.saleController.deleteSaleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

2.2.5 RUTAS DE PRODUCTSALE

src/routes/productSale.ts

```
import { Application } from "express";
import { ProductSaleController } from "../controllers/productSale.controller";

export class ProductSaleRoutes {
  public productSaleController: ProductSaleController = new ProductSaleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/product-sales")
      .get(this.productSaleController.getAllProductSales)
      .post(this.productSaleController.createProductSale);

    app.route("/product-sales/:id")
      .get(this.productSaleController.getProductSaleById)
      .patch(this.productSaleController.updateProductSale)
      .delete(this.productSaleController.deleteProductSale);

    app.route("/product-sales/:id/logic")
      .delete(this.productSaleController.deleteProductSaleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    // Si se requieren rutas protegidas, se pueden agregar aquí:

  }
}
```

2.2 Agregar Rutas a Routes/Index

src/routes/index.ts

```
import { Router } from "express";
import { ClientRoutes } from "./client";
import { SaleRoutes } from "./sale";
import { ProductRoutes } from "./product";
import { ProductTypeRoutes } from "./productType";
import { ProductSaleRoutes } from "./productSale";
import { UserRoutes } from "./authorization/user";
import { RoleRoutes } from "./authorization/role";
import { RoleUserRoutes } from "./authorization/role_user";
import { RefreshTokenRoutes } from "./authorization/refresh_token";
import { ResourceRoutes } from "./authorization/resource"; // Import ResourceRoutes
import { ResourceRoleRoutes } from "./authorization/resourceRole"; // Import
ResourceRoleRoutes

export class Routes {
  public clientRoutes: ClientRoutes = new ClientRoutes();
  public saleRoutes: SaleRoutes = new SaleRoutes();
  public productRoutes: ProductRoutes = new ProductRoutes();
  public productTypeRoutes: ProductTypeRoutes = new ProductTypeRoutes();
  public productSaleRoutes: ProductSaleRoutes = new ProductSaleRoutes();
  public userRoutes: UserRoutes = new UserRoutes();
  public roleRoutes: RoleRoutes = new RoleRoutes();
  public roleUserRoutes: RoleUserRoutes = new RoleUserRoutes();
  public refreshTokenRoutes: RefreshTokenRoutes = new RefreshTokenRoutes();
  public resourceRoutes: ResourceRoutes = new ResourceRoutes(); // Add ResourceRoutes
  public resourceRoleRoutes: ResourceRoleRoutes = new ResourceRoleRoutes(); // Add
ResourceRoutes
}
```


2.3 Agregar Rutas a Config/Index

src/config/index.ts

Agregar modulo de private routes() y su importacion

```
import dotenv from "dotenv";
import express, { Application } from "express";
import morgan from "morgan";
import { sequelize, testConnection, getDatabaseInfo } from "../database/db";
import { Routes } from "../routes/index";

var cors = require("cors");

dotenv.config();

export class App {
  public app: Application;
  public routePrv: Routes = new Routes();

  constructor(private port?: number | string) {
    this.app = express();
    this.settings();
    this.middlewares();
    this.routes();
    this.dbConnection();
  }

  private settings(): void {
    this.app.set('port', this.port || process.env.PORT || 4000);
  }

  private middlewares(): void {
    this.app.use(morgan('dev'));
    this.app.use(cors());
    this.app.use(express.json());
    this.app.use(express.urlencoded({ extended: false }));
  }

  // Route configuration
  private routes(): void {
    this.routePrv.clientRoutes.routes(this.app);
    this.routePrv.saleRoutes.routes(this.app);
    this.routePrv.productRoutes.routes(this.app);
    this.routePrv.productTypeRoutes.routes(this.app);
    this.routePrv.productSaleRoutes.routes(this.app);
    this.routePrv.userRoutes.routes(this.app);
    this.routePrv.roleRoutes.routes(this.app);
    this.routePrv.roleUserRoutes.routes(this.app);
    this.routePrv.refreshTokenRoutes.routes(this.app);
    this.routePrv.resourceRoutes.routes(this.app);
    this.routePrv.resourceRoleRoutes.routes(this.app);
  }

  private async dbConnection(): Promise<void> {
    try {
      // Mostrar información de la base de datos seleccionada
      const dbInfo = getDatabaseInfo();
      console.log(`🔌 Intentando conectar a: ${dbInfo.engine.toUpperCase()}`);

      // Probar la conexión
      const isConnected = await testConnection();
    }
  }
}
```

```

    if (!isConnected) {
        throw new Error(`No se pudo conectar a la base de datos
        ${dbInfo.engine.toUpperCase()}`);
    }

    // Sincronizar la base de datos
    await sequelize.sync({ force: false });
    console.log(`📦 Base de datos sincronizada exitosamente`);

} catch (error) {
    console.error("❌ Error al conectar con la base de datos:", error);
    process.exit(1); // Terminar la aplicación si no se puede conectar
}

}

async listen() {
    await this.app.listen(this.app.get('port'));
    console.log(`🚀 Servidor ejecutándose en puerto ${this.app.get('port')}`);
}
}

```

Luego,

```

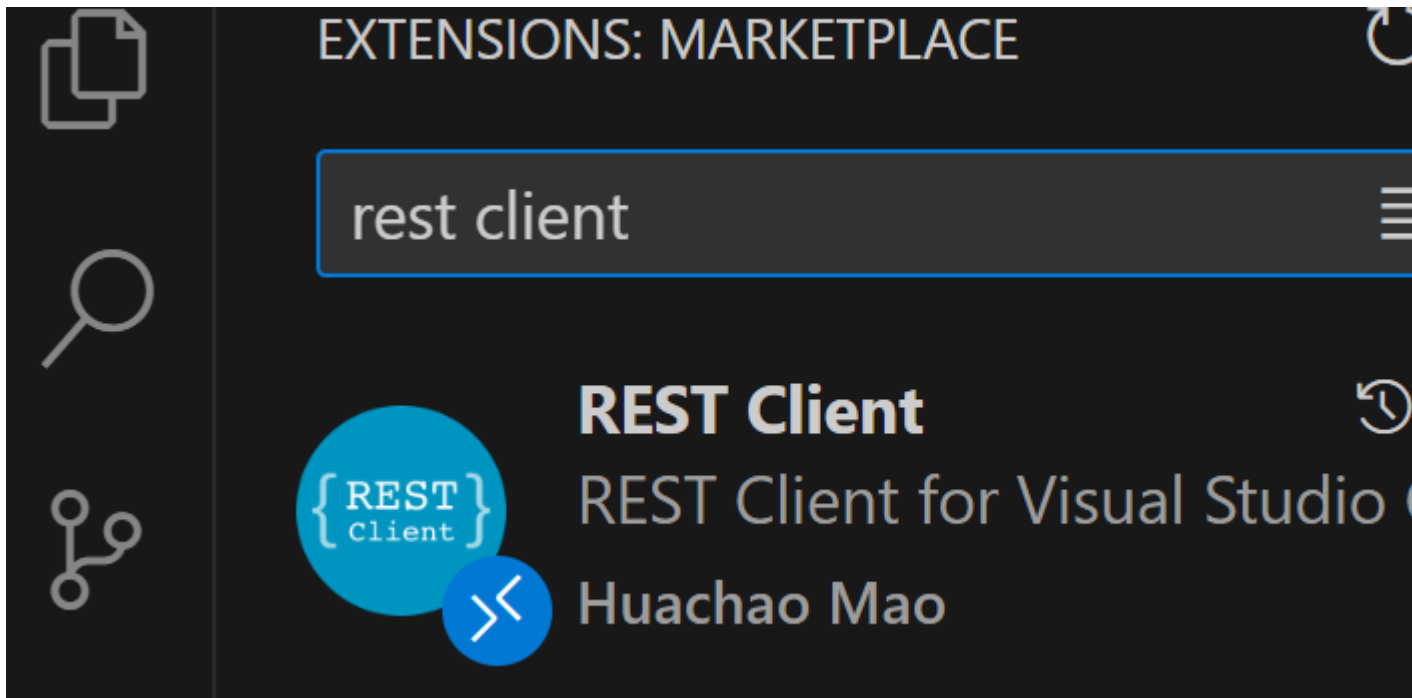
jaider@DESKTOP-Q338KL1:~/app/dlloweb/
[nodemon] starting `ts-node src/serve
[dotenv@17.2.3] injecting env (23) fr
🔌 Conectando a base de datos: ORACLE
🎈 Host: localhost:1521
📁 Base de datos: xe
[dotenv@17.2.3] injecting env (0) fro
🔗 Intentando conectar a: ORACLE
🚀 Servidor ejecutándose en puerto 40
✅ Conexión exitosa a ORACLE
📦 Base de datos sincronizada exitosa

```

2.4 Peticiones API REST

Para las peticiones API REST utilizaremos la extension en VS CODE RESTClient.

Instalar Extension



Luego, Crear carpeta src/http y crear archivo por modelo definidas en los archivos de rutas.

2.4.1 Http de los modelos de Autorizacion

2.4.1.1 HTTP DE DE USER

src/http/authorization/user.http

```
### user.http
```

```
@baseUrl = http://localhost:4000
```

```
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
### Esto es solo como ejemplo academico, dado que este modelo lo debemos manejar  
posteriormente con Auth
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los usuarios
```

```
GET {{baseUrl}}/api/users
```

```
### Crear usuario (usando el mismo endpoint)
```

```
POST {{baseUrl}}/api/users
```

```
Content-Type: application/json
```

```
{  
  "username": "newuser",  
  "email": "newuser@example.com",
```

```
"password": "123456",
"is_active": "ACTIVE",
"avatar": "avatar.jpg"
}
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

2.4.1.2 HTTP DE DE ROLE

src/http/authorization/role.http

```
### role.http
```

```
@baseUrl = http://localhost:4000
```

```
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los roles
```

```
GET {{baseUrl}}/api/roles
```

```
### Obtener rol por ID
```

```
GET {{baseUrl}}/api/roles/1
```

```
### Crear un nuevo rol
```

```
POST {{baseUrl}}/api/roles
```

```
Content-Type: application/json
```

```
{
  "name": "Administrator",
  "is_active": "ACTIVE"
}
```

```
### Actualizar un rol
```

```
PATCH {{baseUrl}}/api/roles/1
```

```
Content-Type: application/json
```

```
{
  "name": "Super Administrator",
  "is_active": "ACTIVE"
}
```

```
### Eliminar un rol físicamente
```

```
DELETE {{baseUrl}}/api/roles/1
```

```
### Eliminar un rol lógicamente
```

```
DELETE {{baseUrl}}/api/roles/1/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

2.4.1.3 HTTP DE DE ROLEUSER

src/http/authorization/role_user.http

```
### roleUser.http
```

```
@baseUrl = http://localhost:4000
```

```
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```

### Obtener todos los roles de usuario
GET {{baseUrl}}/api/roleUsers

### Obtener rol de usuario por ID
GET {{baseUrl}}/api/roleUsers/1

### Crear un nuevo rol de usuario
POST {{baseUrl}}/api/roleUsers
Content-Type: application/json

{
  "role_id": 1,
  "user_id": 1,
  "is_active": "ACTIVE"
}

### Actualizar un rol de usuario
PATCH {{baseUrl}}/api/roleUsers/1
Content-Type: application/json

{
  "role_id": 2,
  "user_id": 1,
  "is_active": "ACTIVE"
}

### Eliminar un rol de usuario físicamente
DELETE {{baseUrl}}/api/roleUsers/1

### Eliminar un rol de usuario lógicamente
DELETE {{baseUrl}}/api/roleUsers/1/logic

### ===== RUTAS CON AUTENTICACIÓN =====

```

2.4.1.4 HTTP DE DE RESOURCE

src/http/authorization/resource.http

```

### resource.http

@baseUrl = http://localhost:4000
@token =eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

### ===== RUTAS SIN AUTENTICACIÓN =====

### Obtener todos los recursos
GET {{baseUrl}}/api/resources

### Obtener recurso por ID
GET {{baseUrl}}/api/resources/1

### Crear un nuevo recurso
POST {{baseUrl}}/api/resources
Content-Type: application/json

{
  "path": "/api/clientes",
  "method": "GET",
  "is_active": "ACTIVE"
}

### Actualizar un recurso
PATCH {{baseUrl}}/api/resources/1

```

Content-Type: `application/json`

```
{
  "path": "/api/clientes/:id",
  "method": "GET",
  "is_active": "ACTIVE"
}
```

Eliminar un recurso físicamente
DELETE {{baseUrl}}/api/resources/**1**

Eliminar un recurso lógicamente
DELETE {{baseUrl}}/api/resources/**1**/logic

===== RUTAS CON AUTENTICACIÓN =====

2.4.1.5 HTTP DE DE RESOURCEROLE

`src/http/authorization/resourceRole.http`

resourceRole.http

@baseUrl = http://localhost:4000

@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

===== RUTAS SIN AUTENTICACIÓN =====

Obtener todos los roles de recursos
GET {{baseUrl}}/api/resource-roles

Obtener rol de recurso por ID
GET {{baseUrl}}/api/resource-roles/**1**

Crear un nuevo rol de recurso
POST {{baseUrl}}/api/resource-roles
Content-Type: `application/json`

```
{
  "resource_id": 1,
  "role_id": 1,
  "is_active": "ACTIVE"
}
```

Actualizar un rol de recurso
PATCH {{baseUrl}}/api/resource-roles/**1**
Content-Type: `application/json`

```
{
  "resource_id": 1,
  "role_id": 2,
  "is_active": "ACTIVE"
}
```

Eliminar un rol de recurso físicamente
DELETE {{baseUrl}}/api/resource-roles/**1**

Eliminar un rol de recurso lógicamente
DELETE {{baseUrl}}/api/resource-roles/**1**/logic

===== RUTAS CON AUTENTICACIÓN =====

2.4.1.6 HTTP DE DE REFRESHTOKEN

src/http/authorization/refreshToken.http

```
### refreshToken.http

@baseUrl = http://localhost:4000
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

### ===== RUTAS SIN AUTENTICACIÓN =====

### Obtener todos los refresh tokens
GET {{baseUrl}}/refresh-token

### ===== RUTAS CON AUTENTICACIÓN =====
```

2.4.2 Http de los modelos de Negocio

2.4.2.1 HTTP DE DE CLIENT

src/http/client.http

```
// client.http
import dotenv from "dotenv";

@token =
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6OCwiaWF0IjoxNzU5OTQyNjMxLCJleHAiOiE3NTk5NDMyMzF9.
EAhQ6CjPXiwsq-hbvUSn9EXKF4CXcD-Q9IYMYlc0h3k
@baseUrl = http://localhost:4000

### Peticiones sin TOKEN

### Obtener todos los clientes
GET {{baseUrl}}/api/clientes

### Obtener cliente por ID
GET {{baseUrl}}/api/clientes/1

### Crear un nuevo cliente
POST {{baseUrl}}/api/clientes
Content-Type: application/json

{
  "name": "Jaider Quintero",
  "address": "Km 1",
  "phone": "3001234567",
  "email": "jqm@gmail.com",
  "password": "12345678",
  "status": "ACTIVE"
}

### Actualizar un cliente
PATCH {{baseUrl}}/api/clientes/1
Content-Type: application/json

{
  "name": "Juan Perez Actualizado",
  "address": "Calle 456",
  "phone": "3007654321",
  "email": "juan.perez@example.com",
  "password": "654321",
}
```

```
    "status": "ACTIVE"
}
```

```
### Eliminar un cliente físicamente
DELETE {{baseUrl}}/api/clientes/1
```

```
### Eliminar un cliente lógicamente
DELETE {{baseUrl}}/api/clientes/1/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

2.4.2.2 HTTP DE DE PRODUCTTYPE

src/http/productType.http

```
### productType.http
```

```
@baseUrl = http://localhost:3000
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los tipos de producto
GET {{baseUrl}}/product-types
```

```
### Obtener tipo de producto por ID
GET {{baseUrl}}/product-types/1
```

```
### Crear un nuevo tipo de producto
POST {{baseUrl}}/product-types
Content-Type: application/json
```

```
{
  "name": "Electrónicos",
  "description": "Productos electrónicos y tecnológicos",
  "status": "ACTIVE"
}
```

```
### Actualizar un tipo de producto
PATCH {{baseUrl}}/product-types/1
Content-Type: application/json
```

```
{
  "name": "Electrónicos Actualizados",
  "description": "Productos electrónicos y tecnológicos actualizados",
  "status": "ACTIVE"
}
```

```
### Eliminar un tipo de producto físicamente
DELETE {{baseUrl}}/product-types/1
```

```
### Eliminar un tipo de producto lógicamente
DELETE {{baseUrl}}/product-types/1/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

2.4.2.3 HTTP DE DE PRODUCT

src/http/product.http

product.http

@baseUrl = http://localhost:3000

@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

===== RUTAS SIN AUTENTICACIÓN =====

Obtener todos los productos

GET {{baseUrl}}/products

Obtener producto por ID

GET {{baseUrl}}/products/1

Crear un nuevo producto

POST {{baseUrl}}/products

Content-Type: application/json

```
{
  "name": "Laptop HP",
  "brand": "HP",
  "price": 1500000,
  "min_stock": 5,
  "quantity": 10,
  "product_type_id": 1,
  "status": "ACTIVE"
}
```

Actualizar un producto

PATCH {{baseUrl}}/products/1

Content-Type: application/json

```
{
  "name": "Laptop HP Actualizada",
  "brand": "HP",
  "price": 1600000,
  "min_stock": 3,
  "quantity": 8,
  "product_type_id": 1,
  "status": "ACTIVE"
}
```

Eliminar un producto físicamente

DELETE {{baseUrl}}/products/1

Eliminar un producto lógicamente

DELETE {{baseUrl}}/products/1/logic

===== RUTAS CON AUTENTICACIÓN =====

2.4.2.4 HTTP DE DE SALE

src/http/sale.http

sale.http

@baseUrl = http://localhost:3000

@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

===== RUTAS SIN AUTENTICACIÓN =====

Obtener todas las ventas

```

GET {{baseUrl}}/ventas

### Obtener venta por ID
GET {{baseUrl}}/ventas/1

### Crear una nueva venta
POST {{baseUrl}}/ventas
Content-Type: application/json

{
  "sale_date": "2024-01-15T10:30:00.000Z",
  "subtotal": 10000,
  "tax": 19000,
  "discounts": 5000,
  "total": 114000,
  "status": "ACTIVE",
  "client_id": 1
}

### Actualizar una venta
PATCH {{baseUrl}}/ventas/1
Content-Type: application/json

{
  "sale_date": "2024-01-15T10:30:00.000Z",
  "subtotal": 120000,
  "tax": 22800,
  "discounts": 10000,
  "total": 132800,
  "status": "ACTIVE",
  "client_id": 1
}

### Eliminar una venta físicamente
DELETE {{baseUrl}}/ventas/1

### Eliminar una venta lógicamente
DELETE {{baseUrl}}/ventas/1/logic

### ===== RUTAS CON AUTENTICACIÓN =====

```

2.4.2.5 HTTP DE DE PRODUCTSALE

src/http/productSale.http

```

### productSale.http

@baseUrl = http://localhost:3000
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

### ===== RUTAS SIN AUTENTICACIÓN =====

### Obtener todas las ventas de productos
GET {{baseUrl}}/product-sales

### Obtener venta de producto por ID
GET {{baseUrl}}/product-sales/1

### Crear una nueva venta de producto
POST {{baseUrl}}/product-sales
Content-Type: application/json

{

```

```

    "total": 1500000,
    "sale_id": 1,
    "product_id": 1
  }

  ### Actualizar una venta de producto
  PATCH {{baseUrl}}/product-sales/1
  Content-Type: application/json

  {
    "total": 1600000,
    "sale_id": 1,
    "product_id": 1
  }

  ### Eliminar una venta de producto físicamente
  DELETE {{baseUrl}}/product-sales/1

  ### Eliminar una venta de producto lógicamente
  DELETE {{baseUrl}}/product-sales/1/logic

  ### ===== RUTAS CON AUTENTICACIÓN =====

```

2.4 Como Probar las Peticiones API REST

Tomaremos como ejemplo las Rutas del Modelo Client

Para ello visualizamos dos archivos

- `src/http/client.http`
- `src/routes/client.ts`

En el archivo `client.http` se encuentran todas las rutas definidas en `src/routes/client.ts`

Se procede así:

≡ client.http ×

src > http > ≡ client.http > ...

```
1  // client.http
   Send Request
2  import dotenv from "dotenv";
3
4
5  @token = eyJhbGciOiJIUzI1NiI
6  @baseUrl = http://localhost:
7
8  ### Peticiones sin TOKEN
9
10 ### Obtener todos los clientes
   Send Request
11 GET {{baseUrl}}/api/clientes
12
13
14 ### Obtener cliente por ID
   Send Request
15 GET {{baseUrl}}/api/clientes
```



Y Asi se ve el resultado

≡ client.http ×

src > http > ≡ client.http > ...

```
1  // client.http
   Send Request
2  import dotenv from "dotenv";
3
4
5  @token = eyJhbGciOiJIUzI1NiIsI
6  @baseUrl = http://localhost:40
7
8  ### Peticiones sin TOKEN
9
10 ### Obtener todos los clientes
   Send Request
11 GET {{baseUrl}}/api/clientes
12
13
14 ### Obtener cliente por ID
   Send Request
15 GET {{baseUrl}}/api/clientes/1
```

Paso 3: Datos de Prueba (Opcional)

3.1 Instalar faker para datos de prueba

```
npm install @faker-js/faker
```

3.2 Crear script de población de datos

```
import { Client } from '../models/Client';
import { ProductType } from '../models/ProductType';
import { Product } from '../models/Product';
import { Sale } from '../models/Sale';
import { ProductSale } from '../models/ProductSale';
import { faker } from '@faker-js/faker';

async function createFakeData() {
  // Crear clientes falsos
  for (let i = 0; i < 50; i++) {
    await Client.create({
      name: faker.person.fullName(),
      address: faker.location.streetAddress(),
      phone: faker.phone.number(), // Genera un número de teléfono aleatorio
      email: faker.internet.email(),
      password: faker.internet.password(),
      status: 'ACTIVE',
    });
  }

  // Crear tipos de productos falsos
  for (let i = 0; i < 10; i++) {
    await ProductType.create({
      name: faker.commerce.department(),
      description: faker.commerce.productDescription(),
      status: 'ACTIVE',
    });
  }

  // Crear productos falsos
  const typeProducts = await ProductType.findAll();
  for (let i = 0; i < 20; i++) {
    await Product.create({
      name: faker.commerce.productName(),
      brand: faker.company.name(),
      price: faker.number.bigInt(),
      min_stock: faker.number.int({ min: 1, max: 10 }),
      quantity: faker.number.int({ min: 1, max: 100 }),
      product_type_id: typeProducts.length > 0
        ? typeProducts[faker.number.int({ min: 0, max: typeProducts.length - 1 })]?.id
        : null,
      status: 'ACTIVE',
    });
  }

  // Crear ventas falsas
  const clients = await Client.findAll();
  for (let i = 0; i < 100; i++) {
    await Sale.create({
      sale_date: faker.date.past(),
      subtotal: faker.number.bigInt(),
      tax: faker.number.bigInt(),
      discounts: faker.number.bigInt(),
      total: faker.number.bigInt(),
      status: 'ACTIVE',
    });
  }
}
```

```

        client_id: clients.length > 0
          ? clients[faker.number.int({ min: 0, max: clients.length - 1 })]?.id ?? null
          : null
      });
    }
  }

  // Crear productos ventas falsos
  const sales = await Sale.findAll();
  const products = await Product.findAll();
  for (let i = 0; i < 200; i++) {
    await ProductSale.create({
      total: faker.number.bigInt(),
      sale_id: sales[faker.number.int({ min: 0, max: sales.length - 1 })]?.id ?? null,
      product_id: products[faker.number.int({ min: 0, max: products.length - 1 })]?.id
    });
  }
}

createFakeData().then(() => {
  console.log('Datos falsos creados exitosamente');
}).catch((error) => {
  console.error('Error al crear datos falsos:', error);
});

// Para ejecutar este script, ejecute el siguiente comando:
// npm install -g ts-node
// ts-node src/faker/populate_data.ts
// npm install @faker-js/faker

```

Paso 4: Gitignore

1.1 Crear .gitignore

```

node_modules
.env
dist/
*.log
.DS_Store

```

Paso 5: Ejecución del Proyecto

5.1 Scripts de package.json

```

{
  "scripts": {
    "build": "tsc",
    "dev": "nodemon src/server.ts --exec ts-node",
    "start": "node dist/server.js",
    "populate": "ts-node src/faker/populate_data.ts"
  }
}

```

5.2 Ejecutar el proyecto

```

npm run dev    # Desarrollo
npm run build  # Producción

```



```
npm start
npm run populate # Poblar datos de prueba
```

Finalmente,

```
jaidier@DESKTOP-Q338KL1:~/app/dllowe
[nodemon] restarting due to changes
[nodemon] starting `ts-node src/ser
🔌 Conectando a base de datos: ORAC
📍 Host: localhost:1521
📁 Base de datos: xe
🔗 Intentando conectar a: ORACLE
🚀 Servidor ejecutándose en puerto
📁 Base de datos activa: ORACLE
✅ Conexión exitosa a ORACLE
📦 Base de datos sincronizada exito
```

Paso 6: Implementacion de Autenticacion y Autorizacion

6.1 Controlador Auth

Incluimos en los cotroladores un nuevo archivo para registrar usuarios y realizar login

```
src/controllers/Authorization/auth.controller.ts
```

```
import { Request, Response } from 'express';
import { User } from '../../models/authorization/User';

export class AuthController {
  public async register(req: Request, res: Response): Promise<void> {
```

```

    try {
      const { username, email, password, is_active, avatar } = req.body;
      const user_interface: User = await User.create({ username, email, password, is_active,
avatar });
      const token = user_interface.generateToken();
      res.status(201).json({ user_interface, token });
    } catch (error) {
      res.status(500).json({ error: 'Error al registrar el usuario' });
    }
  }
}

public async login(req: Request, res: Response): Promise<void> {
  try {
    const { email, password } = req.body;
    const user: User | null = await User.findOne(
      {
        where: {
          email,
          is_active: true
        }
      }
    );
    if (!user || !(await user.checkPassword(password))) {
      res.status(401).json({ error: 'Credenciales inválidas' });
      return;
    }
    const token = user.generateToken();
    res.status(200).json({ user, token });
  } catch (error) {
    res.status(500).json({ error: 'Error al iniciar sesión' });
  }
}
}

```

6.2 Ruta de Auth

src/routes/Authorization/auth.ts

```

import { Application } from "express";
import { AuthController } from '../controllers/authorization/auth.controller';

export class AuthRoutes {
  public authController: AuthController = new AuthController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/register")
      .post(this.authController.register);

    app.route("/api/login")
      .post(this.authController.login);

    // ===== RUTAS CON AUTENTICACIÓN =====
  }
}

```

Agregar estas rutas en

src/routes/Authorization/index.ts

```
.....  
.....  
.....
```

```
import { AuthRoutes } from "../authorization/auth"; // Add this import
```

```
export class Routes {  
    .....  
    .....
```

```
    public authRoutes: AuthRoutes = new AuthRoutes(); // Add this line  
}
```

debe quedar asi:

```
import { Router } from "express";  
import { ClientRoutes } from "../client";  
import { SaleRoutes } from "../sale";  
import { ProductRoutes } from "../product";  
import { ProductTypeRoutes } from "../productType";  
import { ProductSaleRoutes } from "../productSale";  
import { UserRoutes } from "../authorization/user";  
import { RoleRoutes } from "../authorization/role";  
import { RoleUserRoutes } from "../authorization/role_user";  
import { RefreshTokenRoutes } from "../authorization/refresk_token";  
import { ResourceRoutes } from "../authorization/resource"; // Import ResourceRoutes  
import { ResourceRoleRoutes } from "../authorization/resourceRole"; // Import  
ResourceRoleRoutes  
import { AuthRoutes } from "../authorization/auth"; // Add this import
```

```
export class Routes {  
    public clientRoutes: ClientRoutes = new ClientRoutes();  
    public saleRoutes: SaleRoutes = new SaleRoutes();  
    public productRoutes: ProductRoutes = new ProductRoutes();  
    public productTypeRoutes: ProductTypeRoutes = new ProductTypeRoutes();  
    public productSaleRoutes: ProductSaleRoutes = new ProductSaleRoutes();  
    public userRoutes: UserRoutes = new UserRoutes();  
    public roleRoutes: RoleRoutes = new RoleRoutes();  
    public roleUserRoutes: RoleUserRoutes = new RoleUserRoutes();  
    public refreshTokenRoutes: RefreshTokenRoutes = new RefreshTokenRoutes();  
    public resourceRoutes: ResourceRoutes = new ResourceRoutes(); // Add ResourceRoutes  
    public resourceRoleRoutes: ResourceRoleRoutes = new ResourceRoleRoutes(); // Add  
ResourceRoutes  
    public authRoutes: AuthRoutes = new AuthRoutes(); // Add this line  
}
```

y cerrar el ciclo actualizando las rutas en

src/config/index.http

```
// ...existing code...  
// Route configuration
```

```

.....
.....
.....
private routes(): void {
    .....
    .....
    .....
    .....

    this.routePrv.authRoutes.routes(this.app)
}

```

Codigo index completo

```

import dotenv from "dotenv";
import express, { Application } from "express";
import morgan from "morgan";
import { sequelize, testConnection, getDatabaseInfo } from "../database/db";
import { Routes } from "../routes/index";

var cors = require("cors");

dotenv.config();

export class App {
    public app: Application;
    public routePrv: Routes = new Routes();

    constructor(private port?: number | string) {
        this.app = express();
        this.settings();
        this.middlewares();
        this.routes();
        this.dbConnection();
    }

    private settings(): void {
        this.app.set('port', this.port || process.env.PORT || 4000);
    }

    private middlewares(): void {
        this.app.use(morgan('dev'));
        this.app.use(cors());
        this.app.use(express.json());
        this.app.use(express.urlencoded({ extended: false }));
    }

    // Route configuration
    private routes(): void {
        this.routePrv.clientRoutes.routes(this.app);
        this.routePrv.saleRoutes.routes(this.app);
        this.routePrv.productRoutes.routes(this.app);
        this.routePrv.productTypeRoutes.routes(this.app);
        this.routePrv.productSaleRoutes.routes(this.app);
        this.routePrv.userRoutes.routes(this.app);
        this.routePrv.roleRoutes.routes(this.app);
        this.routePrv.roleUserRoutes.routes(this.app);
        this.routePrv.refreshTokenRoutes.routes(this.app);
        this.routePrv.resourceRoutes.routes(this.app);
        this.routePrv.resourceRoleRoutes.routes(this.app);
        this.routePrv.authRoutes.routes(this.app)
    }
}

```

```

private async dbConnection(): Promise<void> {
  try {
    // Mostrar información de la base de datos seleccionada
    const dbInfo = getDatabaseInfo();
    console.log(`🔄 Intentando conectar a: ${dbInfo.engine.toUpperCase()}`);

    // Probar la conexión
    const isConnected = await testConnection();

    if (!isConnected) {
      throw new Error(`No se pudo conectar a la base de datos
      ${dbInfo.engine.toUpperCase()}`);
    }

    // Sincronizar la base de datos
    await sequelize.sync({ force: false });
    console.log(`✅ Base de datos sincronizada exitosamente`);

  } catch (error) {
    console.error("❌ Error al conectar con la base de datos:", error);
    process.exit(1); // Terminar la aplicación si no se puede conectar
  }
}

async listen() {
  await this.app.listen(this.app.get('port'));
  console.log(`🚀 Servidor ejecutándose en puerto ${this.app.get('port')}`);
}
}

```

6.3 Http Auth

src/http/Authorization/auth.http

auth.http

@baseUrl = http://localhost:4000

@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

===== RUTAS SIN AUTENTICACIÓN =====

Registrar un nuevo usuario

POST {{baseUrl}}/api/register

Content-Type: application/json

```

{
  "username": "jquintero",
  "email": "jq@gmail.com",
  "password": "12345678",
  "is_active": "ACTIVE",
  "avatar": "avatar.jpg"
}

```

Iniciar sesión

POST {{baseUrl}}/api/login

Content-Type: application/json

```

{
  "email": "jq@gmail.com",
  "password": "12345678"
}

```

===== RUTAS CON AUTENTICACIÓN =====

Probar Registrar Usuario

> http > authorization > ≡ auth.http >  POS

5

6 ### ===== RUTAS SIN

7

8 ### Registrar un nuevo usuario

Send Request

9 POST {{baseUrl}}/api/register

0 Content-Type: application/json

1

2 {

3 "username": "jqintero",

4 "email": "jq@gmail.com",

5 "password": "12345678",

6 "is_active": "ACTIVE",

7 "avatar": "avatar.jpg"

8 }

9

0 ### Iniciar sesión

Send Request

1 POST {{baseUrl}}/api/login

Probar Registrar Login

src > http > authorization >  auth.http >  P

18 }

19

20 ### Iniciar sesión

Send Request

21 POST {{baseUrl}}/api/login

22 Content-Type: application/json

23

24 {

25 "email": "jq@gmail.com",

26 "password": "12345678"

27 }

28

29 ### ===== RUTAS C

30

31

6.4 Configurar Middleware

Con el Middleware protegemos todas las rutas usando JWT TOKEN

src/middleware/auth.ts

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { RefreshToken } from '../models/authorization/RefreshToken';
import { User } from '../models/authorization/User';

export const authMiddleware = async (req: Request, res: Response, next: NextFunction):
Promise<void> => {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  const currentRoute = req.originalUrl;
  const currentMethod = req.method;

  console.log(currentRoute)
  if (!token) {
    res.status(401).json({ error: 'Acceso denegado: No se proporcionó el token principal.' });
    return;
  }

  try {
    // Verificar si el token principal es válido
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret') as jwt.JwtPayload;

    // Buscar el usuario en la base de datos
    const user: User | null = await User.findOne({ where: { id: decoded.id, is_active:
'ACTIVE' } });
    if (!user) {
      res.status(401).json({ error: 'Usuario no encontrado o inactivo.' });
      return;
    }

    // Continuar con la solicitud
    next();
  } catch (error: any) {
    if (error.name === 'TokenExpiredError') {
      res.status(401).json({ error: 'El token principal ha expirado.' });
    } else if (error.name === 'JsonWebTokenError') {
      res.status(401).json({ error: 'Token inválido.' });
    } else {
      res.status(500).json({ error: 'Error interno del servidor.', details: error.message });
    }
  }
};
```

Y para probar protegemos una ruta con token por ejemplo en cliente

src/routes/client.ts

```
import { Router, Application } from "express";
import { ClientController } from "../controllers/client.controller";
import { authMiddleware } from "../middleware/auth";

export class ClientRoutes {
  public clientController: ClientController = new ClientController();
```

```

public routes(app: Application): void {
  // ===== RUTAS SIN AUTENTICACIÓN =====
  app.route("/api/clientes/public")
    .get(this.clientController.getAllClients)
    .post(this.clientController.createClient);

  app.route("/api/clientes/public/:id")
    .get(this.clientController.getClientById)
    .patch(this.clientController.updateClient)
    .delete(this.clientController.deleteClient);

  app.route("/api/clientes/public/:id/logic")
    .delete(this.clientController.deleteClientAdv);

  // ===== RUTAS CON AUTENTICACIÓN =====

  app.route("/api/clientes")
    .get(authMiddleware, this.clientController.getAllClients)

}
}

```

Notese que cambie el end point de las rutas sin token las que tienen token para que no sean iguales y pueda realizar bien la protección de la ruta

Primero debemos registrar el usuario para obtener el token o en su defecto solo hacer el login si ya lo hemos registrado.

Luego en el http de client.http pegar el token obtenido y ejecutar la ruta protegida de mostrar todos los clientes

src/http/client.http

```

// client.http
import dotenv from "dotenv";

@token =
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMzMlhdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGKfFDKcOLHMmaVa12iOQKdrhZHRNnP6wb6dA
@baseUrl = http://localhost:4000

### Peticiones sin TOKEN

### Obtener todos los clientes
GET {{baseUrl}}/api/clientes/public

### Obtener cliente por ID
GET {{baseUrl}}/api/clientes/public/1

### Crear un nuevo cliente
POST {{baseUrl}}/api/clientes/public
Content-Type: application/json

{
  "name": "Jaider Quintero",
  "address": "Km 1",
  "phone": "3001234567",

```

```

    "email": "jqm@gmail.com",
    "password": "12345678",
    "status": "ACTIVE"
  }

```

```

### Actualizar un cliente
PATCH {{baseUrl}}/api/clientes/public/1
Content-Type: application/json

```

```

{
  "name": "Juan Perez Actualizado",
  "address": "Calle 456",
  "phone": "3007654321",
  "email": "juan.perez@example.com",
  "password": "654321",
  "status": "ACTIVE"
}

```

```

### Eliminar un cliente físicamente
DELETE {{baseUrl}}/api/clientes/public/1

```

```

### Eliminar un cliente lógicamente
DELETE {{baseUrl}}/api/clientes/public/1/logic

```

```

### ===== RUTAS CON AUTENTICACIÓN =====

```

```

### Obtener todos los clientes (CON token)
GET {{baseUrl}}/api/clientes
Authorization: Bearer {{token}}

```

Hacemos lo mismo para el resto de las rutas de clientes e igualmente para el resto de las rutas de los modelos

6.5 Autenticacion con Token Rutas Modelos Autorizacion

6.5.1 Ruta y Http User

No haremos cambio en User dado que lo mabejaremos con Auth

6.5.2 Ruta y Http Role

```

src/routes/authorization/role.ts

```

```

import { Application } from "express";
import { RoleController } from '../controllers/authorization/role.controller';
import { authMiddleware } from '../middleware/auth';

export class RoleRoutes {
  public roleController: RoleController = new RoleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/roles/public")
      .get(this.roleController.getAllRoles)

```

```

        .post(this.roleController.createRole);

    app.route("/api/roles/public/:id")
        .get(this.roleController.getRoleById)
        .patch(this.roleController.updateRole)
        .delete(this.roleController.deleteRole);

    app.route("/api/roles/public/:id/logic")
        .delete(this.roleController.deleteRoleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    app.route("/api/roles")
        .get(authMiddleware, this.roleController.getAllRoles)
        .post(authMiddleware, this.roleController.createRole);

    app.route("/api/roles/:id")
        .get(authMiddleware, this.roleController.getRoleById)
        .patch(authMiddleware, this.roleController.updateRole)
        .delete(authMiddleware, this.roleController.deleteRole);

    app.route("/api/roles/:id/logic")
        .delete(authMiddleware, this.roleController.deleteRoleAdv);
}
}

```

src/http/authorization/role.ts

```
### role.http
```

```
@baseUrl = http://localhost:4000
```

```
@token =
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZhrNnP6wb6dA
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los roles (SIN token)
```

```
GET {{baseUrl}}/api/roles/public
```

```
### Obtener rol por ID (SIN token)
```

```
GET {{baseUrl}}/api/roles/public/1
```

```
### Crear un nuevo rol (SIN token)
```

```
POST {{baseUrl}}/api/roles/public
```

```
Content-Type: application/json
```

```
{
  "name": "Administrator",
  "is_active": "ACTIVE"
}
```

```
### Actualizar un rol (SIN token)
```

```
PATCH {{baseUrl}}/api/roles/public/1
```

```
Content-Type: application/json
```

```
{
  "name": "Super Administrator",
  "is_active": "ACTIVE"
}
```

```
### Eliminar un rol físicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/roles/public/1
```

```
### Eliminar un rol lógicamente (SIN token)
```

```

DELETE {{baseUrl}}/api/roles/public/2/logic

### ===== RUTAS CON AUTENTICACIÓN =====

### Obtener todos los roles (CON token)
GET {{baseUrl}}/api/roles
Authorization: Bearer {{token}}

### Obtener rol por ID (CON token)
GET {{baseUrl}}/api/roles/1
Authorization: Bearer {{token}}

### Crear un nuevo rol (CON token)
POST {{baseUrl}}/api/roles
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "Manager",
  "is_active": "ACTIVE"
}

### Actualizar un rol (CON token)
PATCH {{baseUrl}}/api/roles/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "Senior Manager",
  "is_active": "ACTIVE"
}

### Eliminar un rol físicamente (CON token)
DELETE {{baseUrl}}/api/roles/1
Authorization: Bearer {{token}}

### Eliminar un rol lógicamente (CON token)
DELETE {{baseUrl}}/api/roles/1/logic
Authorization: Bearer {{token}}

```

6.5.3 Ruta y Http RoleUser

src/routes/authorization/role_user.ts

```

import { Application } from "express";
import { RoleUserController } from '../controllers/authorization/role_user.controller';
import { authMiddleware } from '../middleware/auth';

export class RoleUserRoutes {
  public roleUserController: RoleUserController = new RoleUserController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/roleUsers/public")
      .get(this.roleUserController.getAllRoleUsers)
      .post(this.roleUserController.createRoleUser);

    app.route("/api/roleUsers/public/:id")
      .get(this.roleUserController.getRoleUserById)
      .patch(this.roleUserController.updateRoleUser)
      .delete(this.roleUserController.deleteRoleUser);

    app.route("/api/roleUsers/public/:id/logic")

```

```

        .delete(this.roleUserController.deleteRoleUserAdv);

// ===== RUTAS CON AUTENTICACIÓN =====
app.route("/api/roleUsers")
    .get(authMiddleware, this.roleUserController.getAllRoleUsers)
    .post(authMiddleware, this.roleUserController.createRoleUser);

app.route("/api/roleUsers/:id")
    .get(authMiddleware, this.roleUserController.getRoleUserById)
    .patch(authMiddleware, this.roleUserController.updateRoleUser)
    .delete(authMiddleware, this.roleUserController.deleteRoleUser);

app.route("/api/roleUsers/:id/logic")
    .delete(authMiddleware, this.roleUserController.deleteRoleUserAdv);
}
}

```

src/http/authorization/role_user.ts

```
### roleUser.http
```

```

@baseUrl = http://localhost:4000
@token =
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1ldCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKfFdkcOLHMmaVa12iOQKdrhZHRnNp6wb6dA

```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los roles de usuario (SIN token)
```

```
GET {{baseUrl}}/api/roleUsers/public
```

```
### Obtener rol de usuario por ID (SIN token)
```

```
GET {{baseUrl}}/api/roleUsers/public/1
```

```
### Crear un nuevo rol de usuario (SIN token)
```

```
POST {{baseUrl}}/api/roleUsers/public
```

```
Content-Type: application/json
```

```

{
  "role_id": 1,
  "user_id": 1,
  "is_active": "ACTIVE"
}

```

```
### Actualizar un rol de usuario (SIN token)
```

```
PATCH {{baseUrl}}/api/roleUsers/public/1
```

```
Content-Type: application/json
```

```

{
  "role_id": 2,
  "user_id": 1,
  "is_active": "ACTIVE"
}

```

```
### Eliminar un rol de usuario físicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/roleUsers/public/1
```

```
### Eliminar un rol de usuario lógicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/roleUsers/public/2/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

```
### Obtener todos los roles de usuario (CON token)
```

```
GET {{baseUrl}}/api/roleUsers
```

```

Authorization: Bearer {{token}}

### Obtener rol de usuario por ID (CON token)
GET {{baseUrl}}/api/roleUsers/1
Authorization: Bearer {{token}}

### Crear un nuevo rol de usuario (CON token)
POST {{baseUrl}}/api/roleUsers
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "role_id": 1,
  "user_id": 2,
  "is_active": "ACTIVE"
}

### Actualizar un rol de usuario (CON token)
PATCH {{baseUrl}}/api/roleUsers/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "role_id": 3,
  "user_id": 2,
  "is_active": "ACTIVE"
}

### Eliminar un rol de usuario físicamente (CON token)
DELETE {{baseUrl}}/api/roleUsers/1
Authorization: Bearer {{token}}

### Eliminar un rol de usuario lógicamente (CON token)
DELETE {{baseUrl}}/api/roleUsers/1/logic
Authorization: Bearer {{token}}

```

6.5.4 Ruta y Http Resource

src/routes/authorization/resource.ts

```

import { Application } from "express";
import { ResourceController } from "../../controllers/authorization/resource.controller";
import { authMiddleware } from "../../middleware/auth";

export class ResourceRoutes {
  public resourceController: ResourceController = new ResourceController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/resources/public")
      .get(this.resourceController.getAllResources)
      .post(this.resourceController.createResource);

    app.route("/api/resources/public/:id")
      .get(this.resourceController.getResourceById)
      .patch(this.resourceController.updateResource)
      .delete(this.resourceController.deleteResource);

    app.route("/api/resources/public/:id/logic")
      .delete(this.resourceController.deleteResourceAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    app.route("/api/resources")

```



```

        .get(authMiddleware, this.resourceController.getAllResources)
        .post(authMiddleware, this.resourceController.createResource);

    app.route("/api/resources/:id")
        .get(authMiddleware, this.resourceController.getResourceById)
        .patch(authMiddleware, this.resourceController.updateResource)
        .delete(authMiddleware, this.resourceController.deleteResource);

    app.route("/api/resources/:id/logic")
        .delete(authMiddleware, this.resourceController.deleteResourceAdv);
}
}

```

src/http/authorization/resource.ts

```
### resource.http
```

```
@baseUrl = http://localhost:4000
```

```
@token =
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZhrNnNp6wb6dA
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los recursos (SIN token)
```

```
GET {{baseUrl}}/api/resources/public
```

```
### Obtener recurso por ID (SIN token)
```

```
GET {{baseUrl}}/api/resources/public/1
```

```
### Crear un nuevo recurso (SIN token)
```

```
POST {{baseUrl}}/api/resources/public
```

```
Content-Type: application/json
```

```
{
  "path": "/api/clientes",
  "method": "GET",
  "is_active": "ACTIVE"
}
```

```
### Actualizar un recurso (SIN token)
```

```
PATCH {{baseUrl}}/api/resources/public/1
```

```
Content-Type: application/json
```

```
{
  "path": "/api/clientes/:id",
  "method": "GET",
  "is_active": "ACTIVE"
}
```

```
### Eliminar un recurso físicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/resources/public/1
```

```
### Eliminar un recurso lógicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/resources/public/2/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

```
### Obtener todos los recursos (CON token)
```

```
GET {{baseUrl}}/api/resources
```

```
Authorization: Bearer {{token}}
```

```
### Obtener recurso por ID (CON token)
```

```
GET {{baseUrl}}/api/resources/1
```

```

Authorization: Bearer {{token}}

### Crear un nuevo recurso (CON token)
POST {{baseUrl}}/api/resources
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "path": "/api/users",
  "method": "POST",
  "is_active": "ACTIVE"
}

### Actualizar un recurso (CON token)
PATCH {{baseUrl}}/api/resources/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "path": "/api/users/:id",
  "method": "PATCH",
  "is_active": "ACTIVE"
}

### Eliminar un recurso físicamente (CON token)
DELETE {{baseUrl}}/api/resources/1
Authorization: Bearer {{token}}

### Eliminar un recurso lógicamente (CON token)
DELETE {{baseUrl}}/api/resources/1/logic
Authorization: Bearer {{token}}

```

6.5.5 Ruta y Http ResourceRole

src/routes/authorization/resourceRole.ts

```

import { Application } from "express";
import { ResourceRoleController } from
"../../controllers/authorization/resourceRole.controller";
import { authMiddleware } from '../../middleware/auth';

export class ResourceRoleRoutes {
  public resourceRoleController: ResourceRoleController = new ResourceRoleController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/resourceRoles/public")
      .get(this.resourceRoleController.getAllResourceRoles)
      .post(this.resourceRoleController.createResourceRole);

    app.route("/api/resourceRoles/public/:id")
      .get(this.resourceRoleController.getResourceRoleById)
      .patch(this.resourceRoleController.updateResourceRole)
      .delete(this.resourceRoleController.deleteResourceRole);

    app.route("/api/resourceRoles/public/:id/logic")
      .delete(this.resourceRoleController.deleteResourceRoleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    app.route("/api/resourceRoles")
      .get(authMiddleware, this.resourceRoleController.getAllResourceRoles)
      .post(authMiddleware, this.resourceRoleController.createResourceRole);
  }
}

```

```

    app.route("/api/resourceRoles/:id")
      .get(authMiddleware, this.resourceRoleController.getResourceRoleById)
      .patch(authMiddleware, this.resourceRoleController.updateResourceRole)
      .delete(authMiddleware, this.resourceRoleController.deleteResourceRole);

    app.route("/api/resourceRoles/:id/logic")
      .delete(authMiddleware, this.resourceRoleController.deleteResourceRoleAdv);
  }
}

```

src/http/authorization/resourceRole.ts

```
### resourceRole.http
```

```
@baseUrl = http://localhost:4000
```

```
@token =
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGKfFDkcOLHMmaVa12iOQKdrhZhrNnNp6wb6dA
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los roles de recursos (SIN token)
```

```
GET {{baseUrl}}/api/resourceRoles/public
```

```
### Obtener rol de recurso por ID (SIN token)
```

```
GET {{baseUrl}}/api/resourceRoles/public/1
```

```
### Crear un nuevo rol de recurso (SIN token)
```

```
POST {{baseUrl}}/api/resourceRoles/public
```

```
Content-Type: application/json
```

```
{
  "resource_id": 1,
  "role_id": 1,
  "is_active": "ACTIVE"
}
```

```
### Actualizar un rol de recurso (SIN token)
```

```
PATCH {{baseUrl}}/api/resourceRoles/public/1
```

```
Content-Type: application/json
```

```
{
  "resource_id": 1,
  "role_id": 2,
  "is_active": "ACTIVE"
}
```

```
### Eliminar un rol de recurso físicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/resourceRoles/public/1
```

```
### Eliminar un rol de recurso lógicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/resourceRoles/public/2/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

```
### Obtener todos los roles de recursos (CON token)
```

```
GET {{baseUrl}}/api/resourceRoles
```

```
Authorization: Bearer {{token}}
```

```
### Obtener rol de recurso por ID (CON token)
```

```
GET {{baseUrl}}/api/resourceRoles/1
```

```
Authorization: Bearer {{token}}
```

```
### Crear un nuevo rol de recurso (CON token)
```

```

POST {{baseUrl}}/api/resourceRoles
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "resource_id": 2,
  "role_id": 1,
  "is_active": "ACTIVE"
}

### Actualizar un rol de recurso (CON token)
PATCH {{baseUrl}}/api/resourceRoles/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "resource_id": 2,
  "role_id": 3,
  "is_active": "ACTIVE"
}

### Eliminar un rol de recurso físicamente (CON token)
DELETE {{baseUrl}}/api/resourceRoles/1
Authorization: Bearer {{token}}

### Eliminar un rol de recurso lógicamente (CON token)
DELETE {{baseUrl}}/api/resourceRoles/1/logic
Authorization: Bearer {{token}}

```

6.5.5 Ruta y Http RefreshToken

No trabajaremos la parte de autenticacion y autorizacion este modelo

6.5 Autenticacion con Token Rutas Modelos del Negocio

6.5.1 Ruta y Http Client

src/routes/client.ts

```

import { Router, Application } from "express";
import { ClientController } from "../controllers/client.controller";
import { authMiddleware } from "../middleware/auth";

export class ClientRoutes {
  public clientController: ClientController = new ClientController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/clientes/public")
      .get(this.clientController.getAllClients)
      .post(this.clientController.createClient);

    app.route("/api/clientes/public/:id")
      .get(this.clientController.getClientById)
      .patch(this.clientController.updateClient)
      .delete(this.clientController.deleteClient);

    app.route("/api/clientes/public/:id/logic")
      .delete(this.clientController.deleteClientAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====

```

```

    app.route("/api/clientes")
      .get(authMiddleware, this.clientController.getAllClients)
      .post(authMiddleware, this.clientController.createClient);

    app.route("/api/clientes/:id")
      .get(authMiddleware, this.clientController.getClientById)
      .patch(authMiddleware, this.clientController.updateClient)
      .delete(authMiddleware, this.clientController.deleteClient);

    app.route("/api/clientes/:id/logic")
      .delete(authMiddleware, this.clientController.deleteClientAdv);
  }
}

```

src/http/client.http

```

// client.http
import dotenv from "dotenv";

@token =
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMTsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZHrNnP6wb6dA
@baseUrl = http://localhost:4000

### ===== RUTAS SIN AUTENTICACIÓN =====

### Obtener todos los clientes (SIN token)
GET {{baseUrl}}/api/clientes/public

### Obtener cliente por ID (SIN token)
GET {{baseUrl}}/api/clientes/public/1

### Crear un nuevo cliente (SIN token)
POST {{baseUrl}}/api/clientes/public
Content-Type: application/json

{
  "name": "Sandy Romero",
  "address": "Km 1",
  "phone": "3001234567",
  "email": "srm@gmail.com",
  "password": "12345678",
  "status": "ACTIVE"
}

### Actualizar un cliente (SIN token)
PATCH {{baseUrl}}/api/clientes/public/1
Content-Type: application/json

{
  "name": "Juan Perez Actualizado",
  "address": "Calle 456",
  "phone": "3007654321",
  "email": "juan.perez@example.com",
  "password": "654321",
  "status": "ACTIVE"
}

### Eliminar un cliente físicamente (SIN token)
DELETE {{baseUrl}}/api/clientes/public/1

### Eliminar un cliente lógicamente (SIN token)
DELETE {{baseUrl}}/api/clientes/public/2/logic

```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

```
### Obtener todos los clientes (CON token)
GET {{baseUrl}}/api/clientes
Authorization: Bearer {{token}}
```

```
### Obtener cliente por ID (CON token)
GET {{baseUrl}}/api/clientes/1
Authorization: Bearer {{token}}
```

```
### Crear un nuevo cliente (CON token)
POST {{baseUrl}}/api/clientes
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "name": "María García",
  "address": "Calle 123",
  "phone": "3009876543",
  "email": "maria.garcia@example.com",
  "password": "87654321",
  "status": "ACTIVE"
}
```

```
### Actualizar un cliente (CON token)
PATCH {{baseUrl}}/api/clientes/1
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "name": "María García Actualizada",
  "address": "Avenida 456",
  "phone": "3001234567",
  "email": "maria.garcia.updated@example.com",
  "password": "newpassword123",
  "status": "ACTIVE"
}
```

```
### Eliminar un cliente físicamente (CON token)
DELETE {{baseUrl}}/api/clientes/1
Authorization: Bearer {{token}}
```

```
### Eliminar un cliente lógicamente (CON token)
DELETE {{baseUrl}}/api/clientes/1/logic
Authorization: Bearer {{token}}
```

6.5.2 Ruta y Http ProductType

src/routes/productType.ts

```
import { Application } from "express";
import { ProductTypeController } from "../controllers/productType.controller";
import { authMiddleware } from "../middleware/auth";

export class ProductTypeRoutes {
  public productTypeController: ProductTypeController = new ProductTypeController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/product-types/public")
      .get(this.productTypeController.getAllProductTypes)
      .post(this.productTypeController.createProductType);
  }
}
```

```

    app.route("/api/product-types/public/:id")
      .get(this.productTypeController.getProductTypeById)
      .patch(this.productTypeController.updateProductType)
      .delete(this.productTypeController.deleteProductType);

    app.route("/api/product-types/public/:id/logic")
      .delete(this.productTypeController.deleteProductTypeAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    app.route("/api/product-types")
      .get(authMiddleware, this.productTypeController.getAllProductTypes)
      .post(authMiddleware, this.productTypeController.createProductType);

    app.route("/api/product-types/:id")
      .get(authMiddleware, this.productTypeController.getProductTypeById)
      .patch(authMiddleware, this.productTypeController.updateProductType)
      .delete(authMiddleware, this.productTypeController.deleteProductType);

    app.route("/api/product-types/:id/logic")
      .delete(authMiddleware, this.productTypeController.deleteProductTypeAdv);
  }
}

```

src/http/productType.http

```
### productType.http
```

```
@baseUrl = http://localhost:4000
```

```
@token =
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1ldCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZhrNnP6wb6dA
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todos los tipos de producto (SIN token)
```

```
GET {{baseUrl}}/api/product-types/public
```

```
### Obtener tipo de producto por ID (SIN token)
```

```
GET {{baseUrl}}/api/product-types/public/1
```

```
### Crear un nuevo tipo de producto (SIN token)
```

```
POST {{baseUrl}}/api/product-types/public
```

```
Content-Type: application/json
```

```
{
  "name": "Electrónicos",
  "description": "Productos electrónicos y tecnológicos",
  "status": "ACTIVE"
}
```

```
### Actualizar un tipo de producto (SIN token)
```

```
PATCH {{baseUrl}}/api/product-types/public/1
```

```
Content-Type: application/json
```

```
{
  "name": "Electrónicos Actualizados",
  "description": "Productos electrónicos y tecnológicos actualizados",
  "status": "ACTIVE"
}
```

```
### Eliminar un tipo de producto físicamente (SIN token)
```

```
DELETE {{baseUrl}}/api/product-types/public/1
```

```
### Eliminar un tipo de producto lógicamente (SIN token)
```

```

DELETE {{baseUrl}}/api/product-types/public/2/logic

### ===== RUTAS CON AUTENTICACIÓN =====

### Obtener todos los tipos de producto (CON token)
GET {{baseUrl}}/api/product-types
Authorization: Bearer {{token}}

### Obtener tipo de producto por ID (CON token)
GET {{baseUrl}}/api/product-types/1
Authorization: Bearer {{token}}

### Crear un nuevo tipo de producto (CON token)
POST {{baseUrl}}/api/product-types
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "Ropa y Accesorios",
  "description": "Prendas de vestir y accesorios de moda",
  "status": "ACTIVE"
}

### Actualizar un tipo de producto (CON token)
PATCH {{baseUrl}}/api/product-types/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "Ropa y Accesorios Premium",
  "description": "Prendas de vestir y accesorios de moda premium",
  "status": "ACTIVE"
}

### Eliminar un tipo de producto físicamente (CON token)
DELETE {{baseUrl}}/api/product-types/1
Authorization: Bearer {{token}}

### Eliminar un tipo de producto lógicamente (CON token)
DELETE {{baseUrl}}/api/product-types/1/logic
Authorization: Bearer {{token}}

```

6.5.3 Ruta y Http Product

src/routes/product.ts

```

import { Application } from "express";
import { ProductController } from "../controllers/product.controller";
import { authMiddleware } from "../middleware/auth";

export class ProductRoutes {
  public productController: ProductController = new ProductController();

  public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/products/public")
      .get(this.productController.getAllProducts)
      .post(this.productController.createProduct);

    app.route("/api/products/public/:id")
      .get(this.productController.getProductById)
      .patch(this.productController.updateProduct)
      .delete(this.productController.deleteProduct);
  }
}

```



```

app.route("/api/products/public/:id/logic")
    .delete(this.productController.deleteProductAdv);

// ===== RUTAS CON AUTENTICACIÓN =====
app.route("/api/products")
    .get(authMiddleware, this.productController.getAllProducts)
    .post(authMiddleware, this.productController.createProduct);

app.route("/api/products/:id")
    .get(authMiddleware, this.productController.getProductById)
    .patch(authMiddleware, this.productController.updateProduct)
    .delete(authMiddleware, this.productController.deleteProduct);

app.route("/api/products/:id/logic")
    .delete(authMiddleware, this.productController.deleteProductAdv);
}
}

```

src/http/product.http

product.http

@baseUrl = http://localhost:4000

@token =

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZHRnNp6wb6dA

===== RUTAS SIN AUTENTICACIÓN =====

Obtener todos los productos (SIN token)

GET {{baseUrl}}/api/products/public

Obtener producto por ID (SIN token)

GET {{baseUrl}}/api/products/public/1

Crear un nuevo producto (SIN token)

POST {{baseUrl}}/api/products/public
Content-Type: application/json

```

{
  "name": "Laptop HP",
  "brand": "HP",
  "price": 1500000,
  "min_stock": 5,
  "quantity": 10,
  "product_type_id": 1,
  "status": "ACTIVE"
}

```

Actualizar un producto (SIN token)

PATCH {{baseUrl}}/api/products/public/1
Content-Type: application/json

```

{
  "name": "Laptop HP Actualizada",
  "brand": "HP",
  "price": 1600000,
  "min_stock": 3,
  "quantity": 8,
  "product_type_id": 1,
  "status": "ACTIVE"
}

```

```

### Eliminar un producto físicamente (SIN token)
DELETE {{baseUrl}}/api/products/public/1

### Eliminar un producto lógicamente (SIN token)
DELETE {{baseUrl}}/api/products/public/2/logic

### ===== RUTAS CON AUTENTICACIÓN =====

### Obtener todos los productos (CON token)
GET {{baseUrl}}/api/products
Authorization: Bearer {{token}}

### Obtener producto por ID (CON token)
GET {{baseUrl}}/api/products/1
Authorization: Bearer {{token}}

### Crear un nuevo producto (CON token)
POST {{baseUrl}}/api/products
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "MacBook Air M2",
  "brand": "Apple",
  "price": 3500000,
  "min_stock": 2,
  "quantity": 5,
  "product_type_id": 1,
  "status": "ACTIVE"
}

### Actualizar un producto (CON token)
PATCH {{baseUrl}}/api/products/1
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "name": "MacBook Air M2 Updated",
  "brand": "Apple",
  "price": 3600000,
  "min_stock": 3,
  "quantity": 7,
  "product_type_id": 1,
  "status": "ACTIVE"
}

### Eliminar un producto físicamente (CON token)
DELETE {{baseUrl}}/api/products/1
Authorization: Bearer {{token}}

### Eliminar un producto lógicamente (CON token)
DELETE {{baseUrl}}/api/products/1/logic
Authorization: Bearer {{token}}

```

6.5.4 Ruta y Http Sale

src/routes/sale.ts

```

import { Application } from "express";
import { SaleController } from "../controllers/sale.controller";
import { authMiddleware } from "../middleware/auth";

export class SaleRoutes {

```

```

public saleController: SaleController = new SaleController();

public routes(app: Application): void {
    // ===== RUTAS SIN AUTENTICACIÓN =====
    app.route("/api/sales/public")
        .get(this.saleController.getAllSales)
        .post(this.saleController.createSale);

    app.route("/api/sales/public/:id")
        .get(this.saleController.getSaleById)
        .patch(this.saleController.updateSale)
        .delete(this.saleController.deleteSale);

    app.route("/api/sales/public/:id/logic")
        .delete(this.saleController.deleteSaleAdv);

    // ===== RUTAS CON AUTENTICACIÓN =====
    app.route("/api/sales")
        .get(authMiddleware, this.saleController.getAllSales)
        .post(authMiddleware, this.saleController.createSale);

    app.route("/api/sales/:id")
        .get(authMiddleware, this.saleController.getSaleById)
        .patch(authMiddleware, this.saleController.updateSale)
        .delete(authMiddleware, this.saleController.deleteSale);

    app.route("/api/sales/:id/logic")
        .delete(authMiddleware, this.saleController.deleteSaleAdv);

    // ===== RUTAS LEGACY (mantener compatibilidad) =====
    app.route("/ventas")
        .get(this.saleController.getAllSales)
        .post(this.saleController.createSale);

    app.route("/ventas/:id")
        .get(this.saleController.getSaleById)
        .patch(this.saleController.updateSale)
        .delete(this.saleController.deleteSale);

    app.route("/ventas/:id/logic")
        .delete(this.saleController.deleteSaleAdv);
}
}

```

src/http/sale.http

```
### sale.http
```

```
@baseUrl = http://localhost:4000
```

```
@token =
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZHRnNP6wb6dA
```

```
### ===== RUTAS SIN AUTENTICACIÓN =====
```

```
### Obtener todas las ventas (SIN token)
```

```
GET {{baseUrl}}/api/sales/public
```

```
### Obtener venta por ID (SIN token)
```

```
GET {{baseUrl}}/api/sales/public/1
```

```
### Crear una nueva venta (SIN token)
```

```
POST {{baseUrl}}/api/sales/public
```

```
Content-Type: application/json
```

```
{
  "sale_date": "2024-01-15T10:30:00.000Z",
  "subtotal": 100000,
  "tax": 19000,
  "discounts": 5000,
  "total": 114000,
  "status": "ACTIVE",
  "client_id": 1
}
```

Actualizar una venta (SIN token)
PATCH {{baseUrl}}/api/sales/public/1
Content-Type: application/json

```
{
  "sale_date": "2024-01-15T10:30:00.000Z",
  "subtotal": 120000,
  "tax": 22800,
  "discounts": 10000,
  "total": 132800,
  "status": "ACTIVE",
  "client_id": 1
}
```

Eliminar una venta físicamente (SIN token)
DELETE {{baseUrl}}/api/sales/public/1

Eliminar una venta lógicamente (SIN token)
DELETE {{baseUrl}}/api/sales/public/2/logic

===== RUTAS CON AUTENTICACIÓN =====

Obtener todas las ventas (CON token)
GET {{baseUrl}}/api/sales
Authorization: Bearer {{token}}

Obtener venta por ID (CON token)
GET {{baseUrl}}/api/sales/1
Authorization: Bearer {{token}}

Crear una nueva venta (CON token)
POST {{baseUrl}}/api/sales
Authorization: Bearer {{token}}
Content-Type: application/json

```
{
  "sale_date": "2024-02-20T14:45:00.000Z",
  "subtotal": 250000,
  "tax": 47500,
  "discounts": 15000,
  "total": 282500,
  "status": "ACTIVE",
  "client_id": 2
}
```

Actualizar una venta (CON token)
PATCH {{baseUrl}}/api/sales/1
Authorization: Bearer {{token}}
Content-Type: application/json

```
{
  "sale_date": "2024-02-20T15:00:00.000Z",
  "subtotal": 275000,
  "tax": 52250,

```

```

    "discounts": 20000,
    "total": 307250,
    "status": "ACTIVE",
    "client_id": 2
  }

  ### Eliminar una venta físicamente (CON token)
  DELETE {{baseUrl}}/api/sales/1
  Authorization: Bearer {{token}}

  ### Eliminar una venta lógicamente (CON token)
  DELETE {{baseUrl}}/api/sales/1/logic
  Authorization: Bearer {{token}}

  ### ===== RUTAS LEGACY (mantener compatibilidad) =====

  ### Obtener todas las ventas (legacy)
  GET {{baseUrl}}/ventas

  ### Obtener venta por ID (legacy)
  GET {{baseUrl}}/ventas/1

  ### Crear una nueva venta (legacy)
  POST {{baseUrl}}/ventas
  Content-Type: application/json

  {
    "sale_date": "2024-01-15T10:30:00.000Z",
    "subtotal": 100000,
    "tax": 19000,
    "discounts": 5000,
    "total": 114000,
    "status": "ACTIVE",
    "client_id": 1
  }

  ### Actualizar una venta (legacy)
  PATCH {{baseUrl}}/ventas/1
  Content-Type: application/json

  {
    "sale_date": "2024-01-15T10:30:00.000Z",
    "subtotal": 120000,
    "tax": 22800,
    "discounts": 10000,
    "total": 132800,
    "status": "ACTIVE",
    "client_id": 1
  }

  ### Eliminar una venta físicamente (legacy)
  DELETE {{baseUrl}}/ventas/1

  ### Eliminar una venta lógicamente (legacy)
  DELETE {{baseUrl}}/ventas/1/logic

```

6.5.5 Ruta y Http ProductSale

src/routes/productSale.ts

```

import { Application } from "express";
import { ProductSaleController } from "../controllers/productSale.controller";
import { authMiddleware } from "../middleware/auth";

```

```

export class ProductSaleRoutes {
    public productSaleController: ProductSaleController = new ProductSaleController();

    public routes(app: Application): void {
        // ===== RUTAS SIN AUTENTICACIÓN =====
        app.route("/api/product-sales/public")
            .get(this.productSaleController.getAllProductSales)
            .post(this.productSaleController.createProductSale);

        app.route("/api/product-sales/public/:id")
            .get(this.productSaleController.getProductSaleById)
            .patch(this.productSaleController.updateProductSale)
            .delete(this.productSaleController.deleteProductSale);

        app.route("/api/product-sales/public/:id/logic")
            .delete(this.productSaleController.deleteProductSaleAdv);

        // ===== RUTAS CON AUTENTICACIÓN =====
        app.route("/api/product-sales")
            .get(authMiddleware, this.productSaleController.getAllProductSales)
            .post(authMiddleware, this.productSaleController.createProductSale);

        app.route("/api/product-sales/:id")
            .get(authMiddleware, this.productSaleController.getProductSaleById)
            .patch(authMiddleware, this.productSaleController.updateProductSale)
            .delete(authMiddleware, this.productSaleController.deleteProductSale);

        app.route("/api/product-sales/:id/logic")
            .delete(authMiddleware, this.productSaleController.deleteProductSaleAdv);

        // ===== RUTAS LEGACY (mantener compatibilidad) =====
        app.route("/product-sales")
            .get(this.productSaleController.getAllProductSales)
            .post(this.productSaleController.createProductSale);

        app.route("/product-sales/:id")
            .get(this.productSaleController.getProductSaleById)
            .patch(this.productSaleController.updateProductSale)
            .delete(this.productSaleController.deleteProductSale);

        app.route("/product-sales/:id/logic")
            .delete(this.productSaleController.deleteProductSaleAdv);
    }
}

```

src/http/productSale.http

productSale.http

@baseUrl = http://localhost:4000

@token =

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTMsIm1hdCI6MTc2MDA2MDU1NywiZXhwIjoxNzYwMDYxMTU3fQ.19kxHGraKffDkcOLHMmaVa12iOQKdrhZhrNnNp6wb6dA

===== RUTAS SIN AUTENTICACIÓN =====

Obtener todas las ventas de productos (SIN token)

GET {{baseUrl}}/api/product-sales/public

Obtener venta de producto por ID (SIN token)

GET {{baseUrl}}/api/product-sales/public/1

Crear una nueva venta de producto (SIN token)

```
POST {{baseUrl}}/api/product-sales/public
Content-Type: application/json
```

```
{
  "total": 1500000,
  "sale_id": 1,
  "product_id": 1
}
```

```
### Actualizar una venta de producto (SIN token)
PATCH {{baseUrl}}/api/product-sales/public/1
Content-Type: application/json
```

```
{
  "total": 1600000,
  "sale_id": 1,
  "product_id": 1
}
```

```
### Eliminar una venta de producto físicamente (SIN token)
DELETE {{baseUrl}}/api/product-sales/public/1
```

```
### Eliminar una venta de producto lógicamente (SIN token)
DELETE {{baseUrl}}/api/product-sales/public/2/logic
```

```
### ===== RUTAS CON AUTENTICACIÓN =====
```

```
### Obtener todas las ventas de productos (CON token)
GET {{baseUrl}}/api/product-sales
Authorization: Bearer {{token}}
```

```
### Obtener venta de producto por ID (CON token)
GET {{baseUrl}}/api/product-sales/1
Authorization: Bearer {{token}}
```

```
### Crear una nueva venta de producto (CON token)
POST {{baseUrl}}/api/product-sales
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "total": 2500000,
  "sale_id": 2,
  "product_id": 2
}
```

```
### Actualizar una venta de producto (CON token)
PATCH {{baseUrl}}/api/product-sales/1
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "total": 2700000,
  "sale_id": 2,
  "product_id": 2
}
```

```
### Eliminar una venta de producto físicamente (CON token)
DELETE {{baseUrl}}/api/product-sales/1
Authorization: Bearer {{token}}
```

```
### Eliminar una venta de producto lógicamente (CON token)
DELETE {{baseUrl}}/api/product-sales/1/logic
Authorization: Bearer {{token}}
```

```

### ===== RUTAS LEGACY (mantener compatibilidad) =====

### Obtener todas las ventas de productos (legacy)
GET {{baseUrl}}/product-sales

### Obtener venta de producto por ID (legacy)
GET {{baseUrl}}/product-sales/1

### Crear una nueva venta de producto (legacy)
POST {{baseUrl}}/product-sales
Content-Type: application/json

{
  "total": 1500000,
  "sale_id": 1,
  "product_id": 1
}

### Actualizar una venta de producto (legacy)
PATCH {{baseUrl}}/product-sales/1
Content-Type: application/json

{
  "total": 1600000,
  "sale_id": 1,
  "product_id": 1
}

### Eliminar una venta de producto físicamente (legacy)
DELETE {{baseUrl}}/product-sales/1

### Eliminar una venta de producto lógicamente (legacy)
DELETE {{baseUrl}}/product-sales/1/logic

```

6.5 Implementar la Autorizacion

Para ello debemos insertar los registros a todas las tablas de los modelos de Athorization.

Luego, implementar el auth del middleware de la siguiente manera:

src/middleware/auth.ts

```

import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { User } from '../models/authorization/User';
import { Role } from '../models/authorization/Role';
import { ResourceRole } from '../models/authorization/ResourceRole';
import { Resource } from '../models/authorization/Resource';
import { RoleUser } from '../models/authorization/RoleUser';
import { pathToRegexp } from 'path-to-regexp'; // Importar path-to-regexp
import { addEmitHelper } from 'typescript';

export const authMiddleware = async (req: Request, res: Response, next: NextFunction):
Promise<void> => {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  const currentRoute = req.originalUrl;
  const currentMethod = req.method;

  if (!token) {

```



```

    res.status(401).json({ error: 'Acceso denegado: No se proporcionó el token principal.' });
    return;
}

try {
    // Verificar si el token principal es válido
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret') as jwt.JwtPayload;

    // Buscar el usuario en la base de datos
    const user: User | null = await User.findOne({ where: { id: decoded.id, is_active:
'ACTIVE' } });
    if (!user) {
        res.status(401).json({ error: 'Usuario no encontrado o inactivo.' });
        return;
    }

    // Validar autorización
    const isAuthorized = await validateAuthorization(decoded.id, currentRoute, currentMethod);
    if (!isAuthorized) {
        res.status(403).json({ error: 'No está autorizado para ejecutar esta petición.' });
        return;
    }

    // Continuar con la solicitud
    next();
} catch (error: any) {
    if (error.name === 'TokenExpiredError') {
        res.status(401).json({ error: 'El token principal ha expirado.' });
    } else if (error.name === 'JsonWebTokenError') {
        res.status(401).json({ error: 'Token inválido.' });
    } else {
        res.status(500).json({ error: 'Error interno del servidor.', details: error.message });
    }
}
};

export const validateAuthorization = async (userId: number, resourcePath: string,
resourceMethod: string): Promise<boolean> => {
    try {
        // Obtener todos los recursos activos que coincidan con el método
        const resources = await Resource.findAll({
            where: { method: resourceMethod, is_active: "ACTIVE" },
        });

        // Convertir las rutas dinámicas a expresiones regulares y buscar coincidencias
        const matchingResource = resources.find((resource) => {
            const regex = pathToRegexp(resource.path).regexp.test(resourcePath);
            return regex;
        });

        if (!matchingResource) {
            return false; // No hay coincidencias para la ruta y el método
        }

        // Verificar si existe una relación válida entre el usuario, su rol y el recurso
        solicitado
        const resourceRole = await ResourceRole.findOne({
            include: [
                {
                    model: Role,
                    include: [
                        {
                            model: RoleUser,
                            where: { user_id: userId, is_active: "ACTIVE" }, // Validar que el usuario esté
asociado al rol

```

```

    },
    ],
    where: { is_active: "ACTIVE" }, // Validar que el rol esté activo
  },
  ],
  where: { resource_id: matchingResource.id, is_active: "ACTIVE" }, // Validar que la
relación resource_role esté activa
});

return !!resourceRole; // Retorna true si se encuentra un registro coincidente
} catch (error) {
  console.error('Error al validar la autorización:', error);
  return false;
}
};

```

Configurar Paquete CORS

Necesitamos permitir solicitudes a nuestra aplicación de NodeJs desde otros orígenes.\ En este ejemplo, configuraremos CORS para aceptar solicitudes de `localhost:4200`

Primero, instale la biblioteca *cors* en VS Code:

```
\ npm install cors
```

En `src/config/index.ts`

```

import express, { Application } from 'express';
import morgan from 'morgan';
import { Routes } from '../routes/index';
var cors = require("cors"); // install en node y types

export class App {
  public routePrv: Routes = new Routes();
  app: Application;

  constructor(
    private port?: number | string
  ) {
    this.app = express();
    this.settings();
    this.middlewares();
    this.routes()
  }

  private settings() {
    this.app.set('port', this.port || process.env.PORT || 3000);
  }

  private middlewares() {
    this.app.use(morgan('dev'));
    this.app.use(cors());
    this.app.use(express.json()); // leer json raw
  }
}

```

```

        this.app.use(express.urlencoded({ extended: false })); //leer json form
    }

    routes() {
        this.routePrv.clienteRoutes.routes(this.app)
        this.routePrv.productoRoutes.routes(this.app)
        this.routePrv.tipoProductoRoutes.routes(this.app)
        this.routePrv.ventaRoutes.routes(this.app)
    }

    async listen() {
        await this.app.listen(this.app.get('port'));
        // await this.app.listen(this.port);
        // console.log('Server on port', this.port);
        console.log('Server on port', this.app.get('port'));
    }
}

```

```

import express, { Application } from 'express';
import morgan from 'morgan';
import { Routes } from '../routes/index';
var cors = require("cors"); // install en node y types

```



```

export class App {
    public routePrv: Routes = new Routes();
    app: Application;
}

```

También debe agregar una clase de middleware para escuchar las respuestas:

```

private middlewares() {
    this.app.use(morgan('dev'));
    this.app.use(cors());
    this.app.use(express.json()); // leer json raw
    this.app.use(express.urlencoded({ extended: false })); //leer json form
}

```



Angular - Instalación

Instalar vía CLI

Abrimos en Visual Studio Code la terminal.

Requisitos

```
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/appfront$ node --version
v22.18.0
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/appfront$
```

Crear proyecto

```
ng new frontend
```

```
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html$ ng new frontend
? Which stylesheet format would you like to use? (Use arrow keys)
> CSS [ https://developer.mozilla.org/docs/Web/CSS ]
  Sass (SCSS) [ https://sass-lang.com/documentation/syntax#scss ]
  Sass (Indented) [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
  Less [ http://lesscss.org ]

✓ Which stylesheet format would you like to use? CSS [ https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
? Do you want to create a 'zoneless' application without zone.js (Developer Preview)? (y/N) n
```

```
cd frontend
ng s
```

```
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html$ cd frontend/
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/frontend$ ng s

Initial chunk files | Names | Raw size |
main.js | main | 47.76 kB |
polyfills.js | polyfills | 95 bytes |
styles.css | styles | 95 bytes |

| Initial total | 47.95 kB

Application bundle generation complete. [1.736 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
```

Ahora instalamos Tailwin CSS para angular

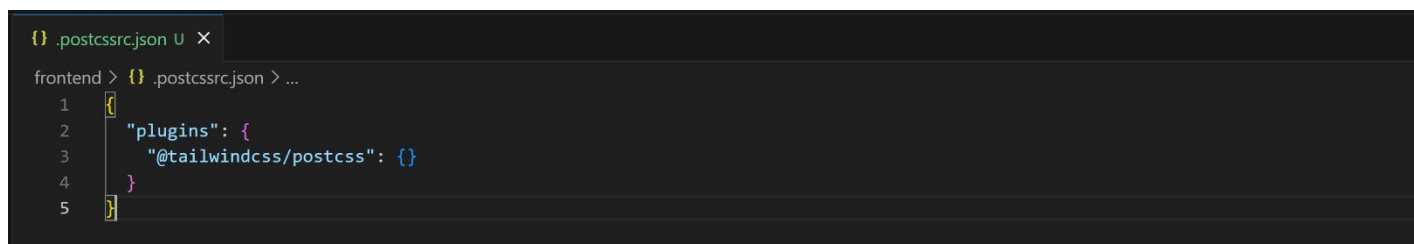
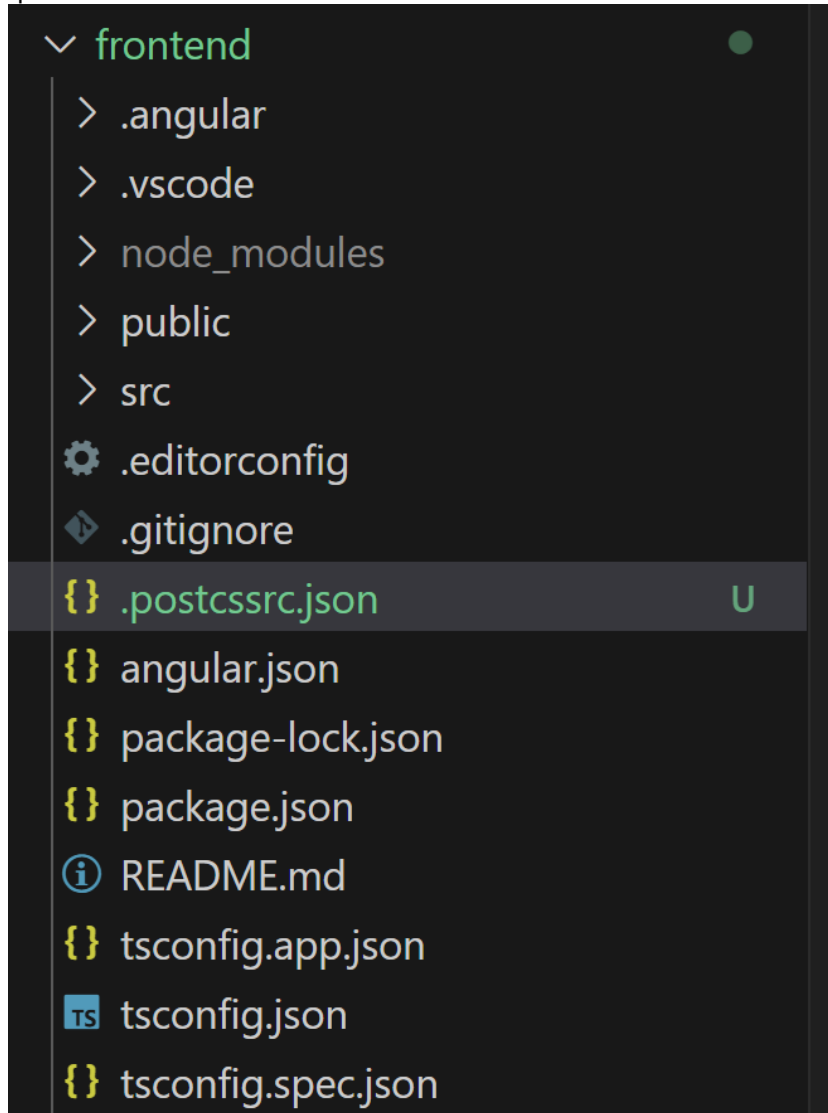
```
npm install tailwindcss @tailwindcss/postcss postcss --force
```

Creamos el archivo `.postcssrc.json` en la ruta frontend/.postcssrc.json

el siguiente codigo

```
{  
  "plugins": {  
    "@tailwindcss/postcss": {}  
  }  
}
```

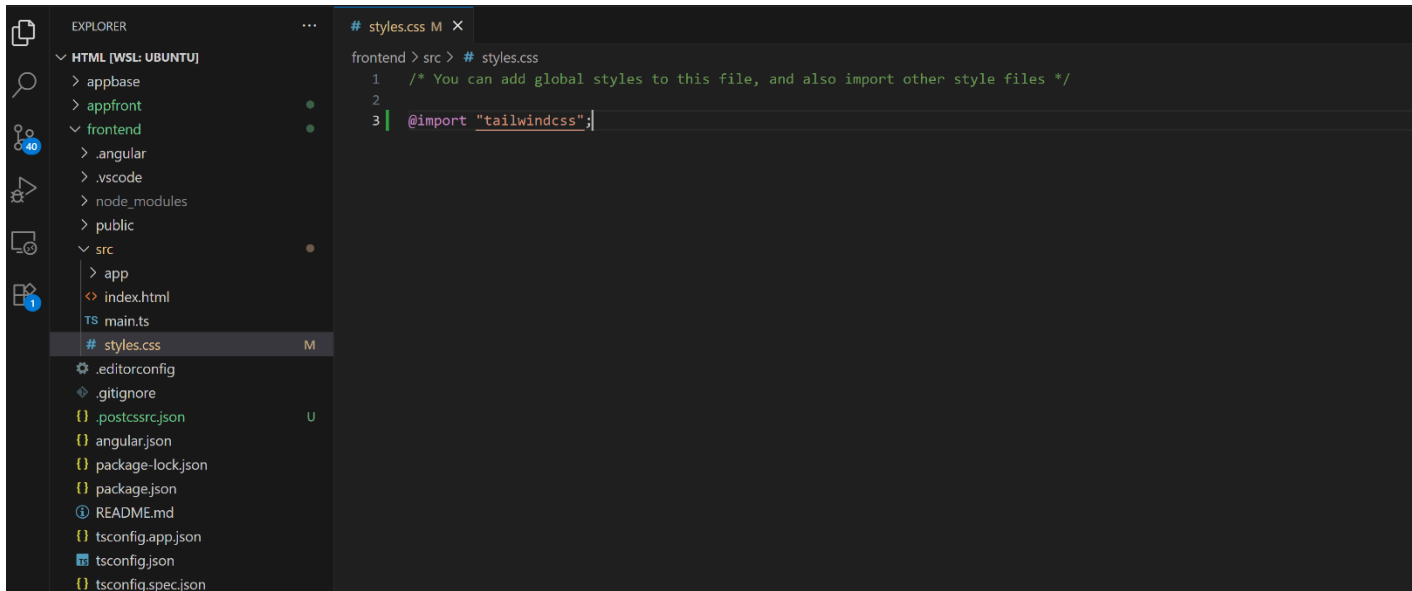
queda asi



Import Tailwind CSS

Adicionamos `@import` en `./src/styles.css`

```
@import "tailwindcss";
```



Levantamos el servidor con

```
ng s
```

Para probar que todo este funcionando bien hacemos lo siguiente:

En `frontend/src/app/app.html`

```
<h1>Desarrollo Web</h1>
```

```
<router-outlet />
```



Si observas el resultado igual a la imagen anterior, incluyendo el tipo de fuente, entonces todo esta correcto.

Usando npm

```
npm install primeng @primeuix/themes
```

Ahora en el archivo `frontend/src/app/app.config.ts`

```

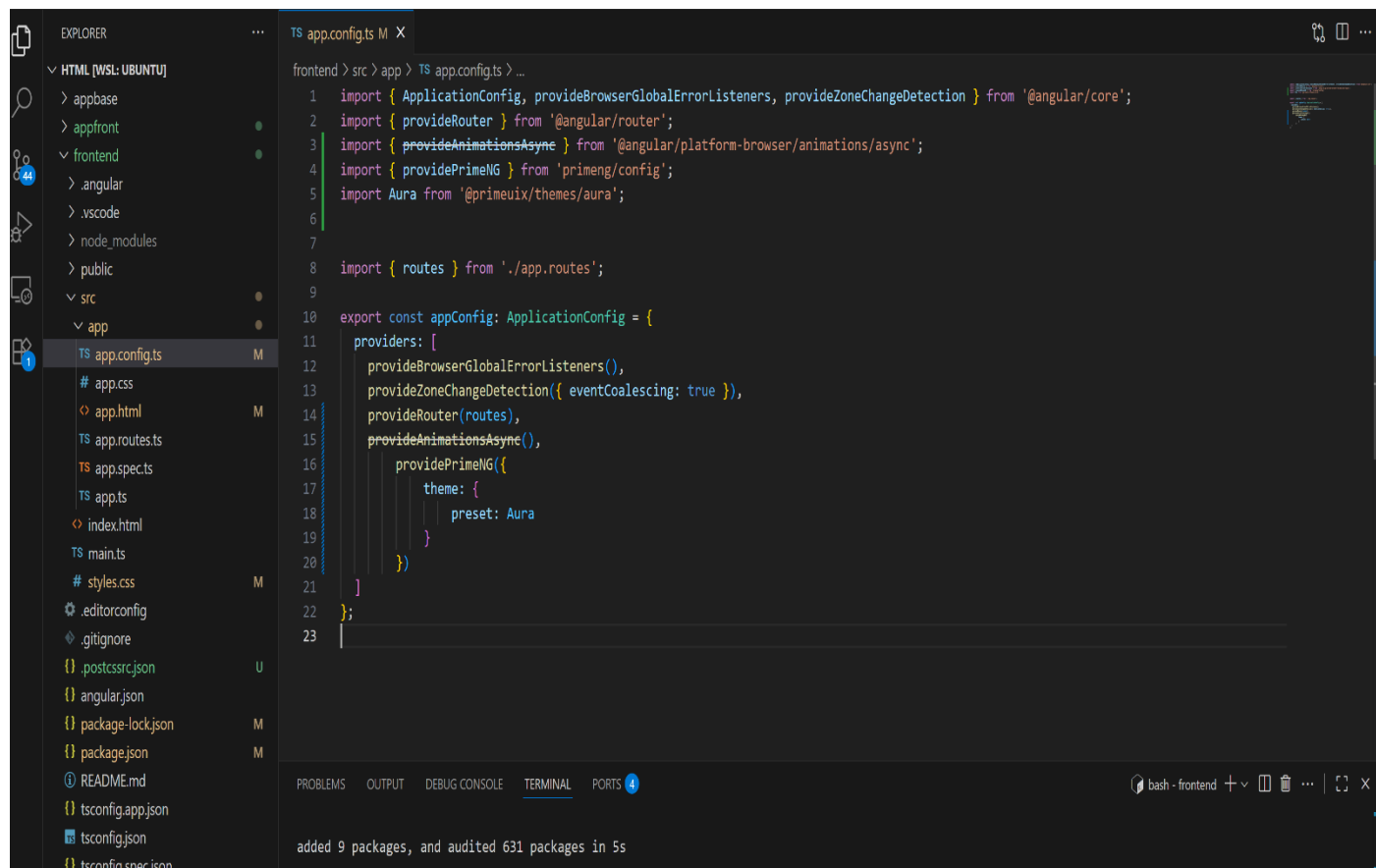
import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection }
from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { providePrimeNG } from 'primeng/config';
import Aura from '@primeuix/themes/aura';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura
      }
    })
  ]
};

```

Quedaria asi:



Creamos los componentes de Layout

```
ng g c components/layout/header
ng g c components/layout/aside
ng g c components/layout/footer
```

```
found 0 vulnerabilities
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/frontend$ ng g c components/layout/header
CREATE src/app/components/layout/header/header.css (0 bytes)
CREATE src/app/components/layout/header/header.spec.ts (528 bytes)
CREATE src/app/components/layout/header/header.ts (185 bytes)
CREATE src/app/components/layout/header/header.html (21 bytes)
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/frontend$ ng g c components/layout/aside
CREATE src/app/components/layout/aside/aside.css (0 bytes)
CREATE src/app/components/layout/aside/aside.spec.ts (521 bytes)
CREATE src/app/components/layout/aside/aside.ts (181 bytes)
CREATE src/app/components/layout/aside/aside.html (20 bytes)
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/frontend$ ng g c components/layout/footer
CREATE src/app/components/layout/footer/footer.css (0 bytes)
CREATE src/app/components/layout/footer/footer.spec.ts (528 bytes)
CREATE src/app/components/layout/footer/footer.ts (185 bytes)
CREATE src/app/components/layout/footer/footer.html (21 bytes)
jaider@DESKTOP-Q338KL1:~/app/dlloweb/2025iisem/html/frontend$
```

Aplicamos la propiedad

```
standalone: true,
```

A todo los componentes incluyendo el componente app-root principal en

frontend/src/app/app.ts

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('frontend');
}
```

frontend/src/app/components/layout/aside/aside.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-aside',
  standalone: true,
  imports: [],
  templateUrl: './aside.html',
  styleUrls: ['./aside.css']
})
export class Aside {
```



```
}
```

frontend/src/app/components/layout/footer/footer.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-footer',
  standalone: true,
  imports: [],
  templateUrl: './footer.html',
  styleUrls: ['./footer.css']
})
export class Footer {

}
```

frontend/src/app/components/layout/header/header.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-header',
  standalone: true,
  imports: [],
  templateUrl: './header.html',
  styleUrls: ['./header.css']
})
export class Header {

}
```

Luego se importan los componentes de layout en app-root

frontend/src/app/app.ts

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from './components/layout/header/header';
import { Footer } from './components/layout/footer/footer';
import { Aside } from './components/layout/aside/aside';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, Header, Footer, Aside],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('frontend');
}
```

frontend/src/app/app.html

```
<main class="min-h-screen flex flex-col border-4 border-red-500">
  <!-- HEADER -->
  <header class="h-16 bg-white border-2 border-blue-500 flex items-center justify-center font-
bold text-blue-600">
```

```

    <app-header></app-header>
  </header>

  <!-- CONTENEDOR CENTRAL (ASIDE + CONTENT) -->
  <div class="flex flex-1">
    <!-- ASIDE -->
    <aside class="w-64 bg-white border-2 border-blue-500 flex items-start justify-start font-
bold text-blue-400">
      <app-aside></app-aside>
    </aside>

    <!-- CONTENT -->
    <section class="flex-1 bg-white border-2 border-blue-500 flex items-center justify-center
font-bold text-blue-600">

      <router-outlet />
    </section>
  </div>

  <!-- FOOTER (pegado al fondo) -->
  <footer class="h-16 bg-white border-2 border-blue-500 flex items-center justify-center font-
bold text-blue-600 mt-auto">
    <app-footer></app-footer>
  </footer>
</main>

```

Levantamos el servidor

ng s

```

jaider@DESKTOP-Q338KL1:~/app/dlloweb/202511sem/html/frontend$ ng s
Initial chunk files | Names | Raw size
main.js | main | 9.84 kB
styles.css | styles | 6.53 kB
polyfills.js | polyfills | 95 bytes
| Initial total | 16.46 kB

Application bundle generation complete. [2.056 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
8:42:33 PM [vite] (client) Re-optimizing dependencies because lockfile has changed
8:42:33 PM [vite] (ssr) Re-optimizing dependencies because lockfile has changed (x2)
→ Local: http://localhost:4200/
→ press h + enter to show help

```

En el navegador se debe ver así:



Procedemos a insertar el menu en aside

Elegimos PanelMenu

frontend/src/app/components/layout/aside/aside.html

```
<div class="card flex justify-center">
  <p-panelmenu [model]="items" class="w-full md:w-80" />
</div>
```

frontend/src/app/components/layout/aside/aside.ts

```
import { Component } from '@angular/core';
import { MenuItem } from 'primeng/api';
import { PanelMenu } from 'primeng/panelmenu';

@Component({
  selector: 'app-aside',
  standalone: true,
  imports: [PanelMenu],
  templateUrl: './aside.html',
  styleUrls: ['./aside.css']
})
export class Aside {
  items: MenuItem[] | undefined;
  ngOnInit() {
    this.items = [
      {
        label: 'Clientes',

```

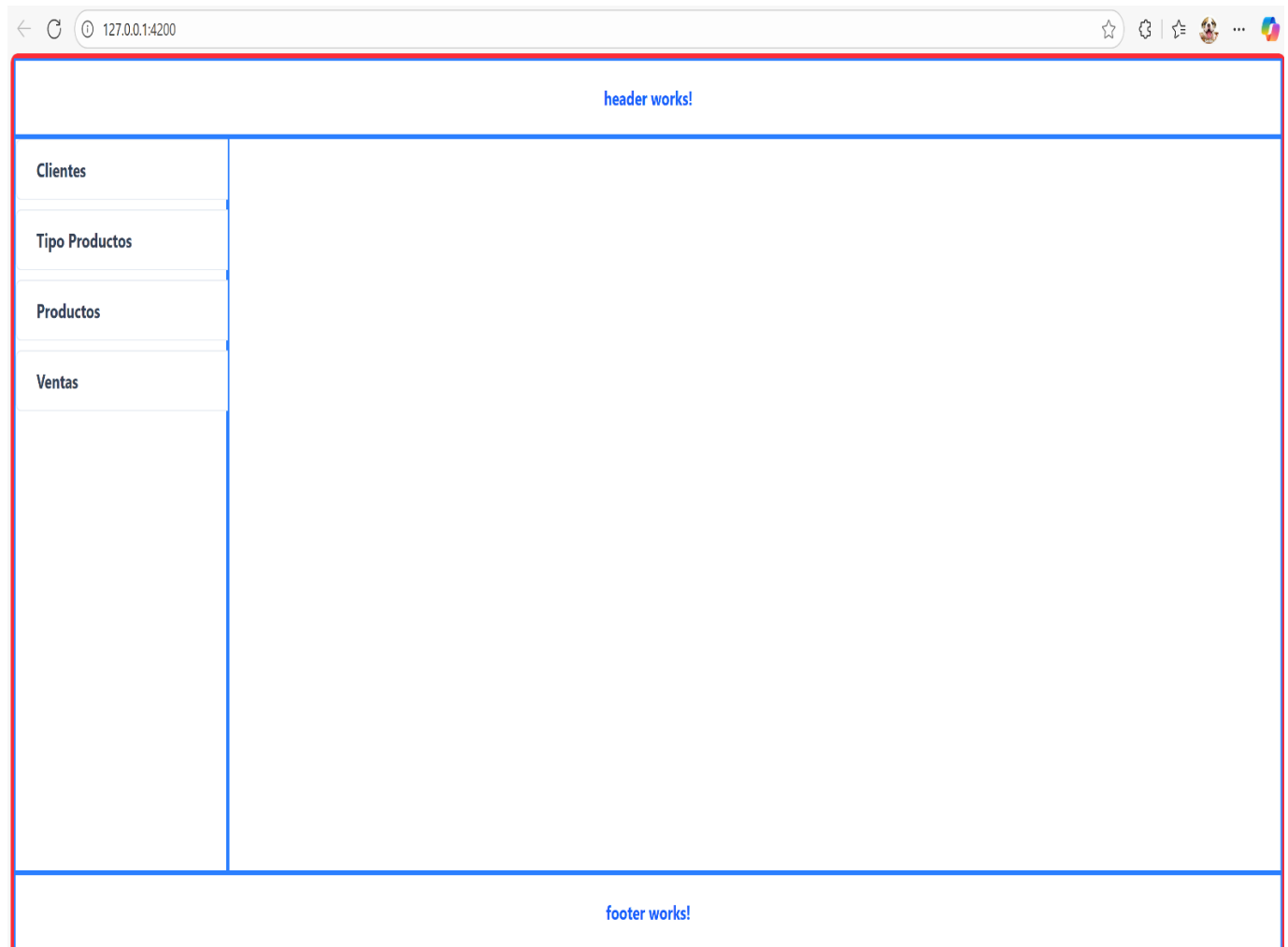
```

        icon: 'pi pi-fw pi-users',
      },
      {
        label: 'Tipo Productos',
        icon: 'pi pi-fw pi-qrcode',
      },
      {
        label: 'Productos',
        icon: 'pi pi-fw pi-shopping-bag',
      },
      {
        label: 'Ventas',
        icon: 'pi pi-fw pi-shopping-cart',
      }
    ];
  }
}

```

Levantamos el servidor

ng s



Podemos personalizar nuestro template como mejor nos parezca, eso queda a su estilo y criterio.

APIS- NODEJS

Navegar entre rutas de Angular

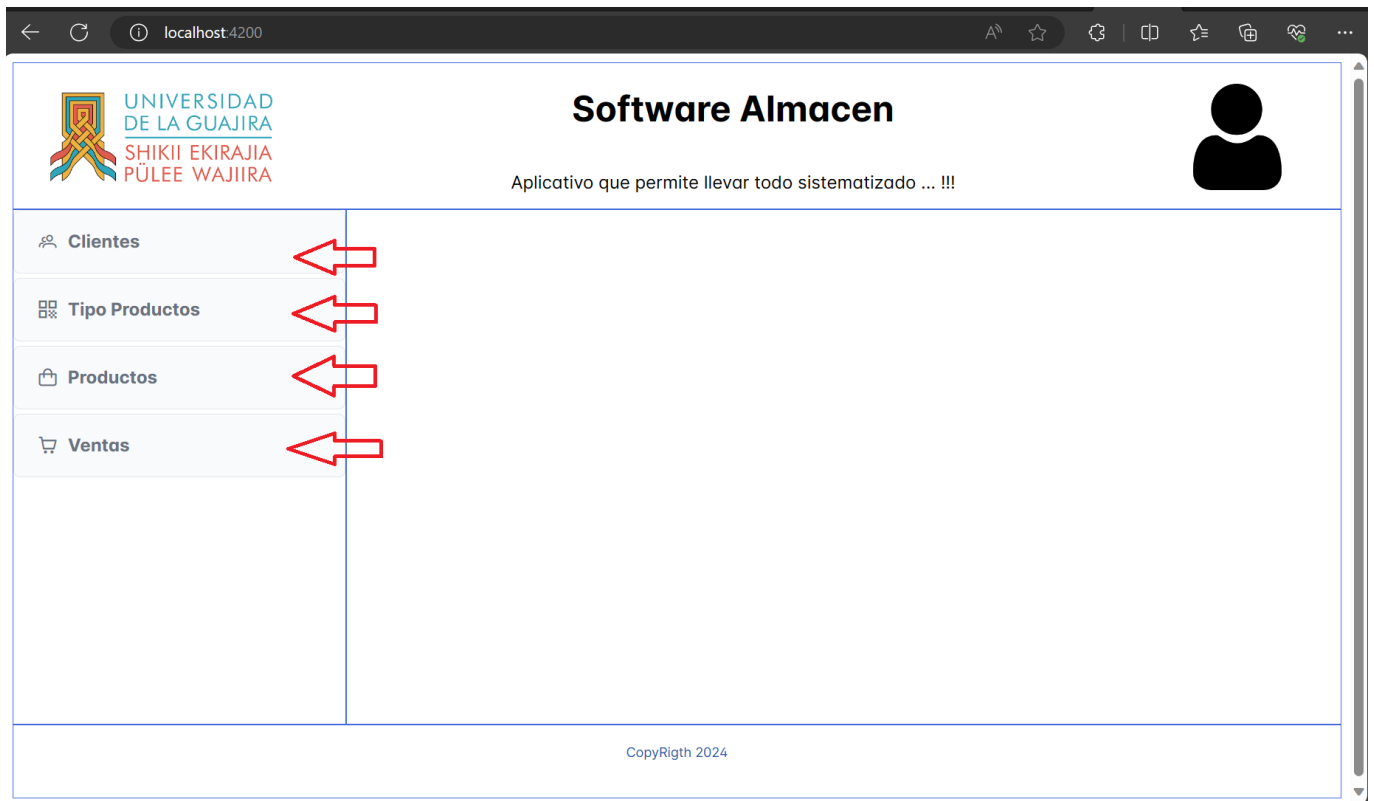
El enrutamiento en Angular ayuda a navegar de una vista a otra a medida que los usuarios realizan tareas en las aplicaciones web. En esta guía, aprenderá sobre la característica principal del enrutador Angular, RouterLink, y cómo usarla `routerLink` en sus aplicaciones Angular.

Las aplicaciones angulares son en su mayoría aplicaciones de una sola página, pero queremos mostrar varias páginas. Por lo tanto, necesitamos enrutamiento del lado del cliente en nuestra aplicación.

Angular viene con su propia biblioteca de enrutamiento del lado del cliente para permitirnos representar el componente que queremos de acuerdo con la URL a la que nos dirigimos. Podemos agregar enlaces que tengan como destino las URL de ruta para que cuando hagamos clic en el enlace, veamos el contenido que esperamos.

Para usarlo, primero tenemos que importar el `AppRoutingModule` módulo que vamos a crear que incluye el código de enrutamiento.

Luego creamos los componentes que hacen parte del ecosistema de componentes de la base de datos que se integran en el menú, y se desplegarán en el SPA.



Esto indica que debemos crear estos componentes:

Client ==> Cliente - CRUD

ProductType ==> TipoProducto - CRUD

Product ==> Producto - CRUD

Sale ==> Venta - CRUD

Componentes y Rutas para Client

Recordemos que para crear los componentes uno por cada verbo del CRUD, tal como sigue en el terminal de VS Code:

```
ng g c components/client/getall
ng g c components/client/create
ng g c components/client/update
ng g c components/client/delete
```

A continuación, asignamos URL a los componentes de ruta que acabamos de crear, pero antes debemos hacer las siguientes configuraciones en **src/app/app.config.ts**:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection }
from '@angular/core';
```

```

import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { providePrimeNG } from 'primeng/config';
import Aura from '@primeuix/themes/aura';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura
      }
    })
  ]
};

```

en **src/app/app.routes.ts**:

```

import { Routes } from '@angular/router';
import { Getall } from './components/client/getall/getall';
import { Create } from './components/client/create/create';
import { Update } from './components/client/update/update';
import { Delete } from './components/client/delete/delete';

export const routes: Routes = [
  {
    path: '',
    redirectTo: '/',
    pathMatch: 'full'
  },
  {
    path: "clients",
    component: Getall
  },
  {
    path: "clients/new",
    component: Create
  },
  {
    path: "clients/edit/:id",
    component: Update
  },
  {
    path: "clients/delete/:id",
    component: Delete
  },
];

```

Y luego, nuestro primer componente **MostrarCliente** lo enlazamos con el menu que tenemos en el componente **Aside**

src/app/components/layout/aside/aside.component.ts

```

import { Component } from '@angular/core';
import { MenuItem } from 'primeng/api';
import { PanelMenu } from 'primeng/panelmenu';

```

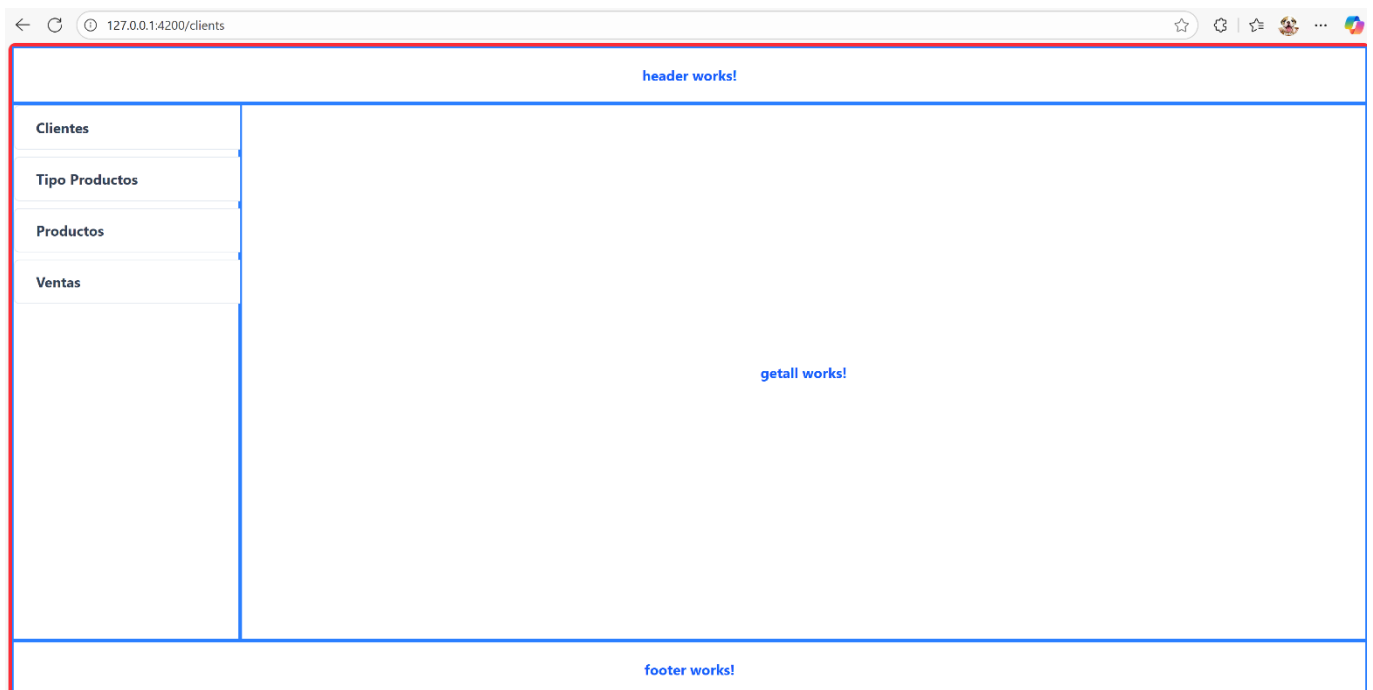
```

@Component({
  selector: 'app-aside',
  standalone: true,
  imports: [PanelMenu],
  templateUrl: './aside.html',
  styleUrls: ['./aside.css']
})
export class Aside {
  items: MenuItem[] | undefined;
  ngOnInit() {
    this.items = [
      {
        label: 'Clientes',
        icon: 'pi pi-fw pi-users',
        routerLink: '/clients',
      },
      {
        label: 'Tipo Productos',
        icon: 'pi pi-fw pi-qrcode',
      },
      {
        label: 'Productos',
        icon: 'pi pi-fw pi-shopping-bag',
      },
      {
        label: 'Ventas',
        icon: 'pi pi-fw pi-shopping-cart',
      }
    ];
  }
}

```

y vamos al navegador <http://localhost:4200>

Hasta este momento funciona la ruta de clientes y muestra el componente getall



Componentes y Rutas para Product

Recordemos que para crear los componentes uno por cada verbo del CRUD, tal como sigue en el terminal de VS Code:

```
ng g c components/product/getall
ng g c components/product/create
ng g c components/product/update
ng g c components/product/delete
```

A continuación:

src/app/app.routes.ts:

```
import { Routes } from '@angular/router';

// Client components with aliases
import { GetAll as ClientGetall } from './components/client/getall/getall';
import { Create as ClientCreate } from './components/client/create/create';
import { Update as ClientUpdate } from './components/client/update/update';
import { Delete as ClientDelete } from './components/client/delete/delete';

// Product components with aliases
import { GetAll as ProductGetall } from './components/product/getall/getall';
import { Create as ProductCreate } from './components/product/create/create';
import { Update as ProductUpdate } from './components/product/update/update';
import { Delete as ProductDelete } from './components/product/delete/delete';

export const routes: Routes = [
  {
    path: '',
    redirectTo: '/',
    pathMatch: 'full'
  },
  {
    path: "clients",
    component: ClientGetall
  },
  {
    path: "clients/new",
    component: ClientCreate
  },
  {
    path: "clients/edit/:id",
    component: ClientUpdate
  },
  {
    path: "clients/delete/:id",
    component: ClientDelete
  },
  {
    path: "products",
    component: ProductGetall
  },
  {
    path: "products/new",
    component: ProductCreate
  },
  {
    path: "products/edit/:id",
    component: ProductUpdate
  }
]
```

```

    },
    {
      path: "products/delete/:id",
      component: ProductDelete
    },
  ],
];

```

Nota: Para que no haya conflicto en las clases dado que tanto en Client, Como en Product se llaman iguales (GetAll, Create, Update y Delete) entonces usamos alias **Getall** as **ClientGetall** para todas las demas clases.

src/app/components/layout/aside/aside.component.ts

```

import { Component } from '@angular/core';
import { MenuItem } from 'primeng/api';
import { PanelMenu } from 'primeng/panelmenu';

@Component({
  selector: 'app-aside',
  standalone: true,
  imports: [PanelMenu],
  templateUrl: './aside.html',
  styleUrls: ['./aside.css']
})
export class Aside {
  items: MenuItem[] | undefined;
  ngOnInit() {
    this.items = [
      {
        label: 'Clientes',
        icon: 'pi pi-fw pi-users',
        routerLink: '/clients',
      },
      {
        label: 'Tipo Productos',
        icon: 'pi pi-fw pi-qrcode',
      },
    ],
    {
      label: 'Productos',
      icon: 'pi pi-fw pi-shopping-bag',
      routerLink: '/products',
    },
    {
      label: 'Ventas',
      icon: 'pi pi-fw pi-shopping-cart',
    }
  ];
}
}

```

y vamos al navegador <http://localhost:4200>



Componentes y Rutas para Sale

Recordemos que para crear los componentes uno por cada verbo del CRUD, tal como sigue en el terminal de VS Code:

```
ng g c components/sale/getall
ng g c components/sale/create
ng g c components/sale/update
ng g c components/sale/delete
```

A continuación:

src/app/app.routes.ts:

```
import { Routes } from '@angular/router';

// Client components with aliases
import { Getall as ClientGetall } from './components/client/getall/getall';
import { Create as ClientCreate } from './components/client/create/create';
import { Update as ClientUpdate } from './components/client/update/update';
import { Delete as ClientDelete } from './components/client/delete/delete';

// Product components with aliases
import { Getall as ProductGetall } from './components/product/getall/getall';
import { Create as ProductCreate } from './components/product/create/create';
import { Update as ProductUpdate } from './components/product/update/update';
import { Delete as ProductDelete } from './components/product/delete/delete';

// Sale compones with aliases
import { Getall as SaleGetall } from './components/sale/getall/getall';
import { Create as SaleCreate } from './components/sale/create/create';
import { Update as SaleSaleUpdate } from './components/sale/update/update';
import { Delete as SaleDelete } from './components/sale/delete/delete';
```

```

export const routes: Routes = [
  {
    path: '',
    redirectTo: '/',
    pathMatch: 'full'
  },
  {
    path: "clients",
    component: ClientGetall
  },
  {
    path: "clients/new",
    component: ClientCreate
  },
  {
    path: "clients/edit/:id",
    component: ClientUpdate
  },
  {
    path: "clients/delete/:id",
    component: ClientDelete
  },
  {
    path: "products",
    component: ProductGetall
  },
  {
    path: "products/new",
    component: ProductCreate
  },
  {
    path: "products/edit/:id",
    component: ProductUpdate
  },
  {
    path: "products/delete/:id",
    component: ProductDelete
  },
  {
    path: "sales",
    component: SaleGetall
  },
  {
    path: "sales/new",
    component: SaleCreate
  },
  {
    path: "sales/edit/:id",
    component: SaleSaleUpdate
  },
  {
    path: "sales/delete/:id",
    component: SaleDelete
  },
];

```

src/app/components/layout/aside/aside.component.ts

```

import { Component } from '@angular/core';
import { MenuItem } from 'primeng/api';
import { PanelMenu } from 'primeng/panelmenu';

@Component({

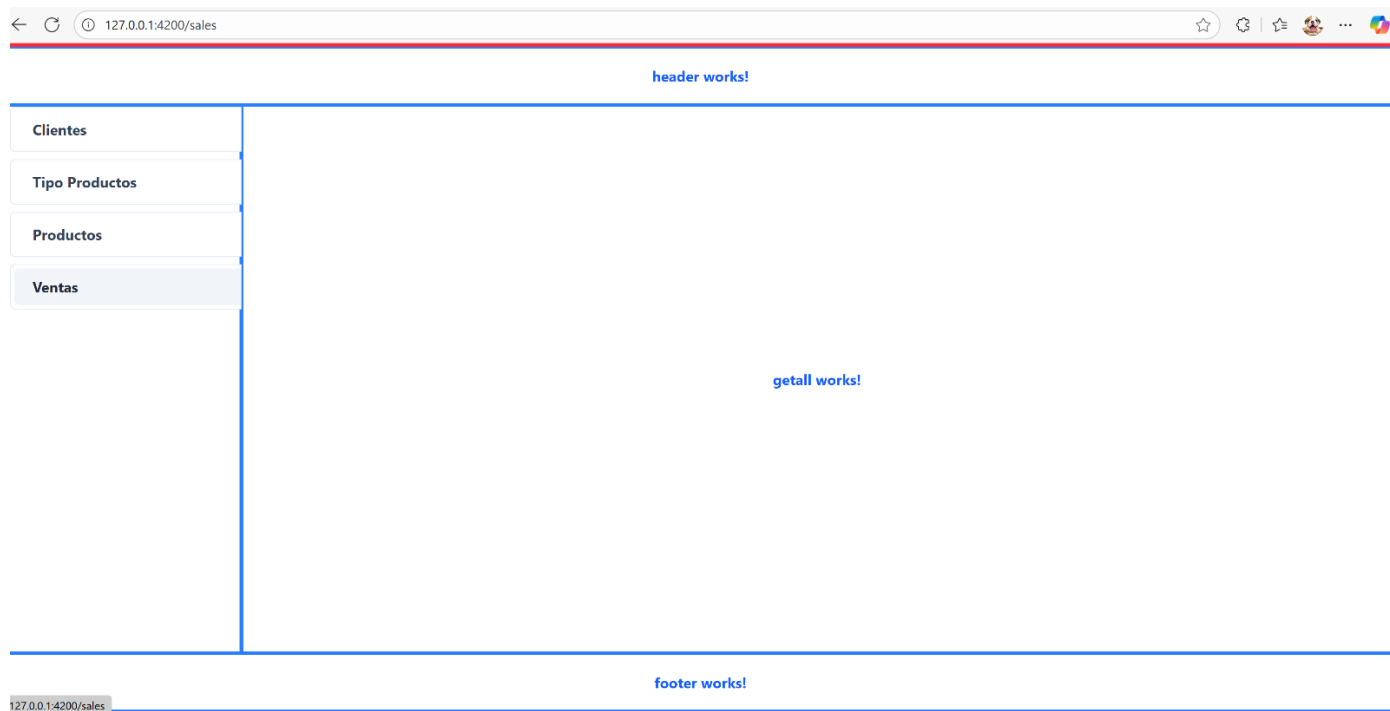
```

```

    selector: 'app-aside',
    standalone: true,
    imports: [PanelMenu],
    templateUrl: './aside.html',
    styleUrls: ['./aside.css']
  })
}
export class Aside {
  items: MenuItem[] | undefined;
  ngOnInit() {
    this.items = [
      {
        label: 'Clientes',
        icon: 'pi pi-fw pi-users',
        routerLink: '/clients',
      },
      {
        label: 'Tipo Productos',
        icon: 'pi pi-fw pi-qrcode',
      },
      {
        label: 'Productos',
        icon: 'pi pi-fw pi-shopping-bag',
        routerLink: '/products',
      },
      {
        label: 'Ventas',
        icon: 'pi pi-fw pi-shopping-cart',
        routerLink: '/sales',
      }
    ];
  }
}

```

y vamos al navegador <http://localhost:4200>



Modelo Client - CRUD

Crearemos las peticiones desde el frontend para el modelo Client.

Para ello realizamos paso a paso para que relliquemos lo mismo a los demas modelos

1. Realizamos el Modelo Client

Creamos el modelo en `src/app/models/client.ts`

```
export interface ClientI {  
  id?: number;  
  name: string;  
  address: string;  
  phone: string;  
  email: string;  
  password: string;  
  status: "ACTIVE" | "INACTIVE";  
}
```

```
export interface ClientResponseI {  
  id?: number;  
  name: string;  
  address: string;  
  phone: string;  
  email: string;  
}
```

2. Realizamos el Servicio Client

Creamos el modelo en `src/app/services/client.ts`

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable, BehaviorSubject, tap, catchError, throwError } from 'rxjs';  
import { map } from 'rxjs/operators';  
import { ClientI, ClientResponseI } from '../models/client';
```

// Interfaz para la respuesta paginada de Django

```
interface PaginatedResponse<T> {  
  count: number;  
  next: string | null;  
  previous: string | null;  
  results: T[];  
}
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class ClientService {  
  private baseUrl = 'http://localhost:8000/api/clients';  
  private clientsSubject = new BehaviorSubject<ClientResponseI[]>([]);  
  public clients$ = this.clientsSubject.asObservable();  
}
```

```
constructor(private http: HttpClient) {}
```

```
getAllClients(): Observable<ClientResponseI[]> {  
  return this.http.get<PaginatedResponse<ClientResponseI>>(`${this.baseUrl}/`)  
    .pipe(  
      map(response => response.results), // Extraer solo el array de results  
      tap(clients => {  
        console.log('Fetched clients:', clients);  
        this.clientsSubject.next(clients);  
      }),  
      catchError(error => {  
        console.error('Error fetching clients:', error);  
        return throwError(() => error);  
      })  
    );  
}
```

```
getClientById(id: number): Observable<ClientResponseI> {  
  return this.http.get<ClientResponseI>(`${this.baseUrl}/${id}/`)  
    .pipe(  
      catchError(error => {  
        console.error('Error fetching client:', error);  
        return throwError(() => error);  
      })  
    );  
}
```

```
createClient(client: ClientI): Observable<ClientResponseI> {  
  return this.http.post<ClientResponseI>(`${this.baseUrl}/`, client)  
    .pipe(  
      tap(response => {  
        console.log('Client created:', response);  
        this.refreshClients();  
      }),  
      catchError(error => {  
        console.error('Error creating client:', error);  
        return throwError(() => error);  
      })  
    );  
}
```

```
updateClient(id: number, client: Partial<ClientI>): Observable<ClientResponseI> {  
  return this.http.put<ClientResponseI>(`${this.baseUrl}/${id}/`, client)  
    .pipe(  
      tap(response => {  
        console.log('Client updated:', response);  
        this.refreshClients();  
      }),  
      catchError(error => {  
        console.error('Error updating client:', error);  
        return throwError(() => error);  
      })  
    );  
}
```

```
partialUpdateClient(id: number, client: Partial<ClientI>): Observable<ClientResponseI> {  
  return this.http.patch<ClientResponseI>(`${this.baseUrl}/${id}/`, client)  
    .pipe(  
      tap(response => {  
        console.log('Client partially updated:', response);  
        this.refreshClients();  
      }),  
      catchError(error => {  
        console.error('Error partially updating client:', error);  
      })  
    );  
}
```

```

        return throwError(() => error);
    })
    );
}

deleteClient(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}/`)
        .pipe(
            tap(() => {
                console.log('Client deleted:', id);
                this.refreshClients();
            }),
            catchError(error => {
                console.error('Error deleting client:', error);
                return throwError(() => error);
            })
        );
}

// Método para actualizar el estado local de clientes
updateLocalClients(clients: ClientResponseI[]): void {
    this.clientsSubject.next(clients);
}

refreshClients(): void {
    this.getAllClients().subscribe({
        next: (clients) => {
            this.clientsSubject.next(clients);
        },
        error: (error) => {
            console.error('Error refreshing clients:', error);
        }
    });
}
}

```

2. Enlazamos con el componente Client

a. Iniciamos con mostrar todo (getall.ts y getall.html)

src/app/components/client/getall.ts

```

import { Component, OnInit, OnDestroy } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';
import { TableModule } from 'primeng/table';
import { Button } from 'primeng/button';
import { ConfirmDialog } from 'primeng/confirmdialog';
import { Toast } from 'primeng/toast';
import { Tooltip } from 'primeng/tooltip';
import { ConfirmationService, MessageService } from 'primeng/api';
import { Subscription } from 'rxjs';
import { ClientService } from '../../../services/client.service';
import { ClientResponseI } from '../../../models/client';

@Component({
    selector: 'app-client-getall',
    standalone: true,
    imports: [
        CommonModule,
        RouterModule,
        TableModule,

```



```

        Button,
        ConfirmDialog,
        Toast,
        Tooltip
    ],
    providers: [ConfirmationService, MessageService],
    templateUrl: './getall.html',
    styleUrls: ['./getall.css']
})
export class Getall implements OnInit, OnDestroy {
    clients: ClientResponseI[] = [];
    loading = false;
    private subscription = new Subscription();

    constructor(
        private clientService: ClientService,
        private confirmationService: ConfirmationService,
        private messageService: MessageService
    ) {}

    ngOnInit(): void {
        this.loadClients();

        // Subscribe to client updates
        this.subscription.add(
            this.clientService.clients$.subscribe(clients => {
                this.clients = clients;
            })
        );
    }

    ngOnDestroy(): void {
        this.subscription.unsubscribe();
    }

    loadClients(): void {
        this.loading = true;
        this.subscription.add(
            this.clientService.getAllClients().subscribe({
                next: (clients) => {
                    this.clients = clients;
                    this.loading = false;
                },
                error: (error) => {
                    console.error('Error loading clients:', error);
                    this.messageService.add({
                        severity: 'error',
                        summary: 'Error',
                        detail: 'No se pudieron cargar los clientes'
                    });
                    this.loading = false;
                }
            })
        );
    }

    confirmDelete(client: ClientResponseI): void {
        this.confirmationService.confirm({
            message: `¿Está seguro de que desea eliminar a ${client.name}?`,
            header: 'Confirmar Eliminación',
            icon: 'pi pi-exclamation-triangle',
            acceptLabel: 'Sí, eliminar',
            rejectLabel: 'Cancelar',
            acceptButtonStyleClass: 'p-button-danger',
            accept: () => {

```

```

        this.deleteClient(client.id!);
    }
    });
}

deleteClient(id: number): void {
    this.subscription.add(
        this.clientService.deleteClient(id).subscribe({
            next: () => {
                this.messageService.add({
                    severity: 'success',
                    summary: 'Éxito',
                    detail: 'Cliente eliminado correctamente'
                });
            },
            error: (error) => {
                console.error('Error deleting client:', error);
                this.messageService.add({
                    severity: 'error',
                    summary: 'Error',
                    detail: 'No se pudo eliminar el cliente'
                });
            }
        })
    );
}
}
}

```

src/app/components/client/getall.html

```

<div class="card p-8">
    <div class="flex justify-between items-center mb-4">
        <h2 class="text-2xl font-bold">Gestión de Clientes</h2>
        <p-button
            label="Nuevo Cliente"
            icon="pi pi-plus"
            [routerLink]="['/clients/new']"
            styleClass="p-button-success"
        ></p-button>
    </div>

    <p-table
        [value]="clients"
        [loading]="loading"
        [paginator]="true"
        [rows]="10"
        [showCurrentPageReport]="true"
        currentPageReportTemplate="Mostrando {first} a {last} de {totalRecords} registros"
        [rowsPerPageOptions]="[5, 10, 25, 50]"
        dataKey="id"
        styleClass="p-datatable-striped"
        [tableStyle]="{'min-width': '50rem'}"
    >
        <ng-template pTemplate="header">
            <tr>
                <th pSortableColumn="id">
                    ID <p-sortIcon field="id"></p-sortIcon>
                </th>
                <th pSortableColumn="name">
                    Nombre <p-sortIcon field="name"></p-sortIcon>
                </th>
                <th pSortableColumn="address">
                    Dirección <p-sortIcon field="address"></p-sortIcon>
                </th>
            </tr>

```

```

        <th pSortableColumn="phone">
            Teléfono <p-sortIcon field="phone"></p-sortIcon>
        </th>
        <th pSortableColumn="email">
            Email <p-sortIcon field="email"></p-sortIcon>
        </th>
        <th class="text-center">Acciones</th>
    </tr>
</ng-template>

<ng-template pTemplate="body" let-client>
    <tr>
        <td>{{ client.id }}</td>
        <td>{{ client.name }}</td>
        <td>{{ client.address }}</td>
        <td>{{ client.phone }}</td>
        <td>{{ client.email }}</td>
        <td class="text-center">
            <div class="flex justify-center gap-2">
                <p-button
                    icon="pi pi-pencil"
                    [routerLink]="['/clients/edit', client.id]"
                    styleClass="p-button-rounded p-button-text p-button-warning"
                    pTooltip="Editar"
                    tooltipPosition="top"
                ></p-button>
                <p-button
                    icon="pi pi-trash"
                    (onClick)="confirmDelete(client)"
                    styleClass="p-button-rounded p-button-text p-button-danger"
                    pTooltip="Eliminar"
                    tooltipPosition="top"
                ></p-button>
            </div>
        </td>
    </tr>
</ng-template>

<ng-template pTemplate="emptymessage">
    <tr>
        <td colspan="6" class="text-center p-4">
            <div class="flex flex-col items-center gap-3">
                <i class="pi pi-info-circle text-4xl text-gray-400"></i>
                <span class="text-gray-500">No se encontraron clientes</span>
                <p-button
                    label="Crear Primer Cliente"
                    icon="pi pi-plus"
                    [routerLink]="['/clients/new']"
                    styleClass="p-button-sm p-button-outlined"
                ></p-button>
            </div>
        </td>
    </tr>
</ng-template>
</p-table>
</div>

<p-confirmDialog></p-confirmDialog>
<p-toast></p-toast>

```

Confirmamos las rutas en `src/app.routes.ts`

```
import { Routes } from '@angular/router';
```

```

// Client components with aliases
import { Getall as ClientGetall } from './components/client/getall/getall';
import { Create as ClientCreate } from './components/client/create/create';
import { Update as ClientUpdate } from './components/client/update/update';
import { Delete as ClientDelete } from './components/client/delete/delete';

// Product components with aliases
import { Getall as ProductGetall } from './components/product/getall/getall';
import { Create as ProductCreate } from './components/product/create/create';
import { Update as ProductUpdate } from './components/product/update/update';
import { Delete as ProductDelete } from './components/product/delete/delete';

// Sale components with aliases
import { Getall as SaleGetall } from './components/sale/getall/getall';
import { Create as SaleCreate } from './components/sale/create/create';
import { Update as SaleUpdate } from './components/sale/update/update';
import { Delete as SaleDelete } from './components/sale/delete/delete';

export const routes: Routes = [
  {
    path: '',
    redirectTo: '/clients',
    pathMatch: 'full'
  },
  {
    path: "clients",
    component: ClientGetall
  },
  {
    path: "clients/new",
    component: ClientCreate
  },
  {
    path: "clients/edit/:id",
    component: ClientUpdate
  },
  {
    path: "clients/delete/:id",
    component: ClientDelete
  },
  {
    path: "products",
    component: ProductGetall
  },
  {
    path: "products/new",
    component: ProductCreate
  },
  {
    path: "products/edit/:id",
    component: ProductUpdate
  },
  {
    path: "products/delete/:id",
    component: ProductDelete
  },
  {
    path: "sales",
    component: SaleGetall
  },
  {
    path: "sales/new",
    component: SaleCreate
  },
  {

```

```

        path: "sales/edit/:id",
        component: SaleUpdate
      },
      {
        path: "sales/delete/:id",
        component: SaleDelete
      },
    ],
  ];

```

Confirmamos en `src/app.config.ts`

```

import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection }
from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';
import { providePrimeNG } from 'primeng/config';
import Aura from '@primeui/themes/aura';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptorsFromDi()),
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura
      }
    })
  ]
};

```

Al ejecutar en el terminal `ng serve`

ID	Nombre	Dirección	Teléfono	Email	Estado	Acciones
1	Keith Hansen	23724 Birch Road	(426) 964-8485 x611	Jackie.Batz-Ferry25@gmail.com	ACTIVE	
2	Jenna Ward V	866 Kovacek Crossroad	1-841-265-6203 x8509	Gavin_Luettgen-Murphy@gmail.com	ACTIVE	
3	Abel Bailey	5075 Church Hill	1-652-379-0495 x001	Raven.Nikolaus2@hotmail.com	ACTIVE	
4	Marc Turner	181 Guido Roads	(525) 576-1917 x86226	Lemuel_Paucek30@hotmail.com	ACTIVE	
5	Terrance Will IV	276 Poplar Close	1-587-558-3286 x8384	Cordia52@yahoo.com	ACTIVE	
6	Tara Mills	3940 Reta Island	(802) 313-0260	Alanna7@gmail.com	ACTIVE	
7	Lauren Beatty	4291 London Road	640-748-3556 x311	Melvina_Bayer11@gmail.com	ACTIVE	
8	Antonio Cumerata	68752 W Broadway Street	569-479-1576 x211	Ezequiel.Brown@yahoo.com	ACTIVE	

Nota: Si no te aparecen los iconos en las acciones, es necesario instalar **PrimeIcons**

En el terminal

```
npm install primeicons
```

Y en el archivo `src/styles.css` adicional al final

```
@import "primeicons/primeicons.css";
```

b. Create (create.ts y create.html)

`src/app/components/client/create.ts`

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { ButtonModule } from 'primeng/button';
import { InputTextModule } from 'primeng/inputtext';
import { Select } from 'primeng/select';
import { MessageService } from 'primeng/api';
import { ToastModule } from 'primeng/toast';
import { ClientService } from '../../services/client.service';

@Component({
  selector: 'app-create',
  imports: [CommonModule, ReactiveFormsModule, ButtonModule, InputTextModule, Select,
    ToastModule],
  templateUrl: './create.html',
  styleUrls: ['./create.css'],
  providers: [MessageService]
```

```

})
export class Create {
  form: FormGroup;
  loading: boolean = false;
  statusOptions = [
    { label: 'Activo', value: 'ACTIVE' },
    { label: 'Inactivo', value: 'INACTIVE' }
  ];

  constructor(
    private fb: FormBuilder,
    private router: Router,
    private clientService: ClientService,
    private messageService: MessageService
  ) {
    this.form = this.fb.group({
      name: ['', [Validators.required, Validators.minLength(2)]],
      address: ['', [Validators.required, Validators.minLength(5)]],
      phone: ['', [Validators.required, Validators.pattern(/^\d{10,15}$/)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]],
      status: ['ACTIVE', Validators.required]
    });
  }

  submit(): void {
    if (this.form.valid) {
      this.loading = true;
      const clientData = this.form.value;

      this.clientService.createClient(clientData).subscribe({
        next: (response) => {
          this.messageService.add({
            severity: 'success',
            summary: 'Éxito',
            detail: 'Cliente creado correctamente'
          });
          setTimeout(() => {
            this.router.navigate(['/clients']);
          }, 1000);
        },
        error: (error) => {
          console.error('Error creating client:', error);
          this.messageService.add({
            severity: 'error',
            summary: 'Error',
            detail: 'Error al crear el cliente'
          });
          this.loading = false;
        }
      });
    } else {
      this.markFormGroupTouched();
      this.messageService.add({
        severity: 'warn',
        summary: 'Advertencia',
        detail: 'Por favor complete todos los campos requeridos'
      });
    }
  }

  cancelar(): void {
    this.router.navigate(['/clients']);
  }
}

```

```

private markFormGroupTouched(): void {
  Object.keys(this.form.controls).forEach(key => {
    this.form.get(key)?.markAsTouched();
  });
}

getFieldError(fieldName: string): string {
  const field = this.form.get(fieldName);
  if (field?.errors && field?.touched) {
    if (field.errors['required']) return `${fieldName} es requerido`;
    if (field.errors['email']) return 'Email no válido';
    if (field.errors['minlength']) return `${fieldName} debe tener al menos
    ${field.errors['minlength'].requiredLength} caracteres`;
    if (field.errors['pattern']) return 'Formato no válido';
  }
  return '';
}
}

```

src/app/components/client/create.html

```

<div class="max-w-4xl mx-auto p-6">
  <div class="bg-white rounded-lg shadow-lg p-8">
    <h2 class="text-2xl font-bold mb-6 text-gray-800">Crear Nuevo Cliente</h2>

    <form [formGroup]="form" (ngSubmit)="submit()">
      <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Nombre *</label>
          <input
            type="text"
            [formControlName]="name"
            class="w-full"
            [class.ng-invalid]="form.get('name')?.invalid && form.get('name')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('name')">
            {{ getFieldError('name') }}
          </small>
        </div>

        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Dirección *</label>
          <input
            type="text"
            [formControlName]="address"
            class="w-full"
            [class.ng-invalid]="form.get('address')?.invalid && form.get('address')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('address')">
            {{ getFieldError('address') }}
          </small>
        </div>

        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Teléfono *</label>
          <input
            type="text"
            [formControlName]="phone"
            class="w-full"
            placeholder="1234567890"
            [class.ng-invalid]="form.get('phone')?.invalid && form.get('phone')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('phone')">
            {{ getFieldError('phone') }}
          </small>
        </div>
      </div>
    </form>
  </div>
</div>

```



```
</small>
</div>
```

```
<div class="flex flex-col">
  <label class="block mb-2 font-semibold text-gray-700">Email *</label>
  <input
    pInputText
    type="email"
    formControlName="email"
    class="w-full"
    [class.ng-invalid]="form.get('email')?.invalid && form.get('email')?.touched"
  />
  <small class="text-red-500 mt-1" *ngIf="getFieldError('email')">
    {{ getFieldError('email') }}
  </small>
</div>
```

```
<div class="flex flex-col">
  <label class="block mb-2 font-semibold text-gray-700">Contraseña *</label>
  <input
    pInputText
    type="password"
    formControlName="password"
    class="w-full"
    [class.ng-invalid]="form.get('password')?.invalid &&
form.get('password')?.touched"
  />
  <small class="text-red-500 mt-1" *ngIf="getFieldError('password')">
    {{ getFieldError('password') }}
  </small>
</div>
```

```
<div class="flex flex-col">
  <label class="block mb-2 font-semibold text-gray-700">Estado *</label>
  <p-select
    formControlName="status"
    [options]="statusOptions"
    placeholder="Seleccionar estado"
    class="w-full"
  ></p-select>
</div>
</div>
```

```
<div class="flex justify-center mt-8">
  <div class="bg-gray-50 rounded-lg p-4 flex gap-4 items-center">
    <button
      pButton
      type="submit"
      class="p-button bg-blue-500 hover:bg-blue-600 text-white"
      label="Guardar"
      icon="pi pi-save"
      [loading]="loading"
      [disabled]="loading"
    ></button>
    <button
      pButton
      type="button"
      class="p-button p-button-secondary"
      label="Cancelar"
      icon="pi pi-times"
      (click)="cancelar()"
      [disabled]="loading"
    ></button>
  </div>
</div>
```

```

    </form>
  </div>
</div>

<p-toast></p-toast>

```

c. Create (update.ts y update.html)

src/app/components/client/update.ts

```

import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { ButtonModule } from 'primeng/button';
import { InputTextModule } from 'primeng/inputtext';
import { Select } from 'primeng/select';
import { MessageService } from 'primeng/api';
import { ToastModule } from 'primeng/toast';
import { ClientService } from '../../../services/client.service';
import { ClientI } from '../../../models/client';

```

```

@Component({
  selector: 'app-update',
  imports: [CommonModule, ReactiveFormsModule, ButtonModule, InputTextModule, Select,
    ToastModule],
  templateUrl: './update.html',
  styleUrls: ['./update.css'],
  providers: [MessageService]
})

```

```

export class Update implements OnInit {
  form: FormGroup;
  loading: boolean = false;
  clientId: number = 0;
  statusOptions = [
    { label: 'Activo', value: 'ACTIVE' },
    { label: 'Inactivo', value: 'INACTIVE' }
  ];
}

```

```

constructor(
  private fb: FormBuilder,
  private router: Router,
  private route: ActivatedRoute,
  private clientService: ClientService,
  private messageService: MessageService
) {
  this.form = this.fb.group({
    name: ['', [Validators.required, Validators.minLength(2)]],
    address: ['', [Validators.required, Validators.minLength(5)]],
    phone: ['', [Validators.required, Validators.pattern(/^\d{10,15}$/)]],
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
    status: ['ACTIVE', Validators.required]
  });
}

```

```

ngOnInit(): void {
  const id = this.route.snapshot.paramMap.get('id');
  if (id) {
    this.clientId = parseInt(id);
    this.loadClient();
  }
}

```

```

loadClient(): void {
  this.loading = true;
  this.clientService.getClientById(this.clientId).subscribe({
    next: (client) => {
      this.form.patchValue(client);
      this.loading = false;
    },
    error: (error) => {
      console.error('Error loading client:', error);
      this.messageService.add({
        severity: 'error',
        summary: 'Error',
        detail: 'Error al cargar el cliente'
      });
      this.loading = false;
    }
  });
}

```

```

submit(): void {
  if (this.form.valid) {
    this.loading = true;
    const clientData = this.form.value;

```

```

    this.clientService.updateClient(this.clientId, clientData).subscribe({
      next: (response) => {
        this.messageService.add({
          severity: 'success',
          summary: 'Éxito',
          detail: 'Cliente actualizado correctamente'
        });
        setTimeout(() => {
          this.router.navigate(['/clients']);
        }, 1000);
      },
      error: (error) => {
        console.error('Error updating client:', error);
        this.messageService.add({
          severity: 'error',
          summary: 'Error',
          detail: 'Error al actualizar el cliente'
        });
        this.loading = false;
      }
    });
  } else {
    this.markFormGroupTouched();
    this.messageService.add({
      severity: 'warn',
      summary: 'Advertencia',
      detail: 'Por favor complete todos los campos requeridos'
    });
  }
}

```

```

cancelar(): void {
  this.router.navigate(['/clients']);
}

```

```

private markFormGroupTouched(): void {
  Object.keys(this.form.controls).forEach(key => {
    this.form.get(key)?.markAsTouched();
  });
}

```

```

getFieldError(fieldName: string): string {
  const field = this.form.get(fieldName);
  if (field?.errors && field?.touched) {
    if (field.errors['required']) return `${fieldName} es requerido`;
    if (field.errors['email']) return 'Email no válido';
    if (field.errors['minlength']) return `${fieldName} debe tener al menos
    ${field.errors['minlength'].requiredLength} caracteres`;
    if (field.errors['pattern']) return 'Formato no válido';
  }
  return '';
}
}

```

src/app/components/client/update.html

```

<div class="max-w-4xl mx-auto p-6">
  <div class="bg-white rounded-lg shadow-lg p-8">
    <h2 class="text-2xl font-bold mb-6 text-gray-800">Editar Cliente</h2>

    <form [formGroup]="form" (ngSubmit)="submit()" *ngIf="!loading">
      <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Nombre *</label>
          <input
            type="text"
            [formControlName]="name"
            class="w-full"
            [class.ng-invalid]="form.get('name')?.invalid && form.get('name')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('name')">
            {{ getFieldError('name') }}
          </small>
        </div>

        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Dirección *</label>
          <input
            type="text"
            [formControlName]="address"
            class="w-full"
            [class.ng-invalid]="form.get('address')?.invalid && form.get('address')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('address')">
            {{ getFieldError('address') }}
          </small>
        </div>

        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Teléfono *</label>
          <input
            type="text"
            [formControlName]="phone"
            class="w-full"
            [class.ng-invalid]="form.get('phone')?.invalid && form.get('phone')?.touched"
          />
          <small class="text-red-500 mt-1" *ngIf="getFieldError('phone')">
            {{ getFieldError('phone') }}
          </small>
        </div>

        <div class="flex flex-col">
          <label class="block mb-2 font-semibold text-gray-700">Email *</label>
          <input
            type="text"

```

```

        pInputText
        type="email"
        formControlName="email"
        class="w-full"
        [class.ng-invalid]="form.get('email')?.invalid && form.get('email')?.touched"
    />
    <small class="text-red-500 mt-1" *ngIf="getFieldError('email')">
        {{ getFieldError('email') }}
    </small>
</div>

```

```

<div class="flex flex-col">
    <label class="block mb-2 font-semibold text-gray-700">Contraseña *</label>
    <input
        pInputText
        type="password"
        formControlName="password"
        class="w-full"
        [class.ng-invalid]="form.get('password')?.invalid &&
form.get('password')?.touched"
    />
    <small class="text-red-500 mt-1" *ngIf="getFieldError('password')">
        {{ getFieldError('password') }}
    </small>
</div>

```

```

<div class="flex flex-col">
    <label class="block mb-2 font-semibold text-gray-700">Estado *</label>
    <p-select
        formControlName="status"
        [options]="statusOptions"
        placeholder="Seleccionar estado"
        class="w-full"
    ></p-select>
</div>
</div>

```

```

<div class="flex justify-center mt-8">
    <div class="bg-gray-50 rounded-lg p-4 flex gap-4 items-center">
        <button
            pButton
            type="submit"
            class="p-button bg-blue-500 hover:bg-blue-600 text-white"
            label="Actualizar"
            icon="pi pi-save"
            [loading]="loading"
            [disabled]="loading"
        ></button>
        <button
            pButton
            type="button"
            class="p-button p-button-secondary"
            label="Cancelar"
            icon="pi pi-times"
            (click)="cancelar()"
            [disabled]="loading"
        ></button>
    </div>
</div>
</form>

```

```

<div class="flex justify-center" *ngIf="loading">
    <i class="pi pi-spin pi-spinner" style="font-size: 2rem"></i>
</div>
</div>

```

```
</div>
```

```
<p-toast></p-toast>
```